

Application Audit and Operations Review

Momentum Electrical Contractors

Application Audit and Operations Review

Prepared for review as one immutable publication.

Report date 2026-08-01

Classification Internal

Table of Contents

[Momentum Electrical Contractors: Final Engagement Report](#)

[Appendix A - Consolidated Finding Register](#)

[Appendix B1 - Intake Application Findings](#)

[Appendix B2 - Production Application Findings](#)

[Appendix B3 - Scheduling Application Findings](#)

[Appendix C1 - GitHub and Release Process Findings](#)

[Appendix C2 - Vercel Configuration and Operations Findings](#)

[Appendix C3 - Supabase and Data Security Findings](#)

[Appendix D - Implementation Roadmap](#)

[Appendix E - Shared Identity and Permission Design](#)

[Appendix F - Operating Playbook](#)

[Appendix F1- Code Organization Guide](#)

[Appendix G - Scope, Decisions, and Method](#)

[Client Decisions and Verification Register](#)

Acceptance candidate

Momentum Electrical Contractors: Final Engagement Report

Source: E12 Final Engagement Report **Status:** Client review draft **Last updated:** 2026-07-31
Assessment type: Read only review

Introduction

Momentum Electrical Contractors engaged us to conduct an independent review of its three internal applications: Intake, Production, and Scheduling, and the GitHub, Vercel, and Supabase services that support them. The engagement focused on application health, security, release practices, platform ownership, recovery, operations, and a future approach to individual user access.

To prepare these reports, we reviewed application source code and dependencies, performed safe checks of selected live routes, collected redacted read only platform records, and gathered input from Momentum about its people, processes, priorities, and recovery needs. We then compared the source, platform configuration, and live releases to identify confirmed strengths, material risks, and areas that still require validation.

The reports give Momentum a clear view of its current position and a practical path forward. Their goal is to help the company protect its data and applications, strengthen company control and recovery, improve how changes reach Production, and prioritize future work without disrupting the current business or application roadmap.

1. Executive decision brief

Momentum has three useful internal applications: Intake, Production, and Scheduling. The applications support important office and field work. They also hold customer, contract, pricing, employee, voice, and scheduling data.

The review found no evidence that a major incident is active now. It did find several paths that can cause one. We also found useful controls that should stay in place. The repositories and live Vercel projects are company owned. The repositories are private. Current live releases match the expected company source revisions. Supabase Row Level Security is enabled on the reviewed business tables. The deployed Intake schema contains the expected final state from its local migrations.

The main risk is not one isolated bug. Several controls depend on one shared boundary or one person. Stale libraries introduce vulnerabilities. Some protected handlers trust middleware alone while using broad database access. Vercel is configured to provide the

same privileged settings to Preview and Production deployments, although no active company Preview deployment was observed in the reviewed sample. Shared passwords limit accountability. Database changes cannot be rebuilt from a clear migration record. Restore ability has not been shown. The reviewed company release path has no required pull request, automated check, or isolated Preview gate. The audits also found specific bugs that can expose files, alter history, omit records, send duplicate messages, reopen closed jobs, or report wrong values.

We propose a staged response. Start with urgent containment and company control. Then add a safe release path, recovery checks, and monitoring. Build individual sign-in and permission controls only after the platform base is stable. Fix the application findings in small groups that do not conflict with the client’s Intake and Production roadmap.

2. What was assessed

Area	Review focus	Detail
Intake	Access, files, sales, documents, database use, tests, and runtime safety	Appendix B
Production	Field and PM access, progress, voice logs, AI use, reporting, and data integrity	Appendix B
Scheduling	Assignments, Intake sync, SMS, dates, database use, and runtime safety	Appendix B
GitHub and Vercel	Ownership, source control, releases, Preview, checks, recovery, and access	Appendix C
Supabase	Migrations, database access, RLS, keys, backups, Storage, and MFA	Appendix C
Security	One register for all 90 source findings and candidates	Appendix A
Operations	Work packages, order, cost choices, and proof of completion	Appendix D
Identity	Individual users, permissions, Backoffice, and migration	Appendix E

Discovery records the approved scope, people, URLs, data types, and open decisions. The source audits record the method, evidence, limits, and detailed findings. No secret value or customer record is included in this report.

3. What we found

Shared causes

Root cause	Business risk	Representative sources
Shared access and weak permission checks	One user cannot be removed alone. Sensitive actions may reach the database without a second check.	E02-F02-F03, E03-F02-F03, E03-F14, E04-F02-F03
Preview and Production are not separated	Test code can receive live credentials or reach live data and providers.	E02-F13, E03-F15, E04-F15, E06-F03-F04
The release path lacks required proof	A change can reach Production without a recorded review, passing checks, or a tested rollback point.	E05-F02-F05, E05-F08, E06-F02, E06-F05-F06
Database change and recovery records are weak	A failed change or data loss event may be hard to repair or explain.	E02-F04, E03-F04, E04-F13-F14, E07-F01-F03, E07-F05
Sensitive data and secrets need tighter care	Logs, files, keys, and retained records can expose business or personal data.	E02-F06, E02-F10, E03-F09, E04-F08, E04-F16, E07-F04, E07-F06
Tests and operating signals are limited	Wrong results, failed sync, missed messages, or access faults may reach users before an owner knows.	E02-F07, E03-F10, E03-F13, E04-F17, E04-F19, E06-F05
Platform ownership and recovery need proof	Loss of one account could delay release, recovery, or incident action.	E05-F01, E05-F06-F07, E06-F01, E06-F07, E06-F09, E07-F07

Appendix A maps every source ID to one of these causes or to a specific workflow problem. Appendix B and Appendix C hold the evidence and advice once, at the source.

Material application defects

Intake

A static file route can escape its intended folder. Missing access settings fail open. Some multi record writes can stop halfway. Sales totals can be wrong. Read requests can change document links. CSRF checks can fail open.

Source IDs: E02-F01-F02, E02-F05, E02-F15, E02-F18-F20

Production

Runtime flaws affect the access boundary. Passwords are stored in long lived cookies. Job and voice log access is too broad. History can be reassigned or deleted. Totals can omit records or mix units. Interrupted voice logs cannot resume safely.

Source IDs: E03-F01-F08, E03-F14, E03-F17-F20

Scheduling

Assignment state is not enforced. Editing one assignment can overwrite job dates. Sync can accept stale data or reopen a closed job. SMS sends can repeat. Date defaults can choose the wrong work day. Some reads can omit records.

Source IDs: E04-F04-F07, E04-F09-F12, E04-F20-F21

The detailed cards keep severity, timing, effort, evidence, impact, advice, limits, and verification steps. No bug card was removed.

What this means for Momentum

The applications can keep serving the business while remediation work is done. The review does not call for a rewrite or a pause on the full client roadmap, but proposes an order of work.

The proposal keeps the current frameworks, private repositories, company live projects, existing RLS base, and Stephanie's practical release role. The change is that each important step gains a clear owner and proof.

4. Recommended response

Phase 1 Contain urgent exposure

Outcome: Known access, runtime, credential, logging, and workflow faults are closed or bounded. Company recovery access is tested.

Work packages: OP01-OP02

Accept when: Focused tests pass. Credentials and access have named owners. Tom can use the approved recovery path.

Phase 2 Control change and release

Outcome: Every change has a pull request record. Required checks run. Stephanie promotes the exact tested revision. Preview is separate from Production.

Work packages: OP03-OP05

Accept when: Protected branch rules, passing checks, Preview proof, release record, smoke results, and a rollback point exist.

Phase 3 Make data and operations recoverable

Outcome: Database changes can be rebuilt. Backup scope is known. A restore and alert path are tested.

Work packages: OP06-OP08

Accept when: A clean database build, migration check, restore report, working alerts, and short runbooks are accepted.

Phase 4 Prove the identity design

Outcome: The chosen provider can supply one stable user identity to both business databases and reject the wrong issuer, app, environment, role, and record.

Work package: OP09

Accept when: Positive and negative identity and RLS tests pass. Momentum approves the provider choice.

Phase 5 Migrate users and applications

Outcome: Users receive individual sign-in, approved app access, direct permission checks, and action records. Shared passwords are retired.

Work packages: OP10-OP14

Accept when: All seven users pass access tests. Shared access is off. Recovery, revocation, and audit tests pass.

Phase 6 Finish application repair

Outcome: Remaining Intake, Production, and Scheduling bugs are fixed in small groups that fit the client roadmap.

Work packages: OP15-OP17

Accept when: Each selected source finding has tests and a closeout decision. Unselected risk has an owner and written decision.

Momentum can approve the phases separately. OP15-OP17 should be split when a smaller group of findings gives faster value or avoids conflict with current feature work.

5. Approval and immediate actions

Approve urgent containment and company source controls (OP01-OP02)

- **Owner:** Tom
- **Trigger:** Before more high risk changes
- **Immediate action:** Confirm owners and change windows. Fix urgent access and workflow faults.
- **Completion evidence:** Signed scope, test results, access record, and recovery check.

Approve the company release path

- **Owner:** Tom and Stephanie
- **Trigger:** With OP03-OP05
- **Immediate action:** Make the company repository the release source. Add required checks and separate Preview.
- **Completion evidence:** Protected branch, passing sample release, and rollback record.

Set recovery goals and retention rules

- **Owner:** Tom, Stephanie, and data owner
- **Trigger:** Before OP07
- **Immediate action:** Confirm data loss, service recovery, record retention, and Storage needs.
- **Completion evidence:** Approved recovery targets and restore test plan.

Confirm user roles and field access

- **Owner:** Tom and Stephanie
- **Trigger:** Before OP09
- **Immediate action:** Approve app access, job visibility, MFA, session, recovery, and administrator rules.
- **Completion evidence:** Final access matrix and identity decision record.

Select the first group of application findings to fix (OP15-OP17)

- **Owner:** Stephanie with Tom
- **Trigger:** After urgent containment
- **Immediate action:** Select findings that do not conflict with the Intake first roadmap.
- **Completion evidence:** Approved OP15-OP17 tranche with source IDs.

Close remaining platform evidence

- **Owner:** Stephanie
- **Trigger:** Before E05 and E06 are final
- **Immediate action:** Complete dashboard, owner, access, release, and recovery checks.
- **Completion evidence:** Updated source audits and final candidate status.

No paid add-on, credential change, access removal, or Production change should occur without separate approval. Exact effort and cost should be set after each package's prerequisites are closed.

6. Report map

Start with Appendix A for the full risk register and Appendix D for the delivery choice. Use Appendices B and C when reviewing evidence or a specific bug. Appendix E explains the identity design. Appendix F is the short operating playbook and code guide. Appendix G holds scope, confirmed facts, open decisions, and method.

Appendix	Source documents	Main reader
A Consolidated Finding Register	E08 - Security Assessment	Owners and implementers
B Application Findings	E02 - Intake , E03 - Production , and E04 - Scheduling	Application owner and developer
C Platform Findings	E05 - GitHub and Deployment , E06 - Vercel , and E07 - Supabase	Platform owner and implementer

Appendix	Source documents	Main reader
D Implementation Roadmap	E11 - Operations Proposal	Decision makers and implementers
E Identity Architecture	E10 - Authentication Architecture	Decision makers and identity implementer
F Operating Playbook	E09 - Best Practices and code appendix	Operators and developers
G Scope, Decisions, and Method	E01 - Discovery and client decisions	Decision makers and reviewers

Appendix A - Consolidated Finding Register

Source: E08 - Consolidated Security Assessment **Status:** Client review draft; E05 and E06 entries are provisional **Last updated:** 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This appendix is the single index for all 90 findings and candidates in E02-E07. Tom, Stephanie, and each implementation owner should use it to choose work, track decisions, and confirm that no finding is lost.

The source audits remain the authority for evidence, impact, advice, limits, and verification. This register does not copy those details. It groups related findings by cause and maps each source ID to an owner and E11 work package.

The register supports three decisions:

- which urgent work to approve;
- which work package owns each source finding; and
- whether each finding will be fixed, accepted, transferred, or closed.

Register rules

- **Confirmed** means source or collected platform evidence supports the finding within the limits in its source audit.
- **Provisional** means E05 or E06 still needs owner, history, or dashboard checks before its wording and rating are final.
- **Pending evidence** means the current review records a check to complete, not a proven weakness.
- **Open** means Momentum has not yet accepted proof that the finding is fixed or has chosen another disposition.
- Effort uses the S, M, and L scale defined in the source audits. E11 uses larger package sizes when several findings must be handled together.

Current source totals are 30 High, 53 Medium, six Low, and one pending evidence. Timing is 33 Immediate, 47 Near term, and 10 Planned. The register contains all 90 source IDs: 20 Intake, 20 Production, 24 Scheduling, 10 GitHub, nine Vercel, and seven Supabase.

Positive controls to preserve

- The application repositories and live Vercel projects are private and company owned.
- Current live Vercel revisions match the expected company repository revisions.
- Supabase Row Level Security is enabled on the reviewed business tables.
- The deployed Intake schema contains the expected final state from its local migrations.
- Vercel protects generated deployment URLs and stores environment values as sensitive records.
- GitHub workflow tokens default to read only, and private fork workflows do not receive secrets or write tokens.
- Some state changing Intake and Scheduling SMS requests check request origin.
- Stephanie reviews changes in testing before release and discusses material changes with Tom. The retained proof for that process still needs review.

Consolidated root causes

Cause and impact

ID	Root cause and outcome	Scope	Rating
E08-F01	Shared access and missing individual permissions. Give each person an account, limit app and record access, and record material actions.	All applications	High; Immediate
E08-F02	Route enforcement and privileged backend boundaries. Make every sensitive operation check identity, permission, and record access before broad database or provider use.	All applications	High; Immediate
E08-F03	Preview and Production separation. Use separate credentials, data, providers, and release paths.	Vercel and all applications	High; Immediate
E08-F04	Vulnerable runtimes and dependencies. Patch known flaws and enforce dependency checks.	Production and Scheduling	High; Immediate
E08-F05	Credential bearing destinations and integration trust. Allow only approved Supabase and provider destinations.	All applications	Medium; Immediate
E08-F06	Database change, access, backup, and recovery. Record migrations, narrow access, enforce SSL, and prove restore.	Both Supabase projects and all applications	High; Immediate
E08-F07	Workflow integrity and safe data handling. Fix partial writes, wrong totals, stale sync, unsafe state changes, message replay, date faults, and maintainability risks.	All applications	High; Immediate

ID	Root cause and outcome	Scope	Rating
E08-F08	Sensitive artifacts, logging, privacy, and retention. Remove exposed artifacts, limit logs, control keys, and set record life cycles.	Repositories, applications, and Supabase	High; Immediate
E08-F09	Browser, response, and session hardening. Add safe headers, private caching rules, CSRF protection, and secure temporary sessions.	All applications	Medium; Near term
E08-F10	Tests, monitoring, rollback, and incident readiness. Block unsafe releases and give failures a clear owner and response.	Applications, GitHub, and Vercel	High; Immediate
E08-F11	Platform ownership, access, and recovery control. Confirm company custody, MFA, access need, cost controls, and emergency recovery.	GitHub, Vercel, and Supabase	High; Immediate
E08-F12	Source control, release gates, and software supply chain. Use the company source, protected change records, required checks, and controlled automation.	GitHub and Vercel	High; Immediate

Ownership and response

ID	Owner	Main packages	Status
E08-F01	Application and identity owners	OP10-OP14	Open
E08-F02	Application owners	OP01, OP11-OP13	Open
E08-F03	Vercel and application owners	OP01, OP05	Open; includes provisional E06 entries
E08-F04	Application owners	OP01, OP04	Open
E08-F05	Application owners	OP01, OP05	Open
E08-F06	Data and Supabase owners	OP06-OP07	Open
E08-F07	Application owners	OP15-OP17	Open; includes provisional E06-F08
E08-F08	Security and data owners	OP01, OP06-OP08, OP15-OP17	Open
E08-F09	Application owners	OP01, OP04, OP15-OP17	Open
E08-F10	Release and operations owners	OP04-OP08	Open; includes provisional E06 entries
E08-F11	Platform owners	OP01-OP02, OP05, OP08	Open; includes provisional and pending entries
E08-F12	GitHub and release owners	OP02-OP05	Open; E05 entries are provisional

Complete source finding register

Each response cell identifies the owner, package, and disposition. All entries remain open until one of the closeout events below is accepted.

E02 - Intake

ID	Finding	Rating and evidence	E08 root cause	Response
E02-F01	Static file path traversal permits arbitrary file reads	High; Immediate; S - Confirmed	E08-F02	Intake owner - OP01, OP04; Open
E02-F02	Missing authentication configuration fails open	High; Immediate; S - Confirmed	E08-F02	Intake owner - OP01, OP04; Open
E02-F03	Shared authentication grants undifferentiated full access	High; Near term; L - Confirmed	E08-F01	Intake owner - OP13, OP14; Open
E02-F04	Deployed schema is current, but migration state is not reproducible or tracked	Medium; Near term; M - Confirmed	E08-F06	Intake owner - OP06; Open
E02-F05	Multi record workflows are not transactional	High; Near term; L - Confirmed	E08-F07	Intake owner - OP15; Open
E02-F06	Local database and log artifacts are committed	High; Immediate; M - Confirmed	E08-F08	Intake owner - OP01, OP02, OP04; Open
E02-F07	Business critical behavior has no automated regression suite	Medium; Near term; M - Confirmed	E08-F10	Intake owner - OP04; Open
E02-F08	Application structure has outgrown the single module design	Medium; Planned; L - Confirmed	E08-F07	Intake owner - OP15, OP08; Open
E02-F09	Human readable identifier allocation is race prone	Medium; Near term; M - Confirmed	E08-F07	Intake owner - OP15, OP08; Open
E02-F10	Logging and client errors can disclose excessive operational data	Medium; Near term; M - Confirmed	E08-F08	Intake owner - OP15, OP08; Open
E02-F11	Application responses lack explicit security and privacy headers	Medium; Near term; S - Confirmed	E08-F09	Intake owner - OP15, OP08; Open
E02-F12	Runtime version and operational health contract are undefined	Low; Planned; S - Confirmed	E08-F07	Intake owner - OP15, OP08; Open

ID	Finding	Rating and evidence	E08 root cause	Response
E02-F13	Preview and Production share privileged configuration	High; Immediate; M - Confirmed	E08-F03	Intake owner - OP01, OP05; Open
E02-F14	Credential bearing outbound URLs are not validated	Medium; Immediate; S - Confirmed	E08-F05	Intake owner - OP01, OP05; Open
E02-F15	Read endpoints can mutate document relationships	Medium; Near term; S - Confirmed	E08-F07	Intake owner - OP15, OP04; Open
E02-F16	A duplicate helper definition removes intended null handling	Medium; Near term; S - Confirmed	E08-F07	Intake owner - OP15, OP04; Open
E02-F17	Collection and report endpoints have no application pagination	Medium; Planned; M - Confirmed	E08-F07	Intake owner - OP15, OP04; Open
E02-F18	Sales reporting can silently produce inaccurate results	Medium; Near term; M - Confirmed	E08-F07	Intake owner - OP15, OP04; Open
E02-F19	Sales and document workflows bypass shared data validation	Medium; Near term; M - Confirmed	E08-F07	Intake owner - OP15, OP04; Open
E02-F20	State changing request CSRF validation fails open	Medium; Near term; S - Confirmed	E08-F09	Intake owner - OP15, OP04; Open

E03 - Production

ID	Finding	Rating and evidence	E08 root cause	Response
E03-F01	Vulnerable runtime dependencies affect the sole authorization boundary	High; Immediate; S - Confirmed	E08-F04	Production owner - OP01, OP12, OP14; Open
E03-F02	Shared passwords are used directly as long lived session cookies	High; Immediate; L - Confirmed	E08-F01	Production owner - OP01, OP12, OP14; Open
E03-F03	Field access lacks user, job, and voice log authorization boundaries	High; Near term; L - Confirmed	E08-F01	Production owner - OP12; Open
E03-F04	Deployed database and migration state are not reproducible or verified	High; Immediate; M - Confirmed	E08-F06	Production owner - OP01, OP06; Open
E03-F05	Historical productivity records can be reassigned or cascade deleted	High; Near term; M - Confirmed	E08-F07	Production owner - OP01, OP16; Open
E03-F06	Daily progress writes are not attributable or idempotent	Medium; Near term; M - Confirmed	E08-F07	Production owner - OP16, OP07; Open
E03-F07	Unpaginated aggregation can silently understate productivity totals	Medium; Near term; M - Confirmed	E08-F07	Production owner - OP16, OP07; Open
E03-F08	Voice upload and AI processing do not match platform reliability limits	Medium; Near term; M - Confirmed	E08-F07	Production owner - OP16, OP07; Open
E03-F09	Voice recordings and derived data have no retention or deletion life cycle	Medium; Near term; M - Confirmed	E08-F08	Production owner - OP16, OP07; Open

ID	Finding	Rating and evidence	E08 root cause	Response
E03-F10	Business critical and security behavior has no automated test or CI coverage	Medium; Near term; M - Confirmed	E08-F10	Production owner - OP04, OP16; Open
E03-F11	Application responses lack explicit browser security headers	Medium; Near term; S - Confirmed	E08-F09	Production owner - OP04, OP16; Open
E03-F12	Large client modules increase change and regression risk	Medium; Planned; L - Confirmed	E08-F07	Production owner - OP16, OP08; Open
E03-F13	Health, logging, and operational telemetry are minimal	Low; Planned; M - Confirmed	E08-F10	Production owner - OP16, OP08; Open
E03-F14	Sensitive handlers and Server Actions rely only on Proxy authorization	High; Immediate; M - Confirmed	E08-F02	Production owner - OP01, OP05; Open
E03-F15	Preview and Production share privileged configuration	High; Immediate; M - Confirmed	E08-F03	Production owner - OP01, OP05; Open
E03-F16	The credential bearing Supabase destination is not validated	Medium; Immediate; S - Confirmed	E08-F05	Production owner - OP01, OP05; Open
E03-F17	Interrupted daily voice logs cannot resume safely	Medium; Near term; M - Confirmed	E08-F07	Production owner - OP16, OP04; Open
E03-F18	Productivity metrics combine incompatible measurement units	Medium; Near term; M - Confirmed	E08-F07	Production owner - OP16, OP04; Open
E03-F19	AI summaries lack content integrity and human review controls	Medium; Near term; M - Confirmed	E08-F07	Production owner - OP16, OP04; Open
E03-F20	Identifier validation accepts unsafe JavaScript integers	Low; Planned; S - Confirmed	E08-F07	Production owner - OP16, OP04; Open

E04 - Scheduling

ID	Finding	Rating and evidence	E08 root cause	Response
E04-F01	Vulnerable production dependencies include the application runtime	High; Immediate; S - Confirmed	E08-F04	Scheduling owner - OP01, OP04; Open
E04-F02	Shared Basic Authentication grants undifferentiated full access	High; Near term; L - Confirmed	E08-F01	Scheduling owner - OP01, OP11, OP14; Open
E04-F03	Protected handlers rely only on middleware authorization	High; Immediate; M - Confirmed	E08-F02	Scheduling owner - OP01, OP11, OP14; Open
E04-F04	Assignment life cycle state is not enforced in operations	High; Immediate; M - Confirmed	E08-F07	Scheduling owner - OP01, OP17; Open
E04-F05	Outbound SMS sends are not safe to retry or trace	Medium; Near term; M - Confirmed	E08-F07	Scheduling owner - OP17, OP04, OP08; Open
E04-F06	Inbound SMS checks, resilience, and abuse controls are incomplete	Medium; Near term; S - Confirmed	E08-F07	Scheduling owner - OP17, OP04, OP08; Open
E04-F07	Intake sync accepts stale updates and uses a race prone upsert	Medium; Near term; M - Confirmed	E08-F07	Scheduling owner - OP17, OP04, OP08; Open
E04-F08	Integration and provider logging can disclose business and personal data	Medium; Immediate; S - Confirmed	E08-F08	Scheduling owner - OP01, OP08; Open
E04-F09	Editing one assignment overwrites the job's global dates	High; Near term; S - Confirmed	E08-F07	Scheduling owner - OP17; Open
E04-F10	Runtime request checks and schedule rules are not enforced on the server	Medium; Near term; M - Confirmed	E08-F07	Scheduling owner - OP17; Open

ID	Finding	Rating and evidence	E08 root cause	Response
E04-F11	Multi step writes are not transactional and deletes can erase history	High; Near term; M - Confirmed	E08-F07	Scheduling owner - OP17; Open
E04-F12	Unpaginated reads can omit schedules, messages, and report data	Medium; Near term; M - Confirmed	E08-F07	Scheduling owner - OP17; Open
E04-F13	Deployed schema and migration state cannot be rebuilt from source	Medium; Near term; M - Confirmed	E08-F06	Scheduling owner - OP01, OP06; Open
E04-F14	The bootstrap schema defines broad role RLS policies	Medium; Immediate; S - Confirmed	E08-F06	Scheduling owner - OP01, OP06; Open
E04-F15	Preview and Production share privileged configuration	High; Immediate; M - Confirmed	E08-F03	Scheduling owner - OP01, OP05; Open
E04-F16	Schedule and SMS records have no retention or deletion life cycle	Medium; Near term; M - Confirmed	E08-F08	Scheduling owner - OP17, OP04, OP08; Open
E04-F17	Business critical and security behavior has no automated test or CI coverage	Medium; Near term; M - Confirmed	E08-F10	Scheduling owner - OP17, OP04, OP08; Open
E04-F18	Large browser and route modules increase change and regression risk	Medium; Planned; L - Confirmed	E08-F07	Scheduling owner - OP17, OP04, OP08; Open
E04-F19	Health, messaging data, and alerts are minimal	Low; Planned; M - Confirmed	E08-F10	Scheduling owner - OP17, OP04, OP08; Open
E04-F20	Intake sync reopens closed Scheduling jobs	Medium; Immediate; S - Confirmed	E08-F07	Scheduling owner - OP01, OP17; Open

ID	Finding	Rating and evidence	E08 root cause	Response
E04-F21	Hard coded and UTC defaults can select the wrong work date	Medium; Immediate; S - Confirmed	E08-F07	Scheduling owner - OP01, OP17; Open
E04-F22	The credential bearing Supabase destination is not validated	Medium; Immediate; S - Confirmed	E08-F05	Scheduling owner - OP01, OP17; Open
E04-F23	Signed in responses lack explicit CSP and private no store controls	Medium; Near term; S - Confirmed	E08-F09	Scheduling owner - OP17, OP04; Open
E04-F24	Repository ignore rules and tracked CLI state increase disclosure risk	Low; Immediate; S - Confirmed	E08-F07	Scheduling owner - OP17, OP04; Open

E05 - GitHub and release process

ID	Finding	Rating and evidence	E08 root cause	Response
E05-F01	Organization MFA enforcement and access recovery controls are not established	High; Immediate; S - Provisional	E08-F11	GitHub owner - OP02; Open
E05-F02	Unprotected main branches deploy directly to Production without a controlled promotion gate	High; Immediate; M - Provisional	E08-F12	GitHub owner - OP02, OP04, OP05; Open
E05-F03	Production changes have no required build, test, security, or deployment checks	High; Immediate; M - Provisional	E08-F12	GitHub owner - OP02, OP04, OP05; Open
E05-F04	The reported personal Development and QA to company source and database promotion process is not documented or auditable	High; Immediate; M - Provisional	E08-F12	GitHub owner - OP02, OP04, OP05; Open
E05-F05	Secret scanning, push protection, dependency alerts, and code scanning are not enabled	High; Immediate; M - Provisional	E08-F12	GitHub owner - OP01, OP04; Open
E05-F06	Vercel GitHub App permissions are broad and exact selected repository access is unverified	Medium; Near term; S - Provisional	E08-F11	GitHub owner - OP02; Open
E05-F07	Repository access, outside collaborator need, access change handling, and annual review are not shown	Medium; Near term; S - Provisional	E08-F11	GitHub owner - OP02; Open

ID	Finding	Rating and evidence	E08 root cause	Response
E05-F08	Thirty two Production redeployments lack a release, operator, reason, rollback, or emergency change link	Medium; Near term; M - Provisional	E08-F12	GitHub owner - OP05, OP08; Open
E05-F09	Commit authenticity is not verified through signatures or an equal source proof	Low; Planned; M - Provisional	E08-F12	GitHub owner - OP02; Open
E05-F10	Future Actions policy permits all actions without fixed commit references	Medium; Near term; S - Provisional	E08-F12	GitHub owner - OP04; Open

E06 - Vercel

ID	Finding	Rating and evidence	E08 root cause	Response
E06-F01	Vercel ownership and recovery depend on one confirmed team Owner	High; Immediate; S - Provisional	E08-F11	Vercel owner - OP02; Open
E06-F02	Changes to main deploy to Production without controlled promotion or blocking checks	High; Immediate; M - Provisional	E08-F10	Vercel owner - OP01, OP05; Open
E06-F03	Preview and Production share privileged credentials and downstream systems	High; Immediate; M - Provisional	E08-F03	Vercel owner - OP01, OP05; Open
E06-F04	No isolated company Preview or staging release path is shown	Medium; Near term; M - Provisional	E08-F03	Vercel owner - OP01, OP05; Open
E06-F05	Monitoring, alerts, and deployment checks are not shown in operations	Medium; Near term; M - Provisional	E08-F10	Vercel owner - OP05, OP08; Open
E06-F06	Rollback artifacts exist, but no procedure has been shown	Medium; Near term; S - Provisional	E08-F10	Vercel owner - OP05, OP08; Open
E06-F07	Ownership for URLs, DNS, and generated deployments is not defined	Medium; Near term; S - Provisional	E08-F11	Vercel owner - OP05, OP08; Open
E06-F08	Scheduling functions and their Supabase database run in different regions	Medium; Planned; M - Provisional	E08-F07	Vercel owner - OP08; Open
E06-F09	Dashboard security, token, spend, and incident controls remain unverified	Pending evidence; Immediate; S - Pending evidence	E08-F11	Vercel owner - OP05, OP08; Open

E07 - Supabase

ID	Finding	Rating and evidence	E08 root cause	Response
E07-F01	Database change history is not recorded or reproducible	Medium; Near term; M - Confirmed	E08-F06	Supabase owner - OP06; Open
E07-F02	Backup and recovery ability cannot be shown	High; Immediate; M - Confirmed	E08-F06	Supabase owner - OP07; Open
E07-F03	Direct database entry is unrestricted and SSL is not enforced	Medium; Near term; M - Confirmed	E08-F06	Supabase owner - OP06; Open
E07-F04	Legacy API key use and custody are not documented	Medium; Near term; M - Confirmed	E08-F08	Supabase owner - OP01, OP06; Open
E07-F05	Unneeded broad Data API table privileges remain	Medium; Near term; S - Confirmed	E08-F06	Supabase owner - OP06; Open
E07-F06	Unused Supabase Auth features retain development defaults	Medium; Immediate; S - Confirmed	E08-F08	Supabase owner - OP01, OP09, OP10; Open
E07-F07	Supabase admin access includes accounts without MFA	High; Immediate; S - Confirmed	E08-F11	Supabase owner - OP01, OP02; Open

How to close an entry

The owner must update the disposition only after one of these events:

- **Resolved:** the source audit verification step passes and proof is retained.
- **Accepted:** Tom or the named business owner accepts the remaining risk and review date in writing.
- **Transferred:** another named owner and tracked package accepts the work.
- **Superseded:** stronger evidence changes or removes the finding. The source audit must explain why.

If E05 or E06 evidence changes a candidate's title, rating, or scope, update that source audit first. Then update this register and E11 in the same review.

Evidence report checks

This register contains all 90 source findings. The current internal E08 evidence packet does not include E05. It also leaves out E06-F08 without recording a reason. The packet must not be used as proof of full coverage until this is fixed.

The evidence process must:

- read E02 through E07, including E05;
- map every source ID or record why it is left out;
- compare the source count with the 90 entries in this register and stop when they differ;
- record the source file, source time, and sample size used by each report;
- keep every file named latest current and give old files a date; and
- test source coverage, report counts, source times, and file permissions.

Limits

This was a point in time, mainly read only assessment. It did not include a full penetration test, destructive testing, a Production restore, secret values, or customer data. E05 still needs full history review and owner checks. E06 still needs dashboard checks for access, tokens, firewall, spend, billing, logs, and incident settings. E07 still needs broader schema and runtime access tests plus an approved restore exercise.

Appendix B1 - Intake Application Findings

Source: E02 - Intake Application Audit **Status:** Client review draft **Last updated:** 2026-07-31
Entry document: [Final Engagement Report](#)

Purpose and reader

This appendix holds the full Intake finding cards. Stephanie, the Intake owner, and the implementation owner should use it when reviewing evidence or closing a source ID. Appendix A is the index. E11 maps each finding to work.

Assessment summary

Intake has useful business coverage and a simple runtime, but several controls depend on one shared access gate and one large server module. The most urgent findings are the unsafe static file path, access that fails open when settings are missing, committed database and log files, shared Preview and Production settings, and unvalidated destinations that receive credentials.

Other findings affect partial writes, ID allocation, Sales totals, document links, data checks, CSRF, pages that stop at a row limit, response headers, logs, health checks, and maintainability. The deployed Intake URL challenged unauthenticated requests, Vercel rejected tested traversal requests, and the deployed database contains the expected final state. Those controls limit current exposure but do not fix the source paths.

Scope and method

The review covered the recorded Intake source revision, Python and browser structure, access, routes, Supabase use, migrations, reports, documents, sync, settings, logs, errors, headers, dependencies, and tests. It used source review, syntax checks, focused local probes, safe unauthenticated live checks, and read only Vercel and Supabase evidence. No secret value, customer record, signed in workflow, or destructive Production action was used.

Effort uses S for a bounded change, M for several files or parts, and L for broad work that should be split.

Positive controls

- The reviewed live URL required access.
- Vercel rejected the tested traversal forms before they reached the app.
- The deployed database contains the expected final Intake schema effects.
- RLS is enabled on the reviewed Intake tables.
- Errors usually avoid returning raw stack traces.
- Some state changing requests check origin.

Findings summary

Effort ratings use relative T-shirt sizes:

- **S - Small:** bounded change in one primary area with limited coordination
- **M - Medium:** change across several files or parts that needs meaningful tests and review
- **L - Large:** broad remediation or coordination that should be planned as several tasks

ID	Finding	Severity	Timing	Effort
E02-F01	Static file path traversal permits arbitrary file reads	High	Immediate	S
E02-F02	Missing authentication configuration fails open	High	Immediate	S
E02-F03	Shared authentication grants undifferentiated full access	High	Near term	L
E02-F04	Deployed schema is current, but migration state is not reproducible or tracked	Medium	Near term	M
E02-F05	Multi record workflows are not transactional	High	Near term	L
E02-F06	Local database and log artifacts are committed	High	Immediate	M
E02-F07	Business critical behavior has no automated regression suite	Medium	Near term	M
E02-F08	Application structure has outgrown the single module design	Medium	Planned	L
E02-F09	Human readable identifier allocation is race prone	Medium	Near term	M

ID	Finding	Severity	Timing	Effort
E02-F10	Logging and client errors can disclose excessive operational data	Medium	Near term	M
E02-F11	Application responses lack explicit security and privacy headers	Medium	Near term	S
E02-F12	Runtime version and operational health contract are undefined	Low	Planned	S
E02-F13	Preview and Production share privileged configuration	High	Immediate	M
E02-F14	Credential bearing outbound URLs are not validated	Medium	Immediate	S
E02-F15	Read endpoints can mutate document relationships	Medium	Near term	S
E02-F16	A duplicate helper definition removes intended null handling	Medium	Near term	S
E02-F17	Collection and report endpoints have no application pagination	Medium	Planned	M
E02-F18	Sales reporting can silently produce inaccurate results	Medium	Near term	M
E02-F19	Sales and document workflows bypass shared data validation	Medium	Near term	M
E02-F20	State changing request CSRF validation fails open	Medium	Near term	S

Detailed findings

E02-F01 - Static file path traversal permits arbitrary file reads

Severity: High **Timing:** Immediate **Effort:** S **Recommended owner:** Intake application maintainer

Evidence

- `intake_app/app.py:4912-4925` joins the untrusted request path to `STATIC_PATH` without resolving the result and verifying that it remains below the static directory.
- A direct local WSGI probe for `/../app.py` returned the application source.
- Direct probes for `/../intake.sqlite3` and `/../server.err.log` returned the tracked 86,016-byte database and 7,514-byte error log with 200 OK.

- A deeper traversal reached a file outside the Intake repository.
- Authentication is evaluated before static serving, but E02-F02 allows that gate to disappear after a configuration mistake.
- Vercel rejected the tested traversal variants with 400; deployed exploitation was not demonstrated.

Impact

Any proxy or WSGI host that passes an unnormalized traversal path can expose source, local environment files, logs, database artifacts, or neighboring readable files. An authenticated shared user could also use the defect to retrieve server side material that the application was never intended to expose.

Recommendation

- Resolve the candidate path and verify it is a descendant of `STATIC_PATH`.
- Prefer an explicit allowlist of deployable static filenames or a proven static file component.
- Return 404 for any path containing traversal or resolving outside the static root.
- Add raw and encoded traversal regression tests.
- Review access logs after remediation and rotate credentials only if evidence indicates sensitive-file access.

E02-F02 - Missing authentication configuration fails open

Severity: High **Timing:** Immediate **Effort:** S **Recommended owner:** Intake application maintainer with Vercel owner

Evidence

- `intake_app/app.py:313-319` returns no configured credentials unless both username and password are present.
- `intake_app/app.py:336-344` treats the absence of configured credentials as authenticated.
- `intake_app/README.md:30-34` explicitly documents that the application is public when both variables are absent.
- All API and static routes depend on this single gate at `intake_app/app.py:4932-4936`.
- The API-confirmed Vercel URL currently returns 401, so the observed deployment has the credentials configured.

Impact

A missing, misspelled, incorrectly scoped, or partially migrated environment variable can silently expose all application data and mutating API operations. Because the server uses a Supabase service role credential, application exposure also provides a path to privileged database operations.

Recommendation

- Fail startup or return 503 when authentication configuration is absent outside an explicit local development mode.
- Require an explicit `APP_ENV=development`-style switch for any unauthenticated local mode.
- Add a deployment smoke test asserting that unauthenticated root and API requests return 401.
- Verify variables independently in Preview and Production during E06.

E02-F03 - Shared authentication grants undifferentiated full access

Severity: High **Timing:** Near term **Effort:** L **Recommended owner:** Identity implementation owner defined in E10

Evidence

- Discovery records shared office credentials and no individual attribution.
- The application implements one Basic Authentication username and password.
- There are no user, role, permission, session, invitation, recovery, or audit history models.
- The same identity can read financial information and perform create, update, conversion, settings, import, synchronization, and deletion operations.
- Basic Authentication provides no application level login throttling, lockout, session expiry, logout, or second factor.

Impact

The application cannot apply least privilege, attribute changes, disable one user, distinguish accidental from malicious activity, or reliably investigate an incident. A single disclosed password grants the complete application capability set.

Recommendation

- Use the target individual identity and authorization architecture from E10.
- Define at least office administrator, estimator/project manager, sales, and read only permission sets based on confirmed business needs.
- Require individual accounts, offboarding, recovery, and audit attribution.

- Until migration, rotate shared credentials, restrict their distribution, and use different values per environment.
- Add edge or identity layer rate limiting immediately; require MFA in the target identity design.

E02-F04 - Deployed schema is current, but migration state is not reproducible or tracked

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Intake application maintainer with Supabase owner

Evidence

- `intake_app/README.md:5-28` instructs a fresh project to run `schema.sql`.
- `schema.sql` does not contain the complete effects of later migrations, including sales stage history, marketing campaigns, later reporting fields, document ownership fields, remaining-profit forecast fields, or migration 035 access hardening.
- The README's migration inventory ends at migration 014, while the repository contains migrations through 039.
- Migration execution is manual; no migration ledger, deployment command, or automated empty database verification is present.
- Application code contains compatibility probes for columns that may or may not exist, confirming that multiple schema states are expected in practice.
- The focused read only check confirmed all 19 expected Intake tables, required final state columns, selected defaults, and ownership indexes in the deployed shared project.
- RLS is enabled on all 19 tables, and no direct grants or policies targeting `PUBLIC`, `anon`, or `authenticated` were found.
- The Supabase Management API returned zero remote migration ledger entries, so the exact execution path and order cannot be demonstrated.

Impact

The deployed Intake schema is not presently missing the checked final state controls. The remaining risk is reconstruction and future drift: a fresh environment created from the documented instructions can fail at runtime or produce different behavior, and operators cannot reliably prove which changes ran or reproduce the deployed baseline.

Recommendation

- Select one canonical schema-evolution process, preferably ordered, versioned migrations with a migration ledger.

- Make fresh setup apply the complete migration chain or regenerate a canonical schema from it.
- Add an automated empty database build and application smoke test.
- Wrap data migrations with several statements in transactions where supported. Add checks before and after each migration.
- Update the README and remove compatibility fallbacks after all controlled environments converge.
- Reconcile the observed deployed baseline once, then record all future changes through the canonical migration process.

E02-F05 - Multi record workflows are not transactional

Severity: High **Timing:** Near term **Effort:** L **Recommended owner:** Intake application maintainer with database owner

Evidence

- Supabase operations are independent HTTP requests from `supabase_request`; the application has no transaction boundary.
- `create_intake` creates up to six related entities and contacts before inserting the intake.
- `replace_contacts` deletes all existing contacts before inserting replacements.
- `save_quick_estimate_for_source` saves the estimate, deletes all lines, then inserts replacement lines.
- `save_quick_estimate_for_source` validates individual replacement lines only after deleting all existing lines, so one invalid line can erase the prior line set before the request fails.
- `convert_sales_lead_to_project` creates an intake, reassigns documents, may create a proposal, updates the lead, and records history in separate calls.
- `update_sales_lead` performs a second, redundant confidence only `PATCH` after the complete lead update. Failure of that second call reports an error even though the first update is already committed.
- `delete_intake` deletes contacts before deleting the intake, then attempts to delete related entities. A foreign key failure after the first step can leave committed data loss.

Impact

Timeouts, validation errors, uniqueness failures, foreign key restrictions, or upstream interruptions can leave orphaned records, missing contacts or estimate lines, duplicate conversions, or incomplete deletions. Retrying may not safely restore the intended state.

Recommendation

- Move compound workflows into database functions or another server side transaction boundary.
- Make conversions and integration operations idempotent with database enforced keys.
- Replace delete then insert operations with transactional replacement.
- Add ownership/reference checks before deleting shared clients, sites, or contacts.
- Add failure injection tests for every step of create, update, conversion, estimate save, and delete.

E02-F06 - Local database and log artifacts are committed

Severity: High **Timing:** Immediate **Effort:** M **Recommended owner:** Repository owner with data owner

Evidence

- `intake.sqlite3`, `server.err.log`, and `server.out.log` are tracked by Git.
- The SQLite file is a valid populated database with two intake rows and related entity rows.
- The tracked error log contains local HTTP request history.
- E02-F01's local traversal probe returned the tracked database and error log through the application's static file handler. Vercel's ignore and edge behavior reduce this combined path on the observed deployment but do not protect another bundle or WSGI host that includes the files.
- `.gitignore` and `.vercelignore` list these artifacts, but ignore rules do not remove files already tracked or erase history.
- The targeted current tree scan did not find obvious token shaped secret values.

Impact

Customer, contact, project, or operational data may be distributed to every repository clone and retained in Git history. Future log or database changes can also be committed accidentally because the files remain tracked.

The High rating is conservative until the data owner confirms whether the two records are real, synthetic, or already public.

Recommendation

- Treat the database as potentially sensitive until the data owner classifies it.
- Preserve any required evidence securely, then remove the database and logs from tracking.
- Assess Git history and clone distribution before deciding whether history rewriting is required.

- Use generated fixtures containing synthetic data for local development.
- Add CI checks that reject database, log, environment, and common secret artifacts.
- Perform a dedicated full history secret and sensitive-data scan in E05.

E02-F07 - Business critical behavior has no automated regression suite

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Intake application maintainer

Evidence

- No automated test files, test configuration, or CI workflows were found.
- The application includes financial calculations, schema compatibility branches, multi step conversions, document generation state, CSV imports, synchronization logic, and 64 router branches.
- Syntax checks passed, but syntax does not validate business behavior or data integrity.

Impact

Changes can silently alter estimates, profit reporting, contracts, conversions, imports, security gates, or synchronization behavior. Manual testing alone is unlikely to cover failure paths and schema variants consistently.

Recommendation

Start with a compact, risk based suite:

1. authentication, CSRF/origin, content-type, body size, and static-path security tests;
2. stored-XSS fixtures across Intake, Accounts, Sales, Estimates, Proposals, Contracts, and reporting views, asserting that payloads remain inert;
3. financial calculation unit tests using approved business examples;
4. API validation and not found tests;
5. transaction/failure path integration tests for compound writes;
6. migration tests from an empty database and from the current production baseline; and
7. one non production smoke workflow for create, edit, estimate, convert, synchronize, and delete.

Run the suite and syntax checks on every pull request in E05.

E02-F08 - Application structure has outgrown the single module design

Severity: Medium **Timing:** Planned **Effort:** L **Recommended owner:** Intake application maintainer

Evidence

- `app.py` contains 5,116 lines and 262 functions.
- The 180-line router uses 64 conditional branches.
- Authentication, transport, validation, persistence, financial calculations, reporting, sales, document workflows, synchronization, and static serving share one module.
- Browser code repeats request/error and HTML-escaping patterns across separate page scripts.
- The README API and migration inventories no longer describe the implemented application.
- Static analysis also found a function name redefinition and broad exception handlers, illustrating the collision and review risk in the shared module.

Impact

Unrelated changes have a wide review surface, ownership is unclear, reuse is difficult, and regression risk increases. The shape also makes focused testing and future role-based authorization harder to introduce.

Recommendation

- Refactor incrementally around business domains rather than rewriting the application.
- First extract request/response security, Supabase access, and routing.
- Then separate intake, sales, estimating, contracts/change orders, reporting, and Scheduling integration services.
- Centralize shared browser request and safe-rendering helpers.
- Keep the README focused on supported setup, architecture, migration, and verification commands.

E02-F09 - Human readable identifier allocation is race prone

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Intake application maintainer with database owner

Evidence

- `next_intake_number` reads the latest number, increments it in Python, and later inserts the record.

- Concurrent requests can calculate the same value; the database uniqueness constraint causes one request to fail after related records may already have been created.
- Marketing campaign and change order numbering use similar read-maximum/increment patterns.

Impact

Concurrent users or retries can produce failed saves, orphaned related records, confusing duplicate number errors, or inconsistent user-visible sequences.

Recommendation

- Allocate identifiers atomically in Postgres using a sequence, counter row with locking, or a transaction scoped database function.
- Keep uniqueness constraints as the final safeguard.
- Define whether gaps are acceptable and avoid deriving business meaning from gap free numbering.
- Add concurrent create tests.

E02-F10 - Logging and client errors can disclose excessive operational data

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Intake application maintainer with operations owner

Evidence

- Scheduling debug logging prints up to 1,000 characters of response and error bodies.
- Supabase HTTP error details are incorporated into runtime exceptions and returned to authenticated clients after only configured secret substitution.
- Logging is performed with `print` and has no level, request identifier, retention rule, or structured event schema.
- Unexpected exceptions are hidden from clients but are not logged by the application.
- Shared authentication prevents reliable actor attribution.

Impact

Logs and client visible errors may contain customer/job payloads, upstream implementation details, identifiers, or database constraint information. At the same time, generic unlogged failures can be difficult to diagnose, and user actions cannot be reconstructed.

Recommendation

- Log structured event metadata rather than upstream bodies.
- Introduce request/correlation IDs and explicit log levels.
- Return stable public error codes and keep sanitized diagnostic detail server side.
- Define log destination, access, retention, and redaction tests.
- Add user and action attribution when E10 is implemented.

E02-F11 - Application responses lack explicit security and privacy headers

Severity: Medium **Timing:** Near term **Effort:** S **Recommended owner:** Intake application maintainer with Vercel owner

Evidence

- Application generated JSON, text, static, and authentication responses set only their immediate content headers.
- The unauthenticated deployed responses included HSTS from Vercel but did not include an application Content Security Policy, `X-Content-Type-Options`, frame restrictions, or a referrer policy.
- The deployed authentication responses used `Cache-Control: public, max-age=0, must-revalidate` rather than an explicit `no-store` policy.
- The 21 browser scripts contain 53 `innerHTML` assignments and repeat hand written HTML escaping helpers across pages. The focused sink review found no `eval`, `document.write`, or confirmed unescaped database to script execution path, so this is defense in depth and regression evidence rather than a confirmed XSS vulnerability.

Impact

Sensitive pages and API responses rely on platform defaults for caching and browser protections. Missing headers increase exposure to framing, content-type confusion, referrer leakage, and the impact of a future injection defect.

Recommendation

- Set `Cache-Control: no-store` for authenticated HTML and API responses.
- Add `X-Content-Type-Options: nosniff`, an appropriate frame policy, and `Referrer-Policy`.
- Replace data bearing `innerHTML` use with `textContent`, form field `.value`, or explicit DOM construction where practical, and centralize the remaining context specific encoders.

- Introduce and test a Content Security Policy compatible with the Google Maps integration, beginning in report only mode. Evaluate Trusted Types after the unsafe sink surface has been reduced.
- Configure these controls consistently at the application or Vercel layer and verify authenticated responses in E06.

E02-F12 - Runtime version and operational health contract are undefined

Severity: Low **Timing:** Planned **Effort:** S **Recommended owner:** Intake application maintainer with Vercel owner

Evidence

- The project has no Python-version declaration.
- `requirements.txt` intentionally contains no packages, which minimizes dependency risk but does not pin interpreter behavior.
- There is no dedicated health/readiness endpoint or startup validation of required configuration.
- Operational checks currently depend on exercising normal routes.

Impact

Platform runtime changes can alter behavior without a source change, and operators cannot distinguish application health from authenticated business route behavior using a minimal check.

Recommendation

- Declare and test the supported Python version.
- Add a health endpoint that discloses no secrets or business data and checks only appropriate readiness dependencies.
- Validate required configuration at startup.
- Record the smoke-test and rollback contract in E06 and E09.

E02-F13 - Preview and Production share privileged configuration

Severity: High **Timing:** Immediate **Effort:** M **Recommended owner:** Vercel owner with Intake and Supabase owners

Evidence

- Vercel has one environment variable record targeting both `preview` and `production` for each of `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SCHEDULING_APP_API_URL`,

`SCHEDULING_EXTERNAL_API_KEY`, the Basic Authentication credentials, and other application settings.

- There are no Intake environment variable records scoped to `development`.
- The same Git repository and service role application code are used for production and preview deployments.

Impact

Preview code can receive the same privileged database and Scheduling access as Production. A feature branch, stale preview, or unreviewed change can therefore read or modify live business data, trigger external synchronization, or expose the shared credentials. Shared Basic Authentication also means Preview does not have an independent access boundary.

Recommendation

- Determine which Supabase project and Scheduling endpoint the shared records currently address before creating or exercising more previews.
- Create separate non production database and Scheduling resources with synthetic or approved sanitized data.
- Use distinct service, integration, and access credentials for Preview and Production; scope each Vercel record to one environment.
- Disable side-effecting external integrations by default in Preview.
- Rotate shared credentials after separation and add an automated deployment check that rejects cross environment endpoints or keys.

E02-F14 - Credential bearing outbound URLs are not validated

Severity: Medium **Timing:** Immediate **Effort:** S **Recommended owner:** Intake application maintainer with Vercel owner

Evidence

- `SUPABASE_URL` is used directly at `intake_app/app.py:359–408`; the service role key is then sent as both an API key and bearer token.
- `SCHEDULING_APP_API_URL` is used directly at `intake_app/app.py:411–415` and `984–1032`; the Scheduling bearer credential is sent to the resulting URL.
- Neither path enforces HTTPS or verifies that the host is an approved Supabase or Scheduling destination.
- Bandit reported both `urlopen` calls as medium severity, high confidence URL-scheme findings.

- A targeted in memory 302 redirect probe confirmed that Python's default handler retains both `Authorization` and `apikey` when redirecting the credential bearing request to a different host.
- If `INTAKE_APP_BASE_URL` is absent, `intake_app/app.py:431-438` builds a record URL from the request `Host` and forwards it to Scheduling. The current Vercel inventory shows that variable is configured, but its value was not inspected.

Impact

A configuration error or redirect from an otherwise approved endpoint can send privileged credentials or job data over an insecure connection or to the wrong service. A person able to alter configuration without reading write only sensitive values could redirect requests and capture those credentials. The `Host` fallback can also place an attacker selected Intake link into Scheduling when the trusted base URL is missing.

Recommendation

- Validate required outbound configuration at startup.
- Require `https` and exact approved hostnames for Supabase and Scheduling.
- Reject embedded credentials, fragments, unexpected ports, and non-HTTP schemes.
- Require a configured canonical Intake base URL outside local development; do not derive externally shared URLs from the request host.
- Disable redirects for credential bearing requests or validate the scheme and hostname of every redirect hop before issuing a new request.
- Add configuration and redirect regression tests.

E02-F15 - Read endpoints can mutate document relationships

Severity: Medium **Timing:** Near term **Effort:** S **Recommended owner:** Intake application maintainer

Evidence

- GET routes for Intake quick estimates, proposals, and contracts call helpers with `attach_converted=True` at `intake_app/app.py:1981-1989, 2148-2168, and 4198-4208`.
- When a converted sales lead document has no `intake_id`, those helpers issue a database `PATCH` at `intake_app/app.py:1658-1665, 1734-1742, and 1813-1821`.
- The same origin check at `intake_app/app.py:4935-4936` applies only to `POST`, `PUT`, `PATCH`, and `DELETE`, not `GET`.

Impact

Opening or prefetching a nominally read only URL can change persistent relationships. GET requests may be retried, cached, prefetched, linked, or triggered cross-site; none should alter business data. This weakens auditability and bypasses the control intended for mutating requests.

Recommendation

- Make all GET handlers strictly read only.
- Attach converted documents during the explicit conversion transaction or through an idempotent POST/PATCH operation.
- Apply authorization and same origin protections to the explicit mutation.
- Add tests asserting that every GET route performs no writes.

E02-F16 - A duplicate helper definition removes intended null handling

Severity: Medium **Timing:** Near term **Effort:** S **Recommended owner:** Intake application maintainer

Evidence

- `site_address` is first defined at `intake_app/app.py:871-884` to accept `None` and return `None`.
- It is redefined at `intake_app/app.py:2380-2391` without the null guard; Python silently replaces the first definition at module load.
- The schema permits a null `intakes.site_id`, application intake validation does not require a site, and several callers pass `context.get("site")` or `intake.get("site")`.
- Ruff reports the redefinition as F811.

Impact

A valid intake without a linked site can raise an unexpected exception when document, change order, profit, or related views format its address. The application returns a generic 500, and the broad exception handler does not log the exception.

Recommendation

- Keep one address formatter that handles null values. Use explicit names if account and site formatting later diverge.
- Add tests for records with missing optional client/site relationships across every caller.
- Enable a small mandatory lint rule set, including redefinitions and undefined names, in CI.

E02-F17 - Collection and report endpoints have no application pagination

Severity: Medium **Timing:** Planned **Effort:** M **Recommended owner:** Intake application maintainer with database owner

Evidence

- Intake, account, sales lead, campaign, estimate review, and profit/report paths request full ordered collections without application `limit/offset` or cursor controls.
- Several endpoints hydrate related tables and calculate reports in Python after loading the collections.
- The Supabase inventory reports a PostgREST `max_rows` of 1,000, but application responses do not expose truncation, continuation, or a total count.

Impact

Response time, memory use, and database requests grow with business data. Once a query reaches the platform cap, screens and reports can silently omit older records and produce incomplete totals rather than clearly failing or offering another page.

Recommendation

- Add bounded cursor pagination and documented maximum page sizes to interactive collections.
- Move aggregate report calculations into tested SQL views or functions with explicit date ranges.
- Return continuation and truncation metadata; never present capped data as a complete report.
- Add volume tests above the configured PostgREST row cap.

E02-F18 - Sales reporting can silently produce inaccurate results

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Intake application maintainer with Sales/data owner

Evidence

- `apply_sales_outcome_values` at `intake_app/app.py:3942-3959` unconditionally replaces `lost_date` with today's date whenever a lead is saved as Lost. A targeted probe showed that an existing 2025-01-15 loss date became 2026-07-24.
- `record_sales_stage_history` at `intake_app/app.py:3914-3928` suppresses every `RuntimeError`. A simulated database failure returned normally, leaving the caller unable to distinguish a recorded transition from a lost audit event.

- `sales_stage_history_by_lead` at `intake_app/app.py:3370-3394` also suppresses database failures. When history is absent, `sales_report_reached_stage` at `intake_app/app.py:3397-3403` infers that a lead reached every ordinal stage before its current stage, even though the board permits direct stage changes.
- Migration 032 adds the immutable looking `first_won_at` field and the application writes it, but no report reads it. Estimate effectiveness instead classifies outcome from the current stage and mutable `won_date` at `intake_app/app.py:3584-3645`.
- `proposal_conversion_rate` at `intake_app/app.py:3759-3815` divides all proposals created in a period by all quick estimates created in that period without linking the proposal to the estimate. The value can exceed 100% and does not measure record level conversion.
- Open pipeline and stage breakdown calculations at `intake_app/app.py:3445-3523` include only leads created inside the selected period, while the interface labels describe current opportunities and open pipeline without that cohort qualification.
- `static/accounts.js:126-130` converts a missing monetary value to zero and contains a mojibake fallback, so absent data can be displayed as \$0 rather than unavailable.

Impact

Historical losses can move between reporting periods, stage conversion can be overstated, reopened opportunities can lose their original won outcome, and unrelated proposal/estimate cohorts can produce misleading conversion rates. Because history read and write failures are silent, users receive plausible reports with no indication that underlying evidence is incomplete. Sales, staffing, and revenue decisions should not rely on these metrics until their definitions and data lineage are corrected.

Recommendation

- Preserve explicit and first outcome timestamps; do not overwrite historical dates merely because the current stage is Won or Lost.
- Make transition history persistence part of the lead transaction and surface failures instead of silently substituting inferred history.
- Calculate reached stages only from durable transition facts, with an explicit migration rule for legacy leads that lack history.
- Define each report's cohort and label it accurately. Link conversion numerators and denominators by business record rather than creation period totals.
- Use `first_won_at` or another approved immutable outcome event for effectiveness metrics, and define how reopened and Intake only estimates are classified.
- Add fixture based report tests for direct stage jumps, reopen/re-win, missing history, prior-period open pipeline, and conversion rate boundaries.

E02-F19 - Sales and document workflows bypass shared data validation

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Intake application maintainer with Sales/data owner

Evidence

- `sales_lead_values` at `intake_app/app.py:3028-3091` validates projected start and finish dates independently but never requires the finish date to be on or after the start date.
- A targeted validation probe accepted 2026-08-10 as the projected start and 2026-08-01 as the projected finish.
- `convert_sales_lead_to_project` at `intake_app/app.py:4030-4117` copies those fields directly into a new Intake record and does not invoke the normal Intake date range validation before insertion.
- `save_contract_for_source` at `intake_app/app.py:4215-4276` likewise validates commencement and completion as individual dates but permits completion before commencement.
- Sales-specific client and site writers at `intake_app/app.py:2756-2897` normalize state and contact values but do not invoke the email and two-letter state validators used by the normal Intake path. Direct API calls can therefore persist values that the Intake workflow rejects.
- Floating point parsers at `intake_app/app.py:493-500` and `intake_app/app.py:4279-4289` do not reject not finite values such as NaN or infinity before those values reach JSON serialization, calculations, or a database request.

Impact

Impossible date ranges and inconsistent contact, location, or numeric values can become persistent Sales, Intake, and contract data. They can flow into scheduling, documents, calculations, and reports, causing inconsistent operational commitments, failed downstream requests, or incorrect timelines.

Recommendation

- Centralize field and relationship validation and reuse it across Intake, Sales, conversion, and contract workflows.
- Validate the complete converted Intake payload before its first database write.
- Add database check constraints for relationships that must hold regardless of the write path.
- Reject not finite numeric input and apply the same email, state, length, and cardinality rules to every write path.

- Add API tests for equal dates, reversed dates, omitted endpoints, and conversion of legacy invalid records, plus invalid contact/location and not finite numeric values.

E02-F20 - State changing request CSRF validation fails open

Severity: Medium **Timing:** Near term **Effort:** S **Recommended owner:** Intake application maintainer with identity implementation owner

Evidence

- `is_same_origin` at `intake_app/app.py:347-356` returns `True` immediately when the request has no `Origin` header.
- The helper compares only `urlparse(origin).netloc` with the request `Host`. It therefore accepted `http://intake.example` for a request whose application origin was `https://intake.example` in a targeted helper probe.
- The comparison trusts the request `Host` instead of matching the complete scheme, host, and port against a configured canonical origin allowlist.
- The mutation gate at `intake_app/app.py:4935-4936` has no CSRF token, validated `Referer` fallback, or `Fetch Metadata` policy.
- `read_json` at `intake_app/app.py:467-483` parses the body without requiring `Content-Type: application/json`, removing a useful restriction against browser simple-request content types.
- The application uses browser cached Basic Authentication credentials, so authorization is ambient rather than tied to a per request anti CSRF value.

Impact

A state changing request is accepted when `Origin` is absent even though the request carries an ambient shared credential. Current browsers commonly send `Origin` for direct cross-site form submissions. The expected JSON body also makes a simple attack harder. Redirects, clients outside a browser, privacy tools, later endpoint changes, or proxy behavior can remove those limits. The weakness applies to all POST, PUT, PATCH, and DELETE routes. E02-F15 separately covers writes hidden inside GET handlers.

Recommendation

- Compare the complete origin tuple-scheme, hostname, and port-against an explicit configured allowlist rather than the request `Host`.
- Reject mutations when both `Origin` and a validated same origin `Referer` are absent.
- Require `Content-Type: application/json` for JSON mutation endpoints and return `415 Unsupported Media Type` for other body types.
- Add an unpredictable CSRF token or authenticated custom request header.

- Enforce `Sec-Fetch-Site` and related Fetch Metadata headers as defense in depth, with explicit exceptions only for documented machine to machine routes.
- Add tests for absent, `null`, malformed, wrong-scheme, wrong-host, redirect, and valid same origin requests.

Closeout and limits

The review covers the recorded company source and the collected company Vercel and Supabase evidence. Stephanie's personal test source, data, settings, checks, and exact promotion path were not inspected. No signed in browser or API workflow was run. Sales cohort and metric rules still need owner approval. Supabase checks covered selected final state and access facts, not ordered migration execution or restore. The history scan was targeted, not a full Git object history scan. No Production state was changed.

Keep each finding open until its stated fix and a focused verification test pass. Use Appendix A for status and E11 for package order. A limitation or stronger result must be updated here before it changes the register.

Appendix B2 - Production Application Findings

Source: E03 - Production Application Audit **Status:** Client review draft **Last updated:** 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This appendix holds the full Production finding cards. Stephanie, the Production owner, and the implementation owner should use it when reviewing evidence or closing a source ID. Appendix A is the index. E11 maps each finding to work.

Assessment summary

Production supports important field, PM, progress, voice, and report work. Its most urgent risks come from vulnerable runtime packages at the access boundary, shared passwords stored in long lived cookies, broad job and voice access, weak migration proof, history changes, sensitive handlers that trust Proxy checks, shared Preview and Production settings, and an unvalidated Supabase destination.

Other findings affect retries, complete totals, voice upload and processing, retention, tests, headers, large client files, health signals, voice resume, mixed productivity units, AI review, and unsafe integer IDs. The reviewed live URL denied tested unauthenticated access, private Storage and RLS provide useful limits, and server held provider values were not exposed to the browser review. These controls do not replace direct operation and record checks.

Scope and method

The review covered the recorded Production source revision, Next.js routes and Server Actions, shared sessions, job and voice rules, Supabase and Storage, migrations, OpenAI work, reports, errors, headers, dependencies, tests, and operating signals. It used source review, build and type checks, focused local tests, safe live requests, and read only platform evidence. No secret value, customer record, signed in workflow, voice content, or destructive Production action was used.

Effort uses S for a bounded change, M for several files or parts, and L for broad work that should be split.

Positive controls

- The reviewed live URL denied tested unauthenticated protected requests.
- TypeScript strict checks and ESLint pass at the reviewed revision.
- The Storage bucket is private with file size and type limits.
- RLS is enabled on the reviewed Production tables.
- Server modules hold privileged values rather than browser code.
- Existing response helpers give a base for safe errors.

Findings summary

Effort ratings use relative T-shirt sizes:

- **S - Small:** bounded change in one primary area with limited coordination
- **M - Medium:** change across several files or parts that needs meaningful tests and review
- **L - Large:** broad remediation or coordination that should be planned as several tasks

ID	Finding	Severity	Timing	Effort
E03-F01	Vulnerable runtime dependencies affect the sole authorization boundary	High	Immediate	S
E03-F02	Shared passwords are used directly as long lived session cookies	High	Immediate	L
E03-F03	Field access lacks user, job, and voice log authorization boundaries	High	Near term	L
E03-F04	Deployed database and migration state are not reproducible or verified	High	Immediate	M
E03-F05	Historical productivity records can be reassigned or cascade deleted	High	Near term	M
E03-F06	Daily progress writes are not attributable or idempotent	Medium	Near term	M
E03-F07	Unpaginated aggregation can silently understate productivity totals	Medium	Near term	M
E03-F08	Voice upload and AI processing do not match platform reliability limits	Medium	Near term	M
E03-F09	Voice recordings and derived data have no retention or deletion lifecycle	Medium	Near term	M

ID	Finding	Severity	Timing	Effort
E03-F10	Business critical and security behavior has no automated test or CI coverage	Medium	Near term	M
E03-F11	Application responses lack explicit browser security headers	Medium	Near term	S
E03-F12	Large client modules increase change and regression risk	Medium	Planned	L
E03-F13	Health, logging, and operational telemetry are minimal	Low	Planned	M
E03-F14	Sensitive handlers and Server Actions rely exclusively on Proxy authorization	High	Immediate	M
E03-F15	Preview and Production share privileged configuration	High	Immediate	M
E03-F16	The credential bearing Supabase destination is not validated	Medium	Immediate	S
E03-F17	Interrupted daily voice logs cannot resume safely	Medium	Near term	M
E03-F18	Productivity metrics combine incompatible measurement units	Medium	Near term	M
E03-F19	AI summaries lack content integrity and human review controls	Medium	Near term	M
E03-F20	Identifier validation accepts unsafe JavaScript integers	Low	Planned	S

Detailed findings

E03-F01 - Vulnerable runtime dependencies affect the sole authorization boundary

Severity: High **Timing:** Immediate **Effort:** S **Recommended owner:** Production application maintainer with deployment owner

Evidence

- `package-lock.json` resolves Next.js 16.2.7.
- `npm audit --omit=dev` reported three high severity production dependency findings.

- [GHSA-m99w-x7hq-7vfj](#) affects App Router applications with at least one Server Action for Next.js 16.0.0 through 16.2.10; crafted unauthenticated requests can cause excessive CPU usage and block further requests in the same process.
- The application uses App Router Server Actions in `app/access/actions.ts` and `app/pm/daily-logs/actions.ts`, so the documented denial-of-service condition applies to the reviewed source. The advisory lists no workaround other than upgrading.
- [GHSA-6gpp-xcg3-4w24](#) is a separate Proxy bypass affecting App Router applications using Turbopack with a single entry in `config.i18n.locales` for versions 16.0.0 through 16.2.10.
- The production build uses Turbopack and `proxy.ts` is the sole authorization boundary, but `next.config.ts` does not configure `i18n.locales`. The reviewed source therefore does not establish the Proxy advisory's documented single-locale prerequisite.
- The same audit identifies a patched Next.js release, 16.2.11.
- `package.json` declares all runtime and development packages as "latest", so a normal dependency update is not constrained to a reviewed release line even though `npm ci` currently reproduces the lockfile.
- No attempt was made to exploit the denial of service or bypass the deployed Proxy.

Impact

The confirmed Server Actions condition allows an unauthenticated attacker to consume excessive application CPU and reduce or deny service. If the Proxy-bypass prerequisite is introduced through future locale configuration or a materially different deployed request path, an unauthenticated request could reach pages and APIs that use the Supabase service role credential. That would bypass both role passwords and expose job, productivity, financial, recording, transcript, or mutation capabilities. The remaining production dependency advisories add component risk that should be removed in the same upgrade.

Recommendation

- Upgrade and lock Next.js to at least the patched 16.2.11 release, update the lockfile, and redeploy.
- Run lint, TypeScript, build, dependency audit, and explicit unauthenticated route matrix tests before and after deployment.
- Verify the deployment is not susceptible to the published bypass without placing exploit details in client facing documents or logs.
- Replace "latest" declarations with reviewed compatible ranges and use controlled dependency update pull requests.
- Add a CI policy that blocks known critical/high runtime vulnerabilities unless an owner records a time-bound exception.

- Review the remaining PostCSS, Sharp, and development only findings for actual build/runtime reachability.

E03-F02 - Shared passwords are used directly as long lived session cookies

Severity: High **Timing:** Immediate **Effort:** L **Recommended owner:** Identity implementation owner defined in E10

Evidence

- `app/access/actions.ts:17-45` compares one shared Field or PM password and writes the same password value into a browser cookie.
- `proxy.ts:40-54` authorizes requests by comparing the cookie value directly to the current environment password.
- Cookies last 12 hours and there is no logout, server side session record, per device revocation, or individual user identity.
- Password comparisons use JavaScript string equality rather than a password verification primitive.
- No application rate limit, failed attempt tracking, lockout, or access alert is present.
- `getSafeNextPath` and the access page accept any value beginning with `/`, including a protocol relative `//host` value, as a post login destination.
- A local Production runtime check with synthetic credentials submitted `next=/\attacker.example`; the successful Server Action response emitted `Location: /\attacker.example`, which URL parsing resolves to the external `http://attacker.example/` origin.
- Discovery records one shared Field credential and one shared PM credential.

Impact

Any field holder, shared browser profile, browser compromise, cookie capture, or disclosed password grants the complete capability set of that role. The password itself is repeatedly presented as the session bearer value, increasing exposure and preventing independent session revocation. Actions cannot be attributed to a person, and one departure or disclosure requires rotating credentials for every user. The permissive redirect check can also send a successfully authenticated user to an external protocol relative destination.

Recommendation

- Implement the individual identity, session, MFA, recovery, and role model selected in E10.
- As an immediate containment step, issue a random opaque signed session value after password verification instead of storing the password itself.

- Add logout and server side or cryptographically verifiable expiration and revocation.
- Add rate limiting, failed attempt telemetry, and safe password verification.
- Accept only single-slash, same origin relative destinations from a small route allowlist.
- Rotate temporary role passwords after changing the cookie format and use different values per environment.

E03-F03 - Field access lacks user, job, and voice log authorization boundaries

Severity: High **Timing:** Near term **Effort:** L **Recommended owner:** Production application maintainer with identity owner

Evidence

- `app/field/page.tsx:13-24` returns up to 200 jobs with job numbers, names, stages, and site addresses to every Field session.
- `proxy.ts:13-18` grants every Field session access to the progress, voice-upload, and transcription APIs.
- `app/api/productivity/daily-voice-logs/transcribe/route.ts:10-23` accepts only a numeric voice log ID.
- `lib/daily-log-transcription.ts:40-60` loads that ID without checking a job assignment, creator, user, or requested job and returns an existing completed transcript.
- The same function can retry failed processing and incur an OpenAI request for any known or guessed voice log ID.
- `entered_by` for progress and voice logs is client controlled and currently sent as the literal "Field App".
- No user, crew, project assignment, or object permission model exists.

Impact

Possession of the Field password provides visibility across the shared job list and allows direct interaction with voice logs outside the current UI selection. A field user can enumerate identifiers, retrieve another log's completed transcript, or trigger processing costs. There is no reliable way to restrict technicians to assigned projects, identify who submitted a record, or investigate inappropriate access.

Recommendation

- Define Field, PM, office, and project assignment permissions through E10.
- Enforce user and job authorization inside every server rendered loader, Server Action, and API handler; do not rely only on path level Proxy checks.

- Bind voice log processing to a verified record the caller is allowed to access and avoid returning transcript text to Field clients unless required.
- Derive `entered_by` and audit identity from the authenticated session rather than request data.
- Add authorization tests covering every role, route, HTTP method, job assignment, and cross object identifier.
- Until individual identity exists, treat Field access as access to all displayed jobs and document that business decision explicitly.

E03-F04 - Deployed database and migration state are not reproducible or verified

Severity: High **Timing:** Immediate **Effort:** M **Recommended owner:** Production application maintainer with Supabase owner

Evidence

- The repository contains five ordered migrations, while the README instructs an operator to apply the additive migrations manually if they have not already been applied.
- The Supabase management inventory returned an empty migration history for the shared project.
- The cross project security check confirmed RLS and no policies on all five Production tables, but it did not validate every Production column, constraint, or trigger.
- The five tables retain effective `anon` and `authenticated` privileges; RLS currently blocks row access because no policy applies.
- Source migrations enable RLS but define no policies and do not explicitly revoke those broad privileges.
- `20260605110500_create_productivity_tracking_tables.sql:80-93`, `20260606193800_create_daily_voice_logs.sql:74-77`, and `20260606210000_create_daily_log_summaries.sql:42-45` create named triggers without `if not exists` handling, so rerunning a partially applied migration can fail.
- `20260606193800_create_daily_voice_logs.sql:4-12` uses `on conflict do nothing` for the bucket, which does not reconcile an existing bucket's privacy or limit settings.
- Production uses a service role key and therefore does not exercise normal RLS policies during application requests.

Impact

The deployed database can drift from source without a reliable record or repeatable reconstruction path. A missing constraint can permit duplicate or invalid business records;

incorrect grants or RLS can expose shared Intake and Production data through the Supabase Data API; and partial manual application can leave an environment that cannot be safely retried or reproduced.

Recommendation

- Adopt one versioned migration command and ledger for all controlled environments.
- Reconcile the remaining deployed schema, triggers, and constraints against source during E07; preserve the confirmed RLS/policy result and revoke unused broad privileges.
- Make migrations safe for the intended fresh, upgrade, and retry paths or explicitly mark and detect one time application.
- Add preflight and postflight assertions for every migration.
- Verify an empty database build and an upgrade from the deployed baseline in automation.
- Explicitly revoke unintended roles and test access using `anon`, authenticated user, and service role contexts.

E03-F05 - Historical productivity records can be reassigned or cascade deleted

Severity: High **Timing:** Near term **Effort:** M **Recommended owner:** Production application maintainer with database owner

Evidence

- `app/api/productivity/work-items/route.ts:108-148` allows a PM request to update a work item's `area_id` to any existing area.
- The update does not check whether the destination area belongs to the same job or whether the work item already has daily entries.
- Daily entries reference only `work_item_id`, so moving the work item to an area under another job also moves all of its historical progress into that job's reports.
- `app/api/productivity/areas/route.ts:96-129` and `work-items/route.ts:161-181` check for daily entries and then delete in separate database requests.
- `20260605110500_create_productivity_tracking_tables.sql:20-53` configures cascade deletes from areas to work items and from work items to daily entries.
- A field insert can occur after the application's existence check but before the PM delete, allowing the database cascade to remove the new history.

- `ProjectManagerSetup.tsx:360–391` and `510–565` save reorder operations as independent concurrent PATCH requests with no transaction; one failure can leave only part of an area or work item order applied.

Impact

A crafted PM request, future UI defect, concurrent Field save, or direct administrative delete can silently move or erase production history. Reports can then attribute work to the wrong project or understate completed quantities with no application audit trail or recovery workflow.

Recommendation

- Prevent moving a work item between jobs after creation; use an explicit reviewed migration workflow if reassignment is ever required.
- Put guarded deletes behind a database function or transaction that locks or atomically checks the relevant rows.
- Replace cascade deletion of historical daily entries with restrictive foreign keys, soft deletion, or an archive workflow aligned with retention requirements.
- Add immutable job ownership or denormalized job constraints where appropriate.
- Record who performed setup changes and destructive actions.
- Add concurrent insert/delete and cross job reassignment tests.

E03-F06 - Daily progress writes are not attributable or idempotent

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Production application maintainer with database owner

Evidence

- `app/api/productivity/field/route.ts:150–236` inserts a new row for every positive submitted quantity.
- The table has indexes but no request, device, user, work item/date, or other idempotency constraint.
- `FieldProgressApp.tsx:289–323` prevents repeat clicks only in the current browser state after a confirmed response.
- A timeout after a committed insert leaves the client unable to know whether retrying will duplicate progress.
- The API accepts an unbounded entries array within the platform request limit.
- The API does not reject repeated work item IDs in one entries array, so one request can count the same item more than once.
- `parseEntryDate` validates only a `YYYY-MM-DD` shape, and the API has no approved backdate/future date window.

- `entered_by` is accepted from the request and is not tied to an authenticated actor.

Impact

Retries, double submissions from different tabs or devices, or ambiguous network failures can overstate completed quantities. Operators cannot reliably determine whether similar entries are duplicates or valid additional work, or which technician submitted them.

Recommendation

- Define whether each save is an increment or a daily final value and encode that business rule in the database.
- Add a client generated idempotency key with a unique constraint, or use an atomic upsert for the approved daily value model.
- Derive the actor from the authenticated session and store request, actor, and timestamp audit metadata.
- Bound the number of entries per request and reject duplicate work item IDs in one payload.
- Provide a correction workflow that preserves the original entry and records the adjustment.
- Add tests for retries, duplicates, use from several devices, and unclear responses.

E03-F07 - Unpaginated aggregation can silently understate productivity totals

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Production application maintainer with Supabase owner

Evidence

- The Supabase management inventory reports a PostgreSQL `max_rows` value of 1,000 for the shared project.
- `app/api/productivity/field/route.ts:96-119` requests every matching daily entry and calculates field totals in application memory without pagination.
- `app/pm/reports/page.tsx:200-236` uses the same unpaginated pattern for report totals.
- `app/pm/reports/detail/page.tsx:140-167` loads all matching dated entries without pagination to build progress charts.
- Supabase can return the configured maximum number of rows successfully; the code does not request or verify an exact count and therefore cannot detect truncation.

Impact

Once a selected job's work items collectively exceed the API row limit, the field screen and PM reports can show internally consistent but incomplete totals. This is silent business data corruption at the presentation layer rather than an explicit failure.

Recommendation

- Move totals and chart aggregation into bounded database queries, views, or functions using `sum` and grouping.
- If row retrieval remains necessary, implement deterministic pagination and verify completion.
- Add a volume test with more than 1,000 daily entries.
- Add reconciliation checks comparing report totals with database aggregates.
- Review other fixed limits, including the 50/200 job lists and 100-record daily-log review, and add intentional pagination or lifecycle filtering.

E03-F08 - Voice upload and AI processing do not match platform reliability limits

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Production application maintainer with Vercel and OpenAI owners

Evidence

- `app/api/productivity/daily-voice-logs/route.ts:11-27` and the Storage bucket allow recordings up to 25 MiB.
- The route calls `request.formData()` before it can enforce the file size check.
- File acceptance trusts the client provided MIME type, and `getAudioExtension` accepts any alphanumeric filename extension without checking the file signature or ensuring the extension matches the declared audio type.
- [Vercel documents a 4.5 MB request body limit](<https://vercel.com/docs/functions/limitations#request> body size) for Vercel Functions; larger requests return 413 before the route's 25 MiB application limit can be useful.
- `FieldProgressApp.tsx:460-472` starts transcription as a second fire and forget browser request after upload.
- `lib/daily-log-transcription.ts:28-165` downloads the complete object and performs an OpenAI request synchronously in that invocation.
- Transcription and summary OpenAI requests have no application timeout or abort signal.
- Concurrent retries can both observe a not completed record, mark it processing, and submit duplicate OpenAI work.
- A recording remains saved when the client closes before starting the transcription request; retry then depends on a PM finding the failed or pending record.

Impact

Recordings between Vercel's request limit and the advertised 25 MiB limit fail before reaching application validation. Slow or interrupted requests can leave records pending or processing, duplicate external cost, consume function duration, or require manual recovery. Users receive an upload contract that the deployed platform cannot satisfy.

Recommendation

- Upload recordings directly to the private Storage bucket using a short lived, scoped signed upload flow issued by an authorized server endpoint.
- Enforce the intended object prefix, size, one answer constraint, media signature, and approved extension/type combination server side after upload.
- Move transcription and summary work to a durable background job or queue with idempotent state transitions.
- Use atomic claim/update logic so only one worker processes a record.
- Add explicit OpenAI timeouts, bounded retries with backoff, and a recoverable terminal state.
- Reconcile and document browser, Vercel, Storage, and OpenAI size limits.
- Add end to end tests for oversize audio, client disconnect, timeout, duplicate trigger, provider failure, and retry.

E03-F09 - Voice recordings and derived data have no retention or deletion lifecycle

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Data owner with Production and operations owners

Evidence

- Voice logs can include crew names, delays, safety concerns, site details, and employee voices.
- Recordings, transcripts, errors, and AI summaries are retained in Storage and database tables.
- No source route, documented procedure, retention interval, legal hold, deletion workflow, or orphan object reconciliation exists.
- The application sends recordings and transcript content to an external OpenAI API.
- E01 has not yet classified the data, confirmed employee notice or consent requirements, or recorded the OpenAI account owner and contractual data settings.
- Positive controls include a private bucket, service-side access, and ten-minute PM playback URLs.

Impact

Sensitive operational and employee data can accumulate indefinitely, remain after the corresponding business need ends, or become orphaned from metadata. The organization cannot consistently respond to retention, privacy, discovery, employee, or incident requirements until ownership and lifecycle rules are explicit.

Recommendation

- Classify recordings, transcripts, summaries, and job metadata with the business and data owners.
- Confirm employee notice/consent, approved use, OpenAI project ownership, contractual data controls, and access expectations.
- Define retention periods separately for audio, transcripts, summaries, errors, and audit history.
- Implement an authorized deletion/archive workflow that keeps Storage and database state consistent and records who approved it.
- Add periodic orphan detection and retention enforcement with a dry run/report mode.
- Minimize data sent to external AI services and avoid including information not required for the approved purpose.

E03-F10 - Business critical and security behavior has no automated test or CI coverage

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Production application maintainer with repository owner

Evidence

- No test files, test framework configuration, coverage configuration, or test package script was found.
- No GitHub Actions or other repository CI workflow was found.
- The only automated source commands are development, build, start, and lint.
- Authorization depends on a manually maintained path/method matrix in `proxy.ts`.
- The application performs productivity aggregation, destructive setup operations, upload/database compensation, external AI state transitions, and shared-database migrations.
- Lint, TypeScript, and build checks passed, but they do not validate role isolation, business totals, retries, failure paths, or data preservation.

Impact

Changes can silently expose PM routes, alter productivity totals, duplicate field work, lose historical records, break upload recovery, or drift schema behavior. The known vulnerable

dependency also demonstrates that no automated gate currently blocks a high severity runtime advisory.

Recommendation

Start with a compact risk based suite:

1. Proxy and handler authorization tests for every role, route, method, and missing config state;
2. idempotency, cross job ownership, guarded delete, and concurrent write integration tests;
3. aggregation tests above the PostgREST row ceiling;
4. migration tests from an empty database and the reconciled deployed baseline;
5. upload, cleanup, timeout, retry, and duplicate processing tests using provider fakes; and
6. one non production Field to PM smoke workflow.

Run lint, TypeScript, build, tests, dependency audit, and secret/artifact checks on every pull request in E05.

E03-F11 - Application responses lack explicit browser security headers

Severity: Medium **Timing:** Near term **Effort:** S **Recommended owner:** Production application maintainer with Vercel owner

Evidence

- `next.config.ts` defines only one development origin setting and no response headers.
- The deployed home, access, protected redirect, health, and denied API responses included Vercel HSTS.
- The tested responses did not include an application Content Security Policy, frame restrictions, `X-Content-Type-Options`, referrer policy, or permissions policy.
- A local Production runtime response emitted `X-Powered-By: Next.js`, disclosing the application framework.
- The public home, protected redirects, health response, and API denials used `Cache-Control: public, max-age=0, must-revalidate`; the access page used an explicit private/no-store policy.
- The application uses microphone access and displays customer/job, financial, productivity, transcript, and signed-audio information on protected pages.

Impact

Sensitive workflows rely on browser and platform defaults for framing, content-type handling, referrer disclosure, caching, and device capabilities. Missing defense in depth controls increase the impact of a future injection, navigation, or content handling defect.

Recommendation

- Add and test a Content Security Policy compatible with Next.js, Supabase signed media, and required runtime resources.
- Add `X-Content-Type-Options: nosniff`, an appropriate `frame-ancestors` or `frame` policy, and a strict referrer policy.
- Add a permissions policy that grants microphone access only as required by the Field workflow.
- Set `poweredByHeader: false` unless an operational dependency requires the framework disclosure header.
- Explicitly use `Cache-Control: private, no-store` for authenticated HTML, APIs, signed URLs, transcripts, and financial reports.
- Verify headers on both authenticated and unauthenticated responses in E06.

E03-F12 - Large client modules increase change and regression risk

Severity: Medium **Timing:** Planned **Effort:** L **Recommended owner:** Production application maintainer

Evidence

- `ProjectManagerSetup.tsx` contains 1,143 lines and owns job selection, area and work item creation/editing/deletion, ordering, request state, messages, and rendering.
- `FieldProgressApp.tsx` contains 786 lines and owns job selection, progress entry, aggregation drafts, microphone lifecycle, recording state, upload, transcription triggering, and two major interface modes.
- `globals.css` contains 1,428 lines for access, PM, Field, reports, daily logs, responsive layouts, and state variants.
- Supabase query and job display patterns are repeated across Field, Jobs, PM, reports, and daily-log pages.
- API request parsing and response behavior are hand built per route.

Impact

Unrelated changes share a broad review surface, state transitions are hard to test independently, and security or business rule changes must be repeated across loaders and handlers. This increases regression risk as individual authentication, job assignment, retention, and background processing are added.

Recommendation

- Refactor incrementally rather than rewriting.
- First extract access/session, authorized job loading, request validation, and shared response controls.
- Split Field progress and voice recording into independently testable components and hooks.
- Split PM setup by area, work item, ordering, and destructive action responsibilities.
- Centralize shared Supabase query services and job summary types.
- Organize CSS by stable application area or introduce the repository's preferred scoped styling convention.

E03-F13 - Health, logging, and operational telemetry are minimal

Severity: Low **Timing:** Planned **Effort:** M **Recommended owner:** Production application maintainer with operations owner

Evidence

- `/api/health` always returns a static application identifier and does not distinguish liveness from dependency readiness.
- Configuration is validated only when a Supabase client is first created.
- Logging uses `console.error` with no request ID, user/session identity, job identifier schema, event type, level policy, or documented retention/redaction rules.
- Transcription and summary states are stored, but there is no automatic stale-pending/processing detection, alert, queue metric, or provider latency/error metric.
- No application monitoring, error tracking, alert route, operational dashboard, or runbook is present in the source.

Impact

Operators can see that the process responds but cannot quickly distinguish configuration, database, Storage, OpenAI, backlog, or authorization failures. Failed workflows are discovered manually, and events cannot be correlated to a request or person.

Recommendation

- Keep a minimal public liveness endpoint and add an authenticated or platform only readiness check with safe dependency signals.
- Validate required production configuration during startup/build or deployment smoke checks.
- Use structured, redacted logging with request IDs, stable event names, and authenticated actor/job identifiers where appropriate.

- Define alerts for authentication anomalies, API error rates, stale processing records, upload cleanup failures, and OpenAI failures.
- Record log access, retention, redaction, incident, and recovery procedures in E09.

E03-F14 - Sensitive handlers and Server Actions rely exclusively on Proxy authorization

Severity: High **Timing:** Immediate **Effort:** M **Recommended owner:** Production application maintainer with identity owner

Evidence

- `proxy.ts:8-37` assigns roles by pathname, and `proxy.ts:64-106` is the only code that verifies a Field or PM cookie.
- None of the 12 exported API HTTP handlers verifies a session or role inside the handler before using the Supabase service role client.
- `app/pm/daily-logs/actions.ts:8-26` exports two PM-only Server Actions that invoke transcription and summary generation without verifying a PM session.
- The production build exposes those functions through the Next.js Server Action manifest.
- [Next.js authentication guidance](#) says Proxy checks are optional/optimistic, Route Handlers and Server Actions must be treated as public facing endpoints, and authorization must be verified inside each operation.
- Cookie authenticated mutation handlers perform no application level Origin/Host validation and use no CSRF token. `SameSite=Lax` reduces ordinary cross-site requests but does not distinguish origins on an untrusted sibling subdomain within the same registrable site.
- The Next.js advisories in E03-F01 demonstrate the consequence of relying on one framework interception layer.

Impact

A framework bypass, matcher regression, rewrite, alternate entry point, or future route added outside the role map can reach privileged code with no second authorization decision. Because every data operation uses a Supabase service role key, a missed Proxy check becomes full application level read or mutation authority rather than a restricted database user request. The PM Server Actions can also trigger external AI cost and alter stored transcript/summary state. If an untrusted sibling origin is present, same-site cookie semantics can also allow it to submit authenticated mutations even when browser CORS prevents that origin from reading the response.

Recommendation

- Create one server only authorization function that validates the session and required role.
- Call it at the start of every Route Handler, Server Action, and sensitive server rendered data loader.
- Perform job and object authorization after role verification and before every read, write, signed URL, or provider call.
- Keep Proxy checks as an early user-experience and traffic filter, not the security boundary.
- Enforce an allowlist of trusted Origin/Host combinations or a synchronizer-token/custom header CSRF control for cookie authenticated mutations in addition to SameSite; prefer SameSite=Strict where the approved workflow permits it.
- Add tests that invoke handlers and Server Actions directly, including paths and methods that do not pass through the expected UI.

E03-F15 - Preview and Production share privileged configuration

Severity: High **Timing:** Immediate **Effort:** M **Recommended owner:** Vercel owner with Production, Supabase, and OpenAI owners

Evidence

- Vercel has one environment variable record targeting both `preview` and `production` for each of `SUPABASE_SERVICE_ROLE_KEY`, `NEXT_PUBLIC_SUPABASE_URL`, `OPENAI_API_KEY`, `FIELD_ACCESS_PASSWORD`, and `PM_ACCESS_PASSWORD`.
- The same configuration pattern also includes the older unused `APP_ACCESS_PASSWORD`.
- No Production application variable records are scoped to `development`.
- Preview builds use the same service role application code and can write productivity data, upload recordings, create signed URLs, and trigger OpenAI transcription and summaries.
- Git fork protection is enabled, but an internal feature branch, stale preview, or unreviewed company repository change can still receive the shared records.

Impact

Preview code can receive Production equivalent database, Storage, AI, and role password authority. A feature branch or stale deployment can read or mutate live jobs and productivity history, expose recordings or transcripts, incur OpenAI cost, or disclose credentials that are also valid on Production. Testing in Preview is not safely isolated from operational data.

Recommendation

- Determine and document the actual destination of each shared record before creating or exercising more previews.
- Create separate non production Supabase and OpenAI resources with synthetic or approved sanitized data.
- Use distinct service, role password, and AI credentials for Preview and Production and scope each Vercel record to one environment.
- Disable recording upload and external AI side effects by default on untrusted previews.
- Remove the unused `APP_ACCESS_PASSWORD` after confirming no active deployment depends on it.
- Rotate shared credentials after separation and add a deployment check that rejects production project identifiers in Preview.

E03-F16 - The credential bearing Supabase destination is not validated

Severity: Medium **Timing:** Immediate **Effort:** S **Recommended owner:** Production application maintainer with Vercel and Supabase owners

Evidence

- `lib/env.ts:1-15` checks only whether `NEXT_PUBLIC_SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` are present.
- `lib/supabase-server.ts:4-15` passes the configured URL and service role key directly to the Supabase client.
- Every resulting database and Storage request sends privileged credential material to the configured destination.
- The code does not require the approved project host or compare it with an environment specific allowlist.
- Vercel stores the URL and service key as separately editable configuration records.

Impact

A configuration mistake can send privileged credentials and job related requests to the wrong project. A person or compromised automation able to change configuration without reading the write only service key can redirect the client to an attacker controlled compatible HTTPS endpoint and capture the credential.

Recommendation

- Parse the URL at startup and require HTTPS and the exact approved Supabase project host for each environment.
- Keep an environment to project allowlist in reviewed configuration without embedding secrets.

- Fail readiness before making a credential bearing request when the destination is invalid.
- Add tests for wrong schemes, lookalike hosts, unexpected ports, user information, and Preview/Production project crossover.
- Rotate the service role key if configuration history indicates it was ever sent to an unintended destination.

E03-F17 - Interrupted daily voice logs cannot resume safely

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Production application maintainer

Evidence

- `FieldProgressApp.tsx:132-140` tracks submitted questions only in browser component state.
- `FieldProgressApp.tsx:179-190` clears that state whenever the job workflow resets.
- The Field API has no read operation that returns already submitted voice log questions for the selected job and date.
- `FieldProgressApp.tsx:435-480` always resumes from the current local question index and advances only after a successful new upload response.
- `20260606193800_create_daily_voice_logs.sql:38-48` correctly prevents a second answer for the same job, date, and question.
- `app/api/productivity/daily-voice-logs/route.ts:122-136` therefore rejects a repeated first answer rather than returning the existing record or allowing the client to continue.
- The selected entry date is calculated once when the component mounts, so a field screen left open across local midnight continues saving to the prior date.

Impact

Refreshing, closing the browser, switching jobs, losing the upload response, or crossing midnight can strand an otherwise valid daily log. The worker returns to the first question, receives an "already submitted" error, and cannot advance to the remaining questions. This also prevents the automatic five question summary from completing and creates avoidable PM recovery work.

Recommendation

- Add an authorized read endpoint that returns question completion and processing state for a job/date.
- Hydrate the workflow from database state and start at the first unanswered question.

- Make upload retry idempotent by returning the existing matching record when the submitted content/request identity represents the same operation.
- Provide an explicit replace/correct workflow with audit history when an answer must change.
- Recompute and visibly confirm the work date before each save, with an approved rule for backdated entries.
- Add refresh, lost response, job switch, partial day, duplicate, and midnight boundary tests.

E03-F18 - Productivity metrics combine incompatible measurement units

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Production application maintainer with business owner

Evidence

- Work items carry free text units such as each, feet, or other measures.
- `app/pm/reports/ProductivityProgress.tsx:32-38` sums every selected item's need, completed, and remaining quantities without grouping or converting by unit.
- The resulting cards present a single "Total Need", "Total Completed", "Total Remaining", and percentage with no unit.
- The same invalid aggregate is calculated per area and phase for progress bars.
- `app/pm/reports/detail/page.tsx:47-70` also divides summed completed quantities by summed need quantities across all items in an area/phase.
- Report and chart percentages cap at 100%, which hides the magnitude of over completion.

Impact

Adding feet of conduit to counts of devices, panels, boxes, or other units produces totals and percentages with no mathematical meaning. PM dashboards can appear precise while overstating or understating real progress, and high volume unit types dominate the result. Capping also conceals quantity overruns that may indicate estimating, entry, or scope problems.

Recommendation

- Confirm the business definition of overall progress with the estimator and PM owners.
- Never sum incompatible units directly; group results by unit or use an approved normalized weighting model.

- Consider explicit per item weights, labor unit estimates, earned value, or milestone completion if one overall percentage is required.
- Show over completion and data quality exceptions rather than silently capping all displays.
- Constrain units to an approved catalog or dimensional model instead of unrestricted text.
- Add mixed unit, zero-need, over completion, and rounding acceptance tests using approved project examples.

E03-F19 - AI summaries lack content integrity and human review controls

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Production application maintainer with PM and data owners

Evidence

- `lib/daily-log-summary.ts:19-45` concatenates field controlled transcript text directly into the user prompt.
- A transcript can contain instructions that conflict with the summary request, whether spoken deliberately, transcribed incorrectly, or introduced as ordinary job language.
- `lib/daily-log-summary.ts:149-219` accepts the model response as unrestricted text and immediately stores it as `completed`.
- `app/pm/daily-logs/DailyLogsReview.tsx:96-110` and `app/pm/reports/page.tsx:445-456` display that text as a normal "Daily Summary".
- The record has no model, prompt version, source-question references, generated by label, reviewer, approval state, or correction history.
- The model has no tool or database authority and React escapes the displayed text, which limits this finding to report integrity rather than direct code execution.

Impact

The generated report can omit, distort, or add emphasis to work, materials, delays, or safety information while appearing authoritative. Prompt like speech and transcription errors can influence the result, and later reviewers cannot determine which model/prompt produced it or whether a PM approved it. Operational or safety decisions may rely on an unverified derivative rather than the source recordings and transcripts.

Recommendation

- Treat transcripts as untrusted quoted data and use a structured output schema with bounded sections.

- Preserve links from every summary statement to its source question/transcript where practical.
- Label summaries as AI-generated and require PM review/approval before they are treated as an official daily report.
- Store model, prompt version, generation time, reviewer, approval/correction history, and source record versions.
- Keep source recordings and transcripts accessible under the approved retention policy.
- Add adversarial transcript, transcription-error, missing answer, conflicting answer, and unsupported-claim tests.

E03-F20 - Identifier validation accepts unsafe JavaScript integers

Severity: Low **Timing:** Planned **Effort:** S **Recommended owner:** Production application maintainer

Evidence

- `lib/productivity.ts:60-68` converts identifier input with `Number(value)` and checks `Number.isInteger`, positive sign, and no upper bound.
- JavaScript numbers cannot represent every integer above `Number.MAX_SAFE_INTEGER` exactly, but values above that boundary can still satisfy `Number.isInteger`.
- The shared parser is used for job, area, work item, and voice log identifiers across Route Handlers and Server Actions.
- Supabase identity values and request/response types are represented as JavaScript `number`, so an out of range identifier can be rounded before a database filter is constructed.
- Sort orders and quantities have separate explicit bounds, which limits this issue primarily to identifiers.
- No current database identifier was observed near the unsafe-integer boundary, and no production request was made to test it.

Impact

A malformed request or future identifier growth beyond JavaScript's exact integer range can select, update, or report against a rounded identifier, or produce inconsistent validation and database behavior. The present likelihood is low because generated identifiers are expected to remain far below the boundary, but the shared parser makes the behavior systemic.

Recommendation

- Require `Number.isSafeInteger` for all identifiers represented as JavaScript numbers.
- Accept only canonical base 10 integer strings or integer JSON values. Reject exponents, hexadecimal values, whitespace only values, booleans, arrays, and values that need type coercion.
- Apply an approved domain or database upper bound in addition to the safe-integer check.
- If identifiers can exceed the safe range, carry them as validated decimal strings or `bigint` through the complete API and database boundary.
- Add parser and handler tests at 0, negative values, `Number.MAX_SAFE_INTEGER`, the first unsafe integer, exponent notation, and large database compatible values.

Closeout and limits

The review covers the recorded company source and collected company platform evidence. Stephanie's personal test source, data, settings, checks, and exact promotion path were not inspected. No signed in field or PM workflow, voice content, AI output, restore, or destructive action was tested. Database definitions and full record rules need further runtime proof. Provider logs and secret values were not collected. The history scan was targeted, not a full Git object history scan. No Production state was changed.

Keep each finding open until its stated fix and a focused verification test pass. Use Appendix A for status and E11 for package order. A limitation or stronger result must be updated here before it changes the register.

Appendix B3 - Scheduling

Application Findings

Source: E04 - Scheduling Application Audit **Status:** Client review draft **Last updated:** 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This appendix holds the full Scheduling finding cards. Stephanie, the Scheduling owner, and the implementation owner should use it when reviewing evidence or closing a source ID. Appendix A is the index. E11 maps each finding to work.

Assessment summary

Scheduling has useful early separation between browser, server, data, and shared code. Its urgent risks include vulnerable runtime packages, shared access, handlers that trust middleware, assignment state faults, logs that may expose data, broad bootstrap policies, shared Preview and Production settings, closed jobs reopened by Intake sync, wrong date defaults, and an unvalidated Supabase destination.

Other findings affect SMS retry and delivery records, inbound webhook safety, stale sync, job dates, server checks, partial writes, pages that stop at a row limit, migrations, retention, tests, large modules, alerts, headers, and repository files. The reviewed live URL denied unauthenticated requests, the deployed database is stricter than the unsafe bootstrap, and the code has reusable server and validation modules. Those controls do not fix the listed workflow faults.

Scope and method

The review covered the recorded Scheduling source revision, Next.js routes, board and report workflows, assignment state, Intake sync, Twilio, Supabase, migrations, dates, logs, headers, dependencies, tests, and operating signals. It used source review, build and type checks, focused local tests, safe live requests, and read only platform evidence. No secret value, employee message content, signed in workflow, live SMS, or destructive Production action was used.

Effort uses S for a bounded change, M for several files or parts, and L for broad work that should be split.

Positive controls

- The reviewed live URL denied tested unauthenticated protected requests.
- TypeScript strict checks and ESLint pass at the reviewed revision.
- Browser pages use a shared API client rather than direct Supabase access.
- Server data, shared validation, date, report, and SMS modules already exist.
- The deployed Scheduling tables use RLS and no broad active policies were found.
- Some SMS writes check origin and provider signatures.

Findings summary

Effort ratings use relative T-shirt sizes:

- **S - Small:** bounded change in one primary area with limited coordination
- **M - Medium:** change across several files or parts that needs meaningful tests and review
- **L - Large:** broad remediation or coordination that should be planned as several tasks

ID	Finding	Severity	Timing	Effort
E04-F01	Vulnerable production dependencies include the application runtime	High	Immediate	S
E04-F02	Shared Basic Authentication grants undifferentiated full access	High	Near term	L
E04-F03	Protected handlers rely exclusively on middleware authorization	High	Immediate	M
E04-F04	Assignment lifecycle state is not operationally enforced	High	Immediate	M
E04-F05	Outbound SMS sends are not idempotent or traceable through delivery	Medium	Near term	M
E04-F06	Inbound SMS authentication, resilience, and abuse controls are incomplete	Medium	Near term	S
E04-F07	Intake synchronization accepts stale updates and uses a race prone upsert	Medium	Near term	M
E04-F08	Integration and provider logging can disclose business and personal data	Medium	Immediate	S
E04-F09	Editing one assignment overwrites the job's global dates	High	Near term	S

ID	Finding	Severity	Timing	Effort
E04-F10	Runtime request validation and schedule rules are not enforced server side	Medium	Near term	M
E04-F11	Multi step writes are not transactional and deletes can erase history	High	Near term	M
E04-F12	Unpaginated reads can silently omit schedules, messages, and report data	Medium	Near term	M
E04-F13	Deployed schema and migration state are not reproducible from source	Medium	Near term	M
E04-F14	The bootstrap schema defines permissive broad role RLS policies	Medium	Immediate	S
E04-F15	Preview and Production share privileged configuration	High	Immediate	M
E04-F16	Schedule and SMS records have no retention or deletion lifecycle	Medium	Near term	M
E04-F17	Business critical and security behavior has no automated test or CI coverage	Medium	Near term	M
E04-F18	Large browser and route modules increase change and regression risk	Medium	Planned	L
E04-F19	Health, messaging telemetry, and operational alerting are minimal	Low	Planned	M
E04-F20	Intake synchronization reopens previously closed Scheduling jobs	Medium	Immediate	S
E04-F21	Hard coded and UTC defaults can select the wrong work date	Medium	Immediate	S
E04-F22	The credential bearing Supabase destination is not validated	Medium	Immediate	S
E04-F23	Authenticated responses lack explicit CSP and private no-store controls	Medium	Near term	S
E04-F24	Repository ignore rules and tracked CLI state increase disclosure risk	Low	Immediate	S

Detailed findings

E04-F01 - Vulnerable production dependencies include the application runtime

Severity: High **Timing:** Immediate **Effort:** S **Recommended owner:** Scheduling application maintainer

Evidence

- `scheduling_app/package.json` declares `Next.js ^15.0.3`; the lockfile resolved `Next.js 15.5.18`.
- `npm audit --omit=dev --json` reported high severity findings involving `Next.js`, `PostCSS 8.5.15`, and `Sharp 0.34.5`.
- The `Next.js` findings include denial-of-service, server side request forgery, cache confusion, unbounded payload, and internal Server Function disclosure advisories across different features and deployment modes.
- Full tree `npm audit` additionally reported high severity development tree findings in `brace-expansion` and `js-yaml`.
- The audit reported a fix available for every affected package.
- The application does not define Server Actions, custom server behavior, or rewrites, so those feature specific advisory paths were not confirmed as exploitable. `Next.js` remains the runtime and middleware authorization boundary.

Impact

The deployed application remains on versions with published high severity advisories. Some advisory paths are not used by this application, but the affected runtime still receives every protected and public request. Delaying updates also makes it harder to distinguish relevant runtime exposure from unused transitive functionality.

Recommendation

- Upgrade `Next.js` and the affected production dependency tree to supported patched versions through a reviewed lockfile change.
- Re-run clean install, lint, TypeScript, production build, `npm audit`, and unauthenticated route checks.
- Add authenticated authorization, synchronization, webhook, and SMS regression tests before deployment.
- Enable automated dependency update visibility and define an advisory triage owner through E05 and E09.
- Treat unused-feature analysis as prioritization evidence, not a reason to keep known vulnerable packages indefinitely.

E04-F02 - Shared Basic Authentication grants undifferentiated full access

Severity: High **Timing:** Near term **Effort:** L **Recommended owner:** Application owner with E10 identity owner

Evidence

- `scheduling_app/middleware.ts:154-184` compares every protected request against one `BASIC_AUTH_USERNAME` and `BASIC_AUTH_PASSWORD`.
- E01 records that office users share Scheduling credentials.
- A caller who knows the shared credential can read employee contact information and inbound messages; create, update, or delete employees, jobs, assignments, and absences; run reports; and send SMS messages.
- There is no individual user ID, role, session record, access review, revocation event, or user attributed audit history.
- There is no application rate limit, progressive delay, or lockout for repeated Basic Authentication attempts.
- Only the SMS route checks request origin; other mutating handlers do not enforce Origin/Host, a CSRF token, or an application session control beyond browser managed Basic credentials.
- CRUD rows and outbound message records do not capture the authenticated actor.

Impact

One copied, reused, or retained credential grants full administrative access. The organization cannot revoke one person's access without rotating the credential for everyone, distinguish an authorized change from misuse, or identify who deleted a schedule or sent a message. The same access model also prevents least privilege separation between schedule editors, message viewers, and administrators.

Recommendation

- Replace shared Basic Authentication through E10 with individual identities, managed sessions, recovery, offboarding, and MFA appropriate to the team.
- Define explicit permissions for schedule viewing, editing, employee administration, reporting, inbound message access, SMS sending, and application administration.
- Record the authenticated actor and request correlation ID on material changes and outbound communications.
- Require an application session CSRF control or validated same origin metadata on every browser mutation.

- Until migration, document who is authorized to hold the shared credential, rotate it when an authorized holder changes or exposure is suspected, and restrict SMS-send responsibility.

E04-F03 - Protected handlers rely exclusively on middleware authorization

Severity: High **Timing:** Immediate **Effort:** M **Recommended owner:** Scheduling application maintainer

Evidence

- Protected route handlers under `scheduling_app/app/api` perform no authentication or authorization of their own.
- The handlers call server modules using the Supabase service role credential, which bypasses RLS.
- `scheduling_app/middleware.ts:142-152` contains path exceptions for assets, the external job API, and Twilio; the matcher at lines 187-189 separately excludes the Twilio route.
- The asset exception is suffix based, so a future application route ending in a recognized static extension would bypass the gate unless separately protected.
- A middleware matcher regression, framework bypass, route rename, or newly added route can therefore expose a handler with no second authorization check.

Impact

The sole route gate protects service role-backed handlers capable of reading personal data, deleting scheduling history, and sending external communications. Failure of that one gate turns a routing defect into privileged database or Twilio access.

Recommendation

- Add a shared server side authorization function and invoke it inside every protected Route Handler before business logic.
- Deny by default, then explicitly classify public webhook and integration routes.
- Require a specific permission for destructive changes and SMS sends.
- Replace suffix based public-asset classification with framework owned static path matching.
- Add route matrix tests covering every method, trailing slash, encoding variant, public exception, unauthenticated request, and future unknown route.

E04-F04 - Assignment lifecycle state is not operationally enforced

Severity: High **Timing:** Immediate **Effort:** M **Recommended owner:** Scheduling application maintainer with operations owner

Evidence

- Assignments have `scheduled`, `completed`, `cancelled`, `missed`, and `no_show` states in `scheduling_app/lib/types.ts:8-13`.
- The Scheduling Board has no assignment status control. It initializes new assignments as `scheduled` and preserves an existing value internally, but the edit panel exposes only job status.
- Board cards and calendar events include assignments from every lifecycle state; `unassignedJobs` and `assignedJobCards` at `scheduling_app/app/schedule/page.tsx:348-399` treat even a cancelled assignment as an active job assignment.
- Remaining available hours at lines 550-570 subtract every assignment regardless of state.
- `scheduling_app/app/api/send-schedule-sms/route.ts:312-321` loads every assignment and absence overlapping the requested range without an assignment status or job status filter.
- `buildScheduleMessage` at lines 102-173 includes every overlapping assignment and never inspects `assignment_status`.
- The same function includes both assignments and absences for a date.
- Days scheduled ahead calculations exclude only `cancelled` and therefore count `completed`, `missed`, and `no_show` assignments as future coverage.
- The Total Hours report UI hard-codes `statusFilter: "all"` at `scheduling_app/app/reports/page.tsx:234-246`, so cancelled, missed, and no-show assignments contribute to its displayed total.
- The Scheduling Board only displays an absence conflict warning at `scheduling_app/app/schedule/page.tsx:461-473`; it does not block the conflicting records.
- The API permits a direct caller to create or update the same conflicting states without the browser warning.

Impact

Operators cannot reliably transition or distinguish assignment lifecycle state in the primary workflow. Inactive work can make a job appear assigned, consume remaining labor capacity, inflate coverage and hour reports, and tell an employee to report to a cancelled, completed, missed, or closed assignment. An employee can also receive both a job and an absence for the same day. These inconsistencies can create wasted travel, missed coverage, payroll disputes, and loss of trust in the schedule.

Recommendation

- Define the lifecycle, allowed transitions, capacity meaning, report meaning, and delivery eligibility for every assignment status.
- Add an explicit, attributable assignment status control and visually distinguish inactive states on the board.
- Centralize the approved status predicates and apply them consistently to board groupings, remaining hours, reports, and messages.
- Exclude closed jobs and resolve assignment/absence conflicts before producing a sendable preview.
- Make blocking conflicts visible in the preview with an explicit, user attributed override only if the business approves overrides.
- Recompute the final message from current database state at send time and show what changed since preview.
- Add tests for every assignment status and transition, capacity totals, reports, closed jobs, absences, overlaps, and messages that span several days.

E04-F05 - Outbound SMS sends are not idempotent or traceable through delivery

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Scheduling application maintainer with Twilio owner

Evidence

- `scheduling_app/app/api/send-schedule-sms/route.ts:402-443` sends messages in a loop with no operation ID, per message idempotency key, duplicate send check, rate limit, or resumable batch state.
- The browser confirmation at `scheduling_app/app/schedule/page.tsx:1169-1179` is the only duplicate send guard.
- `sendTwilioSms` at route lines 184-219 checks only `response.ok`; it does not parse or retain Twilio's message SID.
- The Twilio request has no explicit timeout or abort signal, and all recipients are processed sequentially in one Function invocation.
- Message text has no application length or segment budget; assignment and absence notes are unbounded and a request may cover as many as 31 days.
- The database log is written after the provider request. `logSmsResult` logs a database error and returns, so a successful provider send can have no durable record.
- The log status supports only `sent` and `failed`; it does not represent accepted, queued, delivered, undelivered, or delivery unknown states.
- No Twilio delivery status callback route exists.

Impact

A retry, double submission, client timeout, provider stall, or Function timeout partway through a sequential batch can send duplicate schedule messages. Conversely, the application can tell an operator that a request was sent without retaining the provider identifier needed to prove or reconcile delivery. Long notes and ranges can also create unexpectedly large or rejected messages. There is no safe way to resume only the unsent portion of a batch.

Recommendation

- Create a durable send batch and per recipient message record before calling Twilio.
- Use a deterministic idempotency key for employee, date range, schedule version, and operator approved batch.
- Store Twilio message SID and accepted status, then receive validated delivery callbacks for final status.
- Make retries resume unresolved records and require an explicit override to resend an already accepted message.
- Add provider timeouts, bounded batch execution, message/segment budgets, per user and per batch limits, alerting, and a reconciliation view.

E04-F06 - Inbound SMS authentication, resilience, and abuse controls are incomplete

Severity: Medium **Timing:** Near term **Effort:** S **Recommended owner:** Scheduling application maintainer with Twilio owner

Evidence

- `scheduling_app/app/api/twilio/inbound/route.ts:127-157` parses, verifies, matches, and stores the inbound message.
- The public handler calls `request.formData()` before it can validate the Twilio signature, so the runtime must accept and parse the complete body from an unauthenticated caller.
- No application control limits inbound request size, individual field length, parsing time, or webhook request rate. Deployment platform controls were not established in E04.
- `getPublicRequestUrls` at lines 22-45 accepts `request.url`, `Host`, `X-Forwarded-Host`, and `X-Forwarded-Proto` as signature URL candidates in addition to the configured canonical webhook URL.
- Caller influenced URL candidates do not independently bypass the HMAC requirement, but they unnecessarily broaden which captured Twilio-signed requests can validate and make proxy behavior part of the authentication boundary.

- The catch block at lines 158–162 returns the same successful empty TwiML response used after a stored message.
- A malformed deployed request reached that catch path and returned 200.
- A database outage, schema error, transient network failure, or other post-signature exception therefore also returns success.
- Twilio message SID has a deployed unique index, which is a positive replay control, but duplicate insertion is also handled as a logged exception and 200 rather than a recognized idempotent success.
- Phone matching loads employee rows and chooses the first normalized number match; the database does not require normalized phone uniqueness.

Impact

Twilio is told that delivery succeeded even when an employee reply was not saved, so it has no reason to retry. Replies can disappear without an operator alert. Duplicate or inconsistently formatted employee phone numbers can also attribute a reply to the wrong employee. Before signature rejection, arbitrary public traffic can consume request parsing and function resources up to undocumented platform limits. Accepting request derived signature URLs also creates avoidable ambiguity and replay surface around proxy headers and alternate Twilio webhook URLs.

Recommendation

- Return a retryable failure for transient parsing, database, and dependency failures; reserve 200 for stored or recognized duplicate events.
- Treat Twilio SID conflicts as idempotent success after confirming the existing record.
- Enforce a small request body limit and inbound rate control at the earliest trusted edge or platform layer, before the application parser, and reject unsupported content types.
- Validate signatures against one configured HTTPS webhook URL rather than request derived host or forwarding headers; prefer Twilio's maintained request validator and cover proxy behavior with signed fixtures.
- Store and index one canonical E.164 employee phone value and reject duplicates.
- Add structured webhook outcome metrics and alert on validation, matching, or persistence failures.
- Test signed success, invalid signature, replay, malformed form, database outage, unknown sender, and duplicate number behavior.

E04-F07 - Intake synchronization accepts stale updates and uses a race prone upsert

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Intake and Scheduling application maintainers

Evidence

- `scheduling_app/app/api/external/jobs/route.ts:343-380` first searches by `master_intake_id`, then by `job_number`.
- The subsequent update or-insert decision at lines 441-530 is made in separate requests rather than one constrained database operation.
- `sync_version` and `last_synced_at` are stored, but no incoming version or timestamp is compared with the existing row before overwrite.
- The deployed database makes `job_number` unique but does not make `master_intake_id` unique; it only has a length check.
- Two concurrent creates can both observe no row. Replayed or out of order events can overwrite newer schedule fields with older Intake values.
- A reused master Intake ID with a different job number can produce duplicate source identity and later make `.maybeSingle()` fail.

Impact

Concurrent, retried, or delayed synchronization can fail, create ambiguous source identity, or roll a job back to stale customer, site, schedule, status, or labor hour values. Scheduling has no durable event record from which to explain or replay the integration.

Recommendation

- Make the authoritative Intake source identifier unique in Scheduling.
- Use one database upsert or transaction keyed by that identifier.
- Define monotonic sync-version semantics and reject or record stale events without overwriting current state.
- Record a bounded integration event ID, payload hash, outcome, and correlation ID without retaining unnecessary payload data.
- Add concurrency, replay, out of order, identifier change, and partial failure tests across both applications.

E04-F08 - Integration and provider logging can disclose business and personal data

Severity: Medium **Timing:** Immediate **Effort:** S **Recommended owner:** Scheduling application maintainer with operations owner

Evidence

- `scheduling_app/app/api/external/jobs/route.ts:420-447` logs the full incoming object and normalized job payload.
- Those objects can contain customer name, site address, notes, Intake record URL, source identifiers, schedule dates, and available labor hours.
- Lines 473-517 log detailed update and insert results.
- `scheduling_app/app/api/send-schedule-sms/route.ts:211-216` logs the full Twilio error response body.
- The external job handler returns an arbitrary caught `Error.message` to the integration caller at lines 543-550; provider or PostgREST messages are not separated from safe validation errors.
- Generic server helpers log caught error objects, which can contain database and provider metadata.
- Vercel function log access, retention, export, and alerting were not reviewed in E04.

Impact

Application logs can become a secondary store of customer, project, employee, phone, and provider data with different access and retention controls than the primary database. Verbose error objects can also expose operational structure that is unnecessary for incident response.

Recommendation

- Remove full request, normalized payload, saved-row, and provider body logging.
- Return allowlisted validation errors to callers and map unexpected dependency errors to stable, non sensitive response codes.
- Log allowlisted identifiers, outcome codes, durations, and generated correlation IDs.
- Redact phone numbers, customer/site fields, notes, message bodies, URLs, and provider response bodies.
- Define production log access, retention, alerting, and incident export practices through E06 and E09.

E04-F09 - Editing one assignment overwrites the job's global dates

Severity: High **Timing:** Near term **Effort:** S **Recommended owner:** Scheduling application maintainer with scheduling owner

Evidence

- `scheduling_app/app/schedule/page.tsx:688-707` builds the edit form by copying the job, then replacing `startDate` and `endDate` with the selected assignment's dates.

- On save, lines 969–998 first calls `updateJob` using those assignment dates and then calls `updateAssignment` using the same dates.
- A job can have multiple assignments with different employee and date ranges.
- The operation is presented as editing one schedule card, but it silently changes the shared job record seen by every assignment and future Intake synchronization.

Impact

Saving one employee's assignment can narrow or move the job's overall dates, making job data inconsistent with other assignments and with Intake. Reports, future scheduling decisions, and synchronization can then use a value changed as an accidental side effect rather than an intentional job edit.

Recommendation

- Separate job fields from assignment fields in the UI and API contracts.
- An assignment edit must update only the assignment unless the operator enters a clearly separate job edit workflow.
- Preserve and display the job's authoritative dates independently of the employee assignment range.
- Add regression tests with multiple assignments against the same job and verify that editing one leaves the job and sibling assignments unchanged.

E04-F10 - Runtime request validation and schedule rules are not enforced server side

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Scheduling application maintainer with scheduling owner

Evidence

- `scheduling_app/lib/server/assignments.ts:90–170` validates an assignment's own fields but does not load or verify the employee, job, job status, job dates, other assignments, or absences.
- Most JSON Route Handlers cast `await request.json()` to a TypeScript input type. Type assertions do not validate runtime object shape, reject unknown fields, or prove UUID, string, boolean, enum, or collection types.
- The JSON handlers do not consistently require `Content-Type: application/json`. Parsing JSON regardless of the declared media type also weakens reliance on browser preflight behavior as a CSRF boundary.
- Required and optional text validation trims values but defines no consistent maximum lengths for names, email addresses, phone values, job fields, addresses, notes, identifiers, or URLs.

- The assignment sort-order handler accepts an unbounded caller supplied array and performs one database update per element.
- Assignment status and absence type are copied directly from JSON without a runtime allowlist; TypeScript types do not validate HTTP requests.
- The shared date validator accepts normalized impossible dates such as 2026-02-30, and the SMS date check validates only the YYYY-MM-DD shape.
- The API accepts overlapping assignments, assignment/absence conflicts, assignments to inactive employees, assignments to closed jobs, and assignments outside the job range as long as foreign keys and row local checks pass.
- Closed job checks exist only in selected browser actions at `scheduling_app/app/schedule/page.tsx:740-768` and `840-869`.
- Absence conflicts are visual warnings, not blocked or explicitly overridden.
- The deployed database has row local date and hour constraints but no exclusion constraint, trigger, or transactional scheduling rule.

Impact

A direct API request, concurrent user, alternate browser path, or future UI change can submit malformed, oversized, or semantically conflicting data that the current UI would not produce. Unexpected types currently fall through to JavaScript, PostgREST, or database errors, while large strings and collections can consume application, database, log, and browser resources. Schedule conflicts then flow into reports and employee communications.

Recommendation

- Confirm the business rules for overlaps, inactive employees, closed jobs, job boundaries, and absence overrides.
- Define strict runtime schemas for every route and integration payload. Reject unknown fields where practical and enforce UUID, enum, boolean, date, URL, field length, and collection length constraints before business logic.
- Require the expected content type for each endpoint and apply explicit request-size limits at the platform and application boundaries.
- Enforce approved rules in one server side transactional scheduling service.
- Return structured conflict details and require an attributable override only where the business permits one.
- Add database constraints where a rule can be expressed safely and authorization/invariant tests for every write path.

E04-F11 - Multi step writes are not transactional and deletes can erase history

Severity: High **Timing:** Near term **Effort:** M **Recommended owner:** Scheduling application maintainer with database owner

Evidence

- `scheduling_app/app/schedule/page.tsx:969-1048` updates or creates a job and assignment in separate API calls. Failure of the second call can leave the first change committed.
- `scheduling_app/lib/server/assignments.ts:184-202` updates sort positions one row at a time and stops on the first failure.
- Assignment deletion followed by reorder at `scheduling_app/app/schedule/page.tsx:934-955` is also split across requests.
- `scheduling_app/db/schema.sql:84-85` cascades employee and job deletion into assignments; line 136 cascades employee deletion into absences.
- The UI allows employee and job deletion after a browser confirmation but does not require the record to be unused, archive it, or preserve a tombstone.
- Assignment statuses include completed, missed, and no-show, indicating that assignments also represent historical operational records.

Impact

A transient failure can leave orphan jobs, partially updated schedules, or duplicated sort order. Deleting one employee or job can permanently erase historical assignments and absences that support reporting, scheduling review, and dispute resolution. Shared authentication provides no reliable actor for the deletion.

Recommendation

- Move compound job/assignment save, reorder, and delete operations into database transactions or single transactional service operations.
- Disable repeat submissions while writes are pending and make creation operations idempotent so a double click cannot create duplicate assignments.
- Prefer archival/inactivation for employees and jobs with history.
- Change historical foreign key behavior to restrict deletion or preserve immutable references according to the approved retention model.
- Add optimistic concurrency or version checks for concurrent editors.
- Test failure after each step and verify rollback, retry, and historical preservation.

E04-F12 - Unpaginated reads can silently omit schedules, messages, and report data

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Scheduling application maintainer

Evidence

- The Scheduling Supabase project's PostgREST maximum response size is 1,000 rows.
- `getAssignments`, `getAbsences`, `getEmployees`, `getJobs`, and `getInboundMessages` use unbounded `.select("*")` calls without range paging.
- `scheduling_app/lib/server/reports.ts` builds all reports after loading those complete looking arrays into memory.
- `scheduling_app/app/api/send-schedule-sms/route.ts:305-322` also relies on unpaginated employee, job, assignment, and absence results.
- The Scheduling Board computes remaining job hours from the returned assignment array.

Impact

Once a relation exceeds 1,000 matching rows, schedules, remaining hour totals, reports, employee matching, messages, or outbound texts can silently omit data. The application receives a successful response and has no indication that the result was truncated.

Recommendation

- Add explicit bounded pagination to collection and message APIs.
- Move report and remaining hour aggregation into database queries that return bounded results and explicit counts.
- Scope schedule and SMS queries by the requested date, employee, and job before loading data.
- Detect and fail visibly on unexpected truncation.
- Add tests above the 1,000-row platform ceiling.

E04-F13 - Deployed schema and migration state are not reproducible from source

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Scheduling application maintainer with Supabase owner

Evidence

- The deployed project contains all six expected tables and selected final constraints and indexes.

- `scheduling_app/current_supabase_schema.sql` is a tracked zero-byte file.
- `scheduling_app/db/schema.sql` omits the deployed jobs columns `master_intake_id`, `intake_record_url`, `client_id`, `site_id`, `sync_version`, and `last_synced_at`.
- No source migration adds those six integration columns or the deployed `jobs_master_intake_id_check`.
- The source contains three manually applied upgrade files with no ordering version or ledger.
- The README instructs operators to paste schema fragments into the Supabase SQL Editor.
- Supabase returned zero remote migration history entries.
- The deployed policy state differs from the bootstrap policy definitions.

Impact

An operator cannot build the deployed database from source, prove which changes ran, detect drift reliably, or retry a partially applied change. A recovery or new environment can therefore omit integration fields or recreate materially different access controls.

Recommendation

- Capture the accepted deployed baseline as versioned, reviewed migrations without copying application rows.
- Adopt one migration command and ledger for each controlled environment.
- Reconcile tables, columns, enums, constraints, indexes, views, grants, and RLS state.
- Verify both a fresh database build and an upgrade from the accepted deployed baseline in automation.
- Replace the empty current schema file with a generated schema check or remove the misleading artifact after the canonical migration workflow exists.

E04-F14 - The bootstrap schema defines permissive broad role RLS policies

Severity: Medium **Timing:** Immediate **Effort:** S **Recommended owner:** Scheduling application maintainer with Supabase owner

Evidence

- `scheduling_app/db/schema.sql:16-31` creates unrestricted select, insert, update, and delete policies on employee rows.
- The same `USING (true)` and `WITH CHECK (true)` pattern is repeated for jobs, assignments, and absences.

- Outbound message logs receive unrestricted insert and select policies at lines 190–196.
- Policies without an explicit role apply to `PUBLIC`, which includes Supabase API roles when they also have table privileges.
- The README tells an operator to apply this schema or selected policy statements.
- The deployed database currently has no policies and therefore does not exhibit the source exposure. It does retain broad direct grants on the inbound message table, blocked by the absence of a policy.

Impact

A fresh or repaired environment created from the documented bootstrap can allow broad Data API roles to read employee phone numbers, jobs, schedules, absences, or message logs and to mutate core records, depending on its default grants. The current deployment's more restrictive state does not make the source instructions safe.

Recommendation

- Remove always true broad role policies from the canonical schema.
- Explicitly revoke `PUBLIC`, `anon`, and `authenticated` privileges that are not required by the server only design.
- Define role specific policies only after E10 establishes an authenticated user and authorization model.
- Add migration assertions and runtime role tests proving that broad roles cannot read or mutate Scheduling data.

E04-F15 - Preview and Production share privileged configuration

Severity: High **Timing:** Immediate **Effort:** M **Recommended owner:** Vercel and application owners

Evidence

- Vercel has one sensitive environment variable record targeting both `preview` and `production` for each of `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_FROM_PHONE_NUMBER`, `SCHEDULING_EXTERNAL_API_KEY`, `BASIC_AUTH_USERNAME`, and `BASIC_AUTH_PASSWORD`.
- The reviewed code uses the service role key for unrestricted data access and the Twilio credentials for billable outbound messages.
- No separate non production Supabase project, Twilio sender, integration key, or synthetic dataset is documented.

Impact

Preview code can receive the same database, messaging, integration, and access credentials as Production. A feature branch, stale preview, or unreviewed change can read or modify live schedules, expose employee communications, send billable SMS messages, or disclose shared credentials.

Recommendation

- Determine the current destination of every shared record before creating or exercising more previews.
- Create separate non production Supabase and Twilio resources with synthetic or approved sanitized data and a non production Intake integration.
- Use distinct Basic Authentication or individual access for Preview.
- Rotate production credentials after separation and dependency validation.
- Add preview smoke tests that prove no production database or sender is reachable.

E04-F16 - Schedule and SMS records have no retention or deletion lifecycle

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Business data owner with application maintainer

Evidence

- Employees contain names, email addresses, and mobile phone numbers.
- Jobs and assignments contain customer names, site addresses, schedule details, and notes.
- `schedule_message_logs` stores the full destination phone and outbound body.
- `sms_inbound_messages` stores sender, recipient, full reply body, employee link, and receipt time.
- No source path expires, deletes, exports, redacts, or archives message records.
- No retention period, legal hold, privacy classification, subject-access process, or deletion owner is documented.
- Deleting an employee sets message log employee references to null but retains phone numbers and bodies.

Impact

Personal and operational communication can be retained indefinitely without a defined business purpose or reliable response to privacy, employment, dispute, or deletion requirements. Removing an employee does not remove or consistently anonymize their communications.

Recommendation

- Classify schedule, absence, phone, and message data with the business owner.
- Define retention periods and deletion/anonymization behavior for each record type.
- Preserve only the audit metadata needed after message body retention expires.
- Add an authorized export and deletion process with logged outcomes.
- Include provider side Twilio records and backups in the lifecycle design.

E04-F17 - Business critical and security behavior has no automated test or CI coverage

Severity: Medium **Timing:** Near term **Effort:** M **Recommended owner:** Scheduling application maintainer

Evidence

- No unit, integration, route, browser, database, or security test files or test command were found.
- No GitHub Actions workflow or prior workflow run was found.
- The application contains custom Basic Authentication, Twilio signature verification, integration token validation, job upsert, schedule arithmetic, conflict warnings, report calculations, destructive operations, and external message generation.
- The application was developed in 23 commits with direct production deployments from `main`.
- Local lint, TypeScript, and build checks pass but do not exercise business behavior.

Impact

Changes to authentication, scheduling rules, SMS text, webhook validation, report arithmetic, database mappings, or multi step UI behavior can reach Production without evidence that existing behavior still works.

Recommendation

- Add unit tests for validation, date ranges, status filtering, schedule text, phone normalization, report math, and sync version handling.
- Add Route Handler tests for authentication, authorization, malformed bodies, idempotency, concurrency, and safe errors.
- Add database tests for migrations, grants, RLS, constraints, pagination, and transactional operations.
- Add browser smoke tests for primary office workflows without sending live SMS.
- Run install, lint, TypeScript, tests, build, dependency audit, migration checks, and deployment smoke checks in required CI.

E04-F18 - Large browser and route modules increase change and regression risk

Severity: Medium **Timing:** Planned **Effort:** L **Recommended owner:** Scheduling application maintainer

Evidence

- `scheduling_app/app/schedule/page.tsx` is 2,290 lines and combines data loading, drag-and-drop, assignment editing, absence editing, job editing, remaining hour calculations, conflict warnings, SMS preview/send state, and multiple panels.
- `scheduling_app/app/reports/page.tsx` is 850 lines.
- The external job handler is 553 lines and the schedule SMS handler is 466 lines.
- The same schedule and report concepts are calculated across browser and server modules.
- There is no regression suite to protect extraction or behavioral changes.

Impact

Small changes require understanding unrelated concerns and increase the chance that a UI adjustment alters scheduling, reporting, or messaging behavior. Duplicated calculations can drift between the board, reports, and SMS output.

Recommendation

- First add characterization tests around the current behavior.
- Extract shared pure schedule, date, hours, status, and message formatting functions.
- Separate board state, assignment editing, absence editing, job editing, and SMS operations into focused components and hooks.
- Separate authentication, validation, persistence, and response formatting in the two large integration routes.
- Keep one server owned definition for calculations used by reports, remaining hours, and employee communications.

E04-F19 - Health, messaging telemetry, and operational alerting are minimal

Severity: Low **Timing:** Planned **Effort:** M **Recommended owner:** Application maintainer with operations owner

Evidence

- No health or readiness route exists.

- `package.json` defines no Node.js engine; the Vercel project reports Node.js 24.x, while local verification ran on Node.js 26.5.0.
- Logging is unstructured `console` output without stable event codes or request correlation.
- No source configuration identifies monitoring, alerting, uptime checks, failed send thresholds, webhook failure alerts, stale synchronization, or queue depth.
- `scheduling_app/lib/server/api.ts:5-13` defaults every caught validation, configuration, and dependency error to HTTP 400; most CRUD handlers do not override it, and the external job route also maps unexpected failures to 400.
- Required Supabase, Twilio, integration, and Basic Authentication configuration is validated only when the related request path executes.
- Outbound messages have no provider delivery status and inbound failures can be acknowledged silently.
- Supabase inventory returned a point in time unhealthy Realtime service while the project, database, Auth, REST, Storage, and pooler remained active; the application does not use Realtime or expose whether that condition affects it.

Impact

Operators can discover synchronization, database, or messaging failures only through user reports or manual log review. Server and dependency failures recorded as client errors also distort error rate monitoring and retry decisions. There is no quick readiness signal or documented incident evidence trail.

Recommendation

- Add a non sensitive liveness endpoint and an authenticated readiness check for required dependencies.
- Declare and test one supported Node.js major version across local, CI, Preview, and Production.
- Validate required production configuration before serving traffic and distinguish validation, conflict, authentication, dependency, and internal HTTP outcomes.
- Emit structured, redacted events with correlation IDs for synchronization, schedule mutations, SMS batches, delivery callbacks, and inbound webhooks.
- Alert on repeated failures, missing webhook traffic, unresolved outbound states, and stale Intake synchronization.
- Define service ownership, log access, dashboards, incident response, and recovery runbooks through E09.

E04-F20 - Intake synchronization reopens previously closed Scheduling jobs

Severity: Medium **Timing:** Immediate **Effort:** S **Recommended owner:** Intake and Scheduling application maintainers with scheduling owner

Evidence

- `intake_app/app.py:961-981` builds the Scheduling payload without a job status field.
- `scheduling_app/app/api/external/jobs/route.ts:227-230` maps only an explicit string value of `closed` to `closed` and defaults every other value, including a missing value, to `active`.
- The normalized status is included in the complete job payload at lines 310-346.
- When an existing job is found, lines 463-471 overwrite the row with that complete payload, including `status: "active"`.
- Intake can push manually, and `maybe_auto_push_to_scheduling` at `intake_app/app.py:1098-1127` also pushes whenever synchronized Intake fields change.

Impact

Editing and synchronizing an Intake record reopens the corresponding Scheduling job even when a Scheduling operator intentionally closed it. The job returns to active lists and can be assigned again; existing assignments can also flow back into capacity, reports, and employee messages. The application does not warn either operator that the closure was discarded.

Recommendation

- Define which system owns job lifecycle status and document the state mapping.
- If Scheduling owns closure, omit status from integration updates and preserve the stored value.
- If Intake owns status, send an explicit validated state and define allowed cross system transitions.
- Add an integration test that closes a Scheduling job, changes another Intake field, synchronizes it, and verifies the approved closure behavior.
- Record status changes with source, actor or integration identity, prior value, and correlation ID.

E04-F21 - Hard coded and UTC defaults can select the wrong work date

Severity: Medium **Timing:** Immediate **Effort:** S **Recommended owner:** Scheduling application maintainer with scheduling owner

Evidence

- The Jobs page initializes every new job to 2026-05-29 through 2026-05-30 at `scheduling_app/app/jobs/page.tsx:19-30`.
- The Scheduling Board's blank job and toolbar default are hard coded to 2026-05-29 at `scheduling_app/app/schedule/page.tsx:102-123` and 670-673.
- Blank absences and the new absence toolbar use `new Date().toISOString().slice(0, 10)` at lines 126-129 and 675-680.
- Browser report defaults and the server days ahead report also derive today from UTC at `scheduling_app/app/reports/page.tsx:53-55` and `scheduling_app/lib/server/reports.ts:140-142`.
- The scheduling board and SMS panel separately derive today using local calendar fields at `scheduling_app/app/schedule/page.tsx:234-253` and 1148-1155.
- A targeted check at 5:24 p.m. America/Los_Angeles on 2026-07-24 produced UTC default 2026-07-25 and local default 2026-07-24.

Impact

A toolbar-created job can silently begin almost two months in the past, and an absence or report opened after 5 p.m. Pacific can default to the following workday while the board and schedule texts remain on the current day. An operator can save or communicate the wrong date without an application error.

Recommendation

- Remove fixed calendar literals and derive new record dates from the selected calendar slot or the approved business timezone.
- Define one business time zone date utility and use it consistently in browser and server code.
- Display the effective timezone anywhere "today" drives a report or outbound message.
- Add timezone tests around UTC midnight, Pacific daylight/standard transitions, and year boundaries.

E04-F22 - The credential bearing Supabase destination is not validated

Severity: Medium **Timing:** Immediate **Effort:** S **Recommended owner:** Scheduling application maintainer with Vercel and Supabase owners

Evidence

- `scheduling_app/lib/server/supabase.ts:6-24` checks only whether `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` exist.

- URL normalization removes a REST suffix and trailing slash but does not require HTTPS, reject embedded credentials or unexpected ports, or verify an approved Supabase project host.
- The resulting URL and service role key are passed directly to the Supabase client; every database request then sends privileged credential material to that configured destination.
- Vercel stores the URL and service role key as separately editable configuration records.

Impact

A configuration mistake can send the service role credential and Scheduling data requests to the wrong project or an attacker controlled compatible endpoint. A person or compromised automation able to change configuration without reading the write only service key can redirect the client and capture that key.

Recommendation

- Parse configuration at startup and require HTTPS and the exact approved Supabase project host for each environment.
- Keep an environment to project allowlist in reviewed, non secret configuration.
- Reject user information, fragments, lookalike hosts, unexpected ports, and Preview/Production project crossover.
- Fail readiness before making any credential bearing request when the destination is invalid.
- Rotate the service role key if configuration history indicates it was ever sent to an unintended destination.

E04-F23 - Authenticated responses lack explicit CSP and private no-store controls

Severity: Medium **Timing:** Near term **Effort:** S **Recommended owner:** Scheduling application maintainer with Vercel owner

Evidence

- `scheduling_app/middleware.ts:23-32` adds MIME-sniffing, frame, referrer, and permissions controls but no Content Security Policy or cache policy.
- The reviewed Next.js configuration and Route Handlers define no equivalent global CSP or signed in response cache policy.
- The deployed unauthenticated responses inspected during E04 did not include a CSP.

- An authenticated local production-build request to `/jobs` returned `Cache-Control: s-maxage=31536000, x-nextjs-cache: HIT`, and no CSP.
- Authenticated local API error responses had no explicit `Cache-Control` header.

Impact

Protected UI and API responses rely on framework or platform caching behavior rather than an explicit privacy contract. Shared-cache eligibility is inappropriate for authenticated application pages, and the missing CSP increases the impact of a future injection defect. The client rendered page shell did not contain loaded Scheduling records in the local check, but future server rendering could place sensitive data under the same policy.

Recommendation

- Add and test a Content Security Policy compatible with Next.js and the application's required resources.
- Explicitly set `Cache-Control: private, no-store` on authenticated HTML and API responses.
- Preserve the existing frame, MIME-sniffing, referrer, and permissions controls.
- Disable the unnecessary `X-Powered-By` response header.
- Verify both authenticated and unauthenticated page, API, webhook, and integration responses in E06.

E04-F24 - Repository ignore rules and tracked CLI state increase disclosure risk

Severity: Low **Timing:** Immediate **Effort:** S **Recommended owner:** Scheduling application maintainer with E05 repository owner

Evidence

- `scheduling_app/.gitignore` ignores `.env` and `.env.local` but not conventional variants such as `.env.development`, `.env.production`, `.env.preview`, `.env.test`, or their local forms.
- Eight files under `scheduling_app/supabase/.temp/` are tracked in Git.
- `supabase/.temp/linked-project.json` contains the Supabase project reference, project name, organization identifier, and organization slug.
- `supabase/.temp/pooler-url` contains the database pooler host, region, port, database name, and project scoped database username.
- These values are infrastructure metadata rather than authentication credentials, and the targeted scan found no password, service role key, personal access token, or other credential value in the files.

Impact

A developer can create a conventional environment specific file that is not ignored and accidentally commit credentials into current and historical Git objects. Tracked CLI state also publishes unnecessary project and connection metadata that can assist reconnaissance and creates ambiguity over which generated local artifacts are authoritative source.

Recommendation

- Ignore `.env*` while explicitly allowing a sanitized `.env.example` if the project needs a configuration template.
- Ignore `supabase/` `.temp/` and remove the existing files from Git tracking without treating the non secret metadata itself as a credential incident.
- Keep project to environment mapping in reviewed configuration or operations documentation rather than generated CLI state.
- Add dedicated current tree and full history secret scanning, including Gitleaks or an equivalent scanner, through E05 and required CI.

Closeout and limits

The review covers the recorded company source and collected company platform evidence. Stephanie's personal test source, data, settings, checks, and exact promotion path were not inspected. No signed in board workflow, real employee message, live provider failure, restore, or destructive action was tested. Some schedule, closure, conflict, retention, and time zone rules still need business approval. Provider log content and secret values were not collected. The history scan was targeted, not a full Git object history scan. No Production state was changed.

Keep each finding open until its stated fix and a focused verification test pass. Use Appendix A for status and E11 for package order. A limitation or stronger result must be updated here before it changes the register.

Appendix C1 - GitHub and Release Process Findings

Source: E05 - GitHub and Deployment Process Audit **Status:** Provisional **Last updated:** 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This appendix records the GitHub and company source to Production evidence. The ten entries remain candidates until the remaining owner, history, access, and release checks are complete. Tom, Stephanie, and the GitHub owner should use it to close those checks.

Assessment summary

The reviewed company repositories are private and linked to the expected Vercel projects. Current live revisions match company `main`. The collected period shows Production deployments but no active branch rule, required pull request, application CI workflow, recent pull request, or blocking deployment check. The reported personal test to company promotion path was not inventoried.

The current design does not need a second routine technical reviewer. It needs the company repository as the release source, protected change records, required automated checks, an isolated company Preview, promotion of the exact tested revision by Stephanie, a release record, and rollback proof. E11 OP02-OP05 own that result.

Scope and method

The read only review covered organization and repository settings, access, branches, rules, Actions, environments, installed apps, commits, pull requests, release records, Vercel links, deployments, and current revision matching. Values, source file content through the APIs, customer data, and runtime logs were excluded. Internal commands, hashes, and collection paths are kept outside the client narrative.

Effort uses S for a bounded change, M for several files or parts, and L for broad work that should be split.

Current verification results

These results are based on the current API snapshots and remain subject to token permission and owner validation.

Check	Current result
Repository scope	Four private organization repositories use <code>main</code> : the three applications plus <code>internal-apps</code> , which has no matched Vercel project and requires owner/lifecycle classification
Organization access	Four organization members, including three administrators, plus one outside collaborator; no pending invitations returned
Organization MFA and web commit signoff	Organization wide two-factor authentication enforcement and web commit signoff are disabled
Default permissions	Organization default repository permission is read; members may create public and private repositories
Organization Actions policy	Actions are enabled for all repositories, all actions are allowed, and immutable SHA pinning is not required; workflow tokens default to read only and cannot approve pull requests
Branch controls	No active branch rule, repository ruleset, or classic branch protection configuration returned for any application <code>main</code> branch
Pull requests	No recent pull requests returned for the three application repositories
GitHub Actions	Actions are enabled, but no workflows or recent workflow runs were returned for any application repository
Repository governance files	No workflow, <code>CODEOWNERS</code> , dependency update, security-policy, contribution, or release-automation file was found in the three local application baselines
Commit status	Each current <code>main</code> head has one successful Vercel deployment status; check run visibility was unavailable to the supplied PAT
Commit verification	All 96 sampled application commits are unsigned: 50 Intake, 23 Production, and 23 Scheduling

Check	Current result
Commit attribution	All 96 sampled application commits name outside collaborator <code>rhrspace</code> as GitHub author and committer; the Team plan and commit API do not establish the push actor
GitHub environments	Each application repository has a <code>Production</code> environment; no required reviewers or wait timer were returned and administrator bypass is allowed
Security automation	Dependabot, code scanning, secret scanning, and push protection are not enabled for new repositories; repository security endpoints report disabled features or permission gaps
Vercel GitHub App	Installed with selected repository scope and broad write permissions; the supplied PAT could not enumerate the selected repositories
Vercel linkage	All three Vercel projects link to the expected company repositories and use <code>main</code> as <code>Production</code>
Deployment sample	58 Vercel deployments dated 2026-06-09 through 2026-06-28 returned; all target <code>Production</code> , use <code>main</code> , reached <code>READY</code> , and match the expected repository identity; none of the 28 enriched records skipped its build
Deployment sources and actors	Vercel attributes 23 Git deployments to <code>rhrspace</code> , 32 redeployments to <code>stephanie-wt</code> , and three imports to <code>stephanie-wt</code>
Deployment review signal	The 28 deployments enriched through the checks endpoint have zero Vercel checks and zero pull request IDs; GitHub independently returns zero pull requests for the application repositories
Current revision alignment	Each latest <code>Production</code> deployment matches the current GitHub <code>main</code> head
Deployment history alignment	All 26 unique revisions represented by the 58 Vercel deployments exist in the sampled GitHub <code>main</code> histories

Check	Current result
GitHub deployments	GitHub returns 26 application deployment records; every record has at least one successful <code>vercel[bot]</code> Production status
Hooks, keys, and Actions values	No repository deploy key, repository/organization webhook, or Actions secret/variable name was returned

Positive controls

- All three application repositories are private and company organization owned.
- Organization default repository access is read, and private repository forking is disabled.
- The Vercel projects link to the expected company repositories and current `main` revisions.
- The latest Production deployment for each application matches the recorded application audit baseline.
- The sampled deployments reached `READY` and received their Production aliases.
- GitHub Actions default workflow token permissions are read only, and workflows cannot approve pull request reviews by default.
- Private repository fork workflows are disabled and do not send secrets, variables, or write tokens.
- Vercel is installed with selected repository scope rather than explicit all repository scope, subject to exact scope verification.
- Generated evidence is allowlisted, redacted, read only, and written with owner only file permissions.

Findings summary

The following findings are provisional. Severity, ownership, and wording must be finalized after the remaining evidence and owner validation.

Effort ratings use the same relative T-shirt scale as E02-E04:

- **S - Small:** bounded change in one primary area with limited coordination
- **M - Medium:** change across several files or parts that needs meaningful tests and review
- **L - Large:** broad remediation or coordination that should be planned as several tasks

ID	Candidate finding	Severity	Timing	Effort
E05-F01	Organization MFA enforcement and	High	Immediate	S

ID	Candidate finding	Severity	Timing	Effort
E05-F02	Unprotected main branches deploy directly to Production without a controlled promotion gate	High	Immediate	M
E05-F03	Production changes have no required automated build, test, security, or deployment checks	High	Immediate	M
E05-F04	The reported personal Development and QA to company source and database promotion process is not documented or auditable	High	Immediate	M
E05-F05	Secret scanning, push protection, dependency alerts, and code scanning are not enabled	High	Immediate	M
E05-F06	Vercel GitHub App permissions are broad and exact selected repository access is unverified	Medium	Near term	S
E05-F07	Repository access, outside collaborator need, actual access change handling, and annual confirmation are not evidenced	Medium	Near term	S
E05-F08	Thirty-two Production redeployments lack release, operator, reason, rollback, and emergency change linkage	Medium	Near term	M
E05-F09	Commit authenticity is not verified through signatures or an equivalent provenance control	Low	Planned	M
E05-F10	Future Actions supply chain policy permits all actions without immutable SHA pinning	Medium	Near term	S

Detailed findings

Each finalized finding will use the application audit format:

1. finding title;
2. severity, timing, effort, and recommended owner;
3. reproducible evidence without secret values or customer data;

4. business and technical impact; and
5. specific recommendations scoped as future work.

Before finalization:

- E05-F01 must distinguish lack of organization enforcement from individual member MFA state and include company ownership, billing, and emergency company recovery evidence.
- E05-F02 and E05-F03 must be validated against the actual testing and promotion practice, API permission gaps, Vercel checks, and any controls outside GitHub.
- E05-F04 must trace one real promotion per application and record the personal and company repositories/projects, tested and released revisions, QA evidence, test data boundary, source of truth, release operator, conflict handling, database change handling, secrets boundary, and Production decision.
- E05-F05 requires a full history scan and feature/licensing assessment; no secret value or customer record may enter the report.
- E05-F06 requires dashboard verification of Vercel's selected repositories and a business justification for its requested write permissions.
- E05-F07 must validate every administrator, member, outside collaborator, team, deploy key, hook, and App with the owner, then define prompt removal when access actually changes and one short annual confirmation.
- E05-F08 must reconcile each dashboard redeployment and GitHub environment/release record with its initiator, reason, recorded operator, Vercel retention, rollback, database migrations, incident, and emergency change history.
- E05-F09 must be weighed against the team's size, chosen identity controls, and the provenance provided by reviewed pull requests and protected branches.
- E05-F10 must be validated against organization and repository Actions policy, then incorporated into the CI design so third-party actions are allowlisted and pinned to full commit SHAs where feasible.

Candidate response

ID	Recommended action	Verification
E05-F01	Require MFA. Confirm each member and admin. Give Tom a tested company recovery path.	Access is approved and recovery works without Stephanie.
E05-F02	Protect each app's main branch. Use pull requests as change records. Block force push and delete. Limit bypass.	A direct unsafe change is blocked. A normal pull request passes the rule.
E05-F03	Require install, lint, type or syntax, test, build, dependency, migration, and secret checks that fit each app.	A known bad change fails the right required check.
E05-F04	Trace one real test to company release per app. Replace the personal path with company branch, Preview, release, and recovery steps.	The tested and released source and migrations match. Personal accounts are not required.
E05-F05	Enable available secret and dependency controls. Run a controlled full history scan. Rotate only secrets with confirmed exposure.	Scan results have an owner and decision. Confirmed exposed values are replaced.
E05-F06	Confirm the exact repositories and permissions granted to the Vercel App. Remove access with no stated need.	App scope matches the approved list and has a named owner.
E05-F07	Approve every member, collaborator, key, hook, and app. Remove access when need changes. Review the short list once a year.	Current access and recovery contacts have owners and review dates.
E05-F08	Link each release and redeploy to an operator, reason, source SHA, checks, rollback point, and incident when relevant.	One normal release, redeploy, emergency change, and rollback are traceable.
E05-F09	Decide on signed commits or equal source proof after protected branches, individual users, and CI are stable.	The chosen source proof and its limit are recorded.
E05-F10	Allow only reviewed Actions and fix third party Actions to full commit SHAs where practical.	CI rejects an unapproved or floating Action reference.

Remaining evidence and limits

Still required: company recovery and MFA proof; need for each member, admin, collaborator, app, key, and hook; exact Vercel App repository scope; a full history artifact and secret scan; classification of `internal-apps`; one traced personal test to company release per app; reasons for dashboard redeploys; emergency release and rollback proof; and controls hidden by plan or API limits.

The sample is point in time and limited by token scope and plan access. Commit author data does not prove who pushed. No secret value, customer data, personal repository, Production setting, or service state was changed. Keep all E05 entries provisional until their named checks are complete. Update E05 before changing Appendix A or E11.

Appendix C2 - Vercel Configuration and Operations Findings

Source: E06 - Vercel Configuration and Operations Audit **Status:** Provisional **Last updated:** 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This appendix records the Vercel ownership, release, environment, URL, monitoring, and recovery evidence. The nine entries remain candidates until the remaining dashboard and owner checks are complete. Tom, Stephanie, and the Vercel owner should use it to close those checks.

Assessment summary

The three company projects link to the expected company repositories. Current live revisions match. Recent sampled releases reached **READY**, and generated deployment URLs are protected. The same collected settings target Preview and Production, including privileged values. The reviewed period did not show an isolated company Preview path, blocking release checks, or a tested rollback procedure.

The target is a company owned isolated Preview, separate credentials and data, required source and deployment checks, promotion of the exact tested revision by Stephanie, short smoke and observation proof, and a known rollback point. E11 OP05 and OP08 own that result.

Scope and method

The read only review covered team and project access visible to the collector, Git links, branches, releases, settings names and targets, domains, regions, protection, retention, and selected service state. Values, source content, customer data, build logs, and runtime logs were excluded. Internal commands, hashes, and collection paths are kept outside the client narrative.

Effort uses S for a bounded change, M for several files or parts, and L for broad work that should be split.

Current verification results

Check	Current result
Inventory execution	Schema v2 snapshot completed without collection errors
Team access	One Owner and two Developers; no pending invitations
Project access	No explicit project level members returned
Git connection	Three projects linked to the corresponding company repositories; Production branch is <code>main</code>
Recent company deployments	All 58 returned records are <code>READY</code> Production deployments: 38 Intake, 12 Scheduling, and eight Production; no company Preview target observed; personal Vercel was outside scope
Deployment detail sample	Latest 10 per project collected when available, with runtime, alias, and check metadata; 28 detailed records were returned
Deployment checks	No checks returned for the 28 detailed deployment samples
Environment separation	All 23 records target both Preview and Production
Custom environments	None returned for any project
Domains	One verified <code>vercel.app</code> project domain per application; no registered or custom company domain returned
Deployment protection	<code>all_except_custom_domains</code> returned for all three projects
Source/build log visibility	Detailed deployments report <code>public: false</code>
Repository protections	Git fork protection enabled and directory listing disabled for all projects
OIDC	Enabled with team issuer mode for all projects
Retention	180 days for non production, 365 for Production, and at least 10 deployments retained
Observability	Web Analytics configured; Speed Insights has no data; no drains returned
Hooks and integrations	No webhooks or account integrations returned

Positive controls

- All three projects are connected to the expected company GitHub repositories and their current Production commits match the application audit baselines.
- The sampled recent deployments reached `READY` and received their Production aliases.
- Git fork protection is enabled on every project.
- Directory listing is disabled and source maps are protected.
- Detailed deployments report `public: false`, preserving Vercel source and build log protection.
- Generated deployment URLs use Vercel deployment protection.
- Environment variable values are stored as sensitive records and were not exposed to the audit collector.
- OIDC is enabled with team issuer mode, providing a path toward short lived backend credentials where supported.
- Deployment retention is explicitly configured.
- Project domains are verified and their API-reported DNS configuration is not misconfigured.

Findings summary

The following findings are provisional. Severity, ownership, and wording must be finalized after the remaining evidence and owner validation.

Effort ratings use the same relative T-shirt scale as E02-E04:

- **S - Small:** bounded change in one primary area with limited coordination
- **M - Medium:** change across several files or parts that needs meaningful tests and review
- **L - Large:** broad remediation or coordination that should be planned as several tasks

ID	Candidate finding	Severity	Timing	Effort
E06-F01	Vercel ownership and recovery depend on one confirmed team Owner	High	Immediate	S
E06-F02	Changes to <code>main</code> deploy directly to Production without controlled promotion or blocking deployment checks	High	Immediate	M

Momentum Electrical Contractors				Internal
ID	Candidate finding	Severity	Timing	Effort
E06-F03	Preview and Production share privileged credentials and downstream systems	High	Immediate	M
E06-F04	No isolated company Preview or staging release path is evidenced	Medium	Near term	M
E06-F05	Monitoring, alerting, and deployment validation are not operationally evidenced	Medium	Near term	M
E06-F06	Rollback capability exists as retained artifacts but has no demonstrated procedure	Medium	Near term	S
E06-F07	URL, DNS, and generated deployment lifecycle ownership is not defined	Medium	Near term	S
E06-F08	Scheduling functions and their Supabase database operate in different regions	Medium	Planned	M
E06-F09	dashboard only security, token, spend, and incident controls remain unverified	Pending evidence	Immediate	S

Detailed findings

Each finalized finding will use the application audit format:

1. finding title;
2. severity, timing, effort, and recommended owner;
3. reproducible evidence without secret values or customer data;
4. business and technical impact; and
5. specific recommendations scoped as future work.

Before finalization:

- E06-F01 must be validated against legal/company ownership, billing contact, MFA, emergency recovery, and actual access change evidence.
- E06-F02 must be reconciled with E05 branch controls, GitHub checks, and the actual Preview to Production promotion practice.

- E06-F03 must distinguish intentional public identifiers from service role, authentication, AI, messaging, and integration credentials.
- E06-F04 must confirm whether Preview is unused by design or simply absent from the collected history.
- E06-F05 must include dashboard alerting and log retention evidence without copying runtime log content.
- E06-F06 must account for database migrations and other changes that a Vercel code rollback cannot reverse.
- E06-F07 must classify current and historical URLs and confirm DNS ownership.
- E06-F08 must be supported by latency measurements and workload requirements, not region labels alone.
- E06-F09 must either become one or more findings supported by evidence or be removed once the dashboard controls are verified.

Candidate response

ID	Recommended action	Verification
E06-F01	Confirm company ownership, billing, MFA, and member need. Give Tom a tested company recovery path.	Recovery works and every member has an approved role.
E06-F02	Require source and deployment checks before a Production alias. Let Stephanie promote the exact tested revision.	One release links the pull request, tested SHA, checks, operator, smoke result, and rollback point.
E06-F03	Give Preview separate privileged settings, data, and provider paths. Rotate shared values after separation.	Preview cannot reach Production data or use Production service keys.
E06-F04	Create an isolated company Preview or record a clear decision not to use Preview until it is safe.	A pull request creates a protected test release with safe data and scoped settings.
E06-F05	Add release smoke checks, uptime and error signals, safe logs, alerts, and named response owners.	A test failure reaches the owner and links back to the release.
E06-F06	Practice rollback outside Production. Record database and integration changes that code rollback cannot undo.	A second operator can follow the steps and state the data limit.
E06-F07	Classify live, Preview, branch, generated, and old URLs. Name DNS, domain, retention, and access owners.	The approved URL list and removal rules match the dashboard and DNS.
E06-F08	Measure Scheduling to Supabase delay under real work before changing regions.	The region decision cites measured user impact, migration risk, and cost.
E06-F09	Review tokens, firewall, bypass, spend, billing alerts, logs, and incident contacts in the dashboard.	Each control has evidence, an owner, and either a final finding or a closed check.

Remaining evidence and limits

Still required: company ownership and emergency recovery proof; MFA and need for each member; dashboard checks for tokens, firewall, bypass, spend, billing, alerts, logs, and incident contacts; exact Preview behavior and data; historical URL and domain ownership; release and redeploy reasons; one complete promotion trace; and a rollback exercise that includes database limits.

The sample is point in time and limited by token scope and dashboard only controls. A retained code artifact does not prove that database rollback is safe. Region labels alone do not prove a performance issue. No secret value, customer data, personal project, Production setting, or service state was changed. Keep E06-F01-F08 provisional and E06-F09 pending until their named checks are complete. Update E06 before changing Appendix A or E11.

Appendix C3 - Supabase and Data Security Findings

Source: E07 - Supabase Configuration and Data Security Audit **Status:** Client review draft

Last updated: 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This appendix holds the full Supabase finding cards. Stephanie, the data owner, and the Supabase owner should use it when reviewing database change, access, key, backup, Storage, Auth, and MFA evidence.

Assessment summary

Momentum uses one Supabase project for Intake and Production and another for Scheduling. RLS is enabled on the reviewed business tables, no public Storage bucket was found, and the deployed Intake schema contains the expected final state. These are useful controls.

The main gaps are repeatable migration history, shown backup and restore ability, open direct database entry with SSL enforcement off at the reviewed control, undocumented legacy key use, broad but currently blocked Data API grants, unused Auth defaults, and admin accounts without MFA. A database backup also does not prove recovery of required Storage objects.

Scope and method

The read only review covered organization and project metadata, selected schema facts, migration history, grants, RLS, policies, network and SSL controls, backup metadata, Storage settings, legacy key metadata, unused Auth settings, and admin MFA state. No secret value, user record, business row, Storage object name or content, log, or destructive test was used. A restore was not run.

Effort uses S for a bounded change, M for several files or parts, and L for broad work that should be split.

Findings summary

Effort ratings use relative T-shirt sizes:

- **S - Small:** bounded change in one primary area with limited coordination

- **M - Medium:** change across several files or parts that needs meaningful tests and review
- **L - Large:** broad remediation or coordination that should be planned as several tasks

ID	Finding	Severity	Timing	Effort
E07-F01	Database change history is not recorded or reproducible	Medium	Near term	M
E07-F02	Backup and recovery capability cannot be demonstrated	High	Immediate	M
E07-F03	Direct database ingress is unrestricted and SSL is not enforced	Medium	Near term	M
E07-F04	Legacy API key dependencies and custody are not documented	Medium	Near term	M
E07-F05	Unnecessary broad Data API table privileges remain	Medium	Near term	S
E07-F06	Unused Supabase Auth surfaces retain development grade defaults	Medium	Immediate	S
E07-F07	Supabase administrative access includes accounts without MFA	High	Immediate	S

Detailed findings

E07-F01 - Database change history is not recorded or reproducible

Severity: Medium

Timing: Near term

Effort: M

Recommended owner: Supabase owner with application maintainers

Evidence

- Both projects returned zero remote migration history entries.
- Intake's 39 local migrations and selected deployed final state effects are present, but the README and bootstrap schema are stale.
- Production uses five source migrations applied through a documented manual process.
- No automated empty database build or deployed baseline upgrade check is documented.

Impact

The current databases may be operational, but an operator cannot reliably prove which changes ran, reconstruct the same state, detect drift, or retry a partially applied deployment. This raises future release and recovery risk.

Recommendation

Adopt one versioned migration workflow and ledger for each controlled environment. Reconcile each deployed baseline once, document how it was established, then verify fresh builds and upgrades in automation. Treat the observed final state as evidence during reconciliation rather than replaying unverified migrations against production.

E07-F02 - Backup and recovery capability cannot be demonstrated

Severity: High

Timing: Immediate

Effort: M

Recommended owner: Supabase owner with business data owner

Evidence

- Both projects report WAL-G enabled and PITR disabled.
- Neither collected backup response includes a backup entry or completed timestamp.
- No retention window, recovery point objective, recovery time objective, or successful restore evidence was provided.
- Restore was not tested.
- Database backups do not recover the contents of Storage objects.

Impact

The organization cannot currently demonstrate how much database or voice recording data could be lost, how long recovery would take, or whether recovery procedures work. A database incident, mistaken change, or Storage loss may therefore produce unplanned downtime or permanent data loss.

Recommendation

Confirm the plan entitlement and actual backup schedule with the Supabase owner, record the latest usable recovery point for each project, define recovery objectives, and perform a separately authorized non production restore drill. Create an explicit retention and recovery strategy for `daily-voice-logs` objects and document how application/database consistency will be handled.

E07-F03 - Direct database ingress is unrestricted and SSL is not enforced

Severity: Medium

Timing: Near term

Effort: M

Recommended owner: Supabase owner with deployment owners

Evidence

- Both projects allow 0.0.0.0/0 and ::/0 for direct database access.
- Database SSL enforcement is disabled on both projects.
- Approved direct Postgres and pooler clients, source addresses, and SSL requirements are not documented.

Impact

Any internet host can reach the database authentication boundary, and a direct client can connect without enforced transport encryption. Credentials remain required, but the projects lack two defense in depth controls for direct database connectivity.

Recommendation

Inventory every required direct database and pooler client. Restrict source networks where stable egress addresses permit it, require SSL for direct connections, verify application and administrative compatibility first, and document any necessary exception. HTTPS Data API clients should be assessed separately because these settings do not restrict them.

E07-F04 - Legacy API key dependencies and custody are not documented

Severity: Medium

Timing: Near term

Effort: M

Recommended owner: Supabase owner with application and Vercel owners

Evidence

- Legacy API keys are enabled on both projects.
- Intake and Production use server side privileged database credentials.
- The inventory does not identify every key consumer, custodian, environment, rotation date, or revocation dependency.

Impact

The organization cannot rotate or retire keys confidently without risking an outage, and unnecessary privileged or legacy credentials may remain active after users, environments, or integrations change.

Recommendation

Inventory key consumers by application and environment without copying secret values into the report. Record owner, purpose, privilege, storage location, rotation process, and revocation dependency. Migrate to the supported key model through separately scoped work, validate each consumer, and disable unused legacy keys only after dependencies are proven removed.

E07-F05 - Unnecessary broad Data API table privileges remain

Severity: Medium

Timing: Near term

Effort: S

Recommended owner: Supabase owner with Production and Scheduling maintainers

Evidence

- All five Production tables grant effective `anon` and `authenticated` access for the seven checked table privilege types.
- Scheduling's `sms_inbound_messages` table has the same effective privileges.
- RLS is enabled and no policies are deployed, so those privileges do not currently produce row access.
- The applications use service role credentials on their servers and do not require these broad direct API privileges.

Impact

RLS currently fails closed. However, accidentally disabling RLS, applying an unsafe policy, or recreating a table without the same controls would immediately combine with the retained grants and expose read or mutation capabilities. The grants also obscure the intended service only access model.

Recommendation

Revoke unused `anon` and `authenticated` privileges from the six service only tables through versioned migrations, verify the applications still work through the service role, and add metadata plus runtime negative tests. If future user-context access is introduced, grant only required operations alongside reviewed role specific policies.

E07-F06 - Unused Supabase Auth surfaces retain development grade defaults

Severity: Medium

Timing: Immediate

Effort: S

Recommended owner: Supabase owner with identity architect

Evidence

- Both projects permit email sign-up even though the current applications do not use Supabase Auth.
- Both Auth configurations use `http://localhost:3000` as the site URL and have an empty redirect allowlist.
- The minimum password length is six with no character requirements.
- Breached password checking, CAPTCHA, and password change reauthentication are disabled.
- Email automatic confirmation is disabled, and the current table/Storage RLS state limits application data access by newly created users.

Impact

An unnecessary public identity surface can accumulate unwanted accounts and consume email or Auth resources. If either project is later used for identity without a deliberate setup step, weak password and redirect defaults can become part of Production behavior. The current RLS state is an important mitigation but is not a substitute for configuring or disabling Auth.

Recommendation

If Auth is unused, disable public sign-up and monitor for unexpected users until the dedicated identity design is implemented. If either project will provide identity, first configure exact HTTPS site and redirect URLs, stronger password and breached password controls, appropriate CAPTCHA/rate limits, sensitive-action reauthentication, email delivery, MFA policy, audit retention, and tested recovery. Do not silently reuse these business data projects as the E10 identity environment.

E07-F07 - Supabase administrative access includes accounts without MFA

Severity: High

Timing: Immediate

Effort: S

Recommended owner: Accountable company owner with Supabase organization owners

Evidence

- The Supabase organization has two Owners and one Developer.
- The Management API records one Owner and the Developer with `mfa_enabled = false`.
- An Owner can control organization and project resources; a Developer has project content access.
- Business need, current access need, emergency recovery ownership, and token custody for all three memberships are not yet documented.

Impact

A compromised password for the non-MFA Owner can lead to administrative takeover, configuration changes, credential access, or destructive project actions. A compromised Developer account can expose or alter project content within its role. Existing application RLS does not protect against authorized platform administration.

Recommendation

Require MFA for every Supabase organization member, beginning with Owners. Validate the need and least privilege role for each membership. Keep Stephanie as the primary operator and give Tom a tested company controlled emergency recovery method. Review active Management API tokens and other administrative credentials after MFA is established, when access actually changes, and during one short annual ownership confirmation. Revoke unused or personally controlled access paths.

Positive controls

- Both projects are active and report healthy database, Auth, REST, Storage, and pooler services in the corrected inventory.
- Intake and Production use a shared database by design, while Scheduling is in a separate project and region.
- All 19 expected Intake tables exist and match the selected final state schema markers.
- RLS is enabled on all expected Intake tables.
- No direct `PUBLIC`, `anon`, or `authenticated` table grants were found for the Intake tables.
- No RLS policies target those broad roles on the Intake tables.
- No public Storage bucket was found.
- The shared Storage bucket is private and has an explicit size and MIME allowlist.
- Core Storage tables have RLS enabled and no broad Storage policies.
- All Production and Scheduling tables have RLS enabled with no deployed policies, so current direct `anon` and `authenticated` row access fails closed.

- No broadly readable exposed schema view or broadly executable non-trigger security definer function was found.
- The focused check returned no collection errors and made no data changes.

Closeout and limits

The Intake checks cover selected final state and access facts. Full Production definitions and some wider runtime role tests remain. Personal Supabase projects were outside scope. Metadata does not replace allow and deny tests as normal users, service tasks, signed links, or S3 clients. Backup metadata does not prove restore or Storage recovery. The platform inventory and focused checks were repeated on 2026-07-31. No Production state was changed.

Still required: full Production definition review; personal test project mapping; any Scheduling Realtime need; backup entitlement, retention, targets, and an approved restore; direct database client need; key consumers and custody; service role separation; removal or acceptance of unused grants and Auth; and MFA, recovery, and access need for each admin.

Keep each finding open until its stated fix and verification pass. Use Appendix A for status and E11 for package order.

Appendix D - Implementation Roadmap

Source: E11 - Operations Implementation Proposal and Roadmap **Status:** Client review draft
Last updated: 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This appendix turns the audit findings into work that Momentum can approve in stages. Tom and Stephanie should use it to choose scope, order, owners, and cost limits. Implementation owners should use each package's acceptance checks.

This is a planning proposal, not a fixed price quote. The source audits hold the evidence. [Appendix A](#) is the full finding register. E05 and E06 findings are provisional until the remaining owner and dashboard checks are done.

Program summary

Momentum needs one company controlled way to build, test, release, recover, and manage its three applications. The program has six outcomes:

1. Close the most urgent access, runtime, credential, and workflow risks.
2. Put source, platforms, recovery, and billing under clear company control.
3. Require recorded changes, automated checks, an isolated Preview, and a tested release path.
4. Make database changes, backups, restores, alerts, and incident work repeatable.
5. Replace shared passwords with individual sign-in and direct permission checks.
6. Fix the remaining application bugs in small groups that fit the client roadmap.

Tom is the accountable company owner. Stephanie is the Controller, application developer, release operator, and engagement lead, with Tom copied. Her personal testing accounts are a temporary source path. The final system must not depend on them for source, release, operations, or recovery.

The recommended first approval is OP01-OP08. These packages contain urgent risk and create the base needed for later identity and application work. OP09-OP14 deliver individual identity. OP15-OP17 fix remaining bugs. Momentum can approve each group separately.

Effort and approval rules

Size	Meaning
S	One bounded area with limited coordination
M	Several files or services with meaningful tests and review
L	Broad work that should be split into several tasks
XL	A large package with several work areas or owners
XXL	A program sized package that should be approved in finding based groups

Sizes show relative effort. Do not add them into a total. Confirm scope and price after the package needs are met.

Each approved package must name the client owner, implementation owner, access needs, change window, acceptance tests, and work that can run in parallel. If client work touches the same route, module, schema, or integration, freeze only that small area for the shortest useful period.

Work package matrix

ID	Outcome	Timing	Effort	Needs
OP01	Contain urgent access, runtime, secret, and workflow risks	Immediate	XL	Owner access and change windows
OP02	Confirm company ownership and protect GitHub source	Immediate	S	Company owner approval
OP03	Set safe rules for coding agents and pull requests	Near term	S	OP02
OP04	Add required tests, CI, and security checks	Immediate / Near term	M	OP01-OP02
OP05	Build an isolated Preview to Production release path	Near term	M	OP02, OP04
OP06	Make Supabase changes and access controlled and repeatable	Near term	L	Data owner decisions
OP07	Prove backup, restore, and recovery	Near term	S	Recovery decisions
OP08	Add monitoring, alerts, incident work, and release signals	Near term	M	OP05, OP07
OP09	Prove the identity provider and cross project trust	Planned; blocking	S	E10 decisions
OP10	Build the identity base, Backoffice, and app adapters	Planned	XL	OP09
OP11	Move Scheduling to individual users	Planned	M	OP06, OP10
OP12	Move Production to individual users	Planned	L	OP06, OP10
OP13	Move Intake to individual users	Planned	L	OP06, OP10
OP14	Cut over users, retire shared access, and hand off	Planned	S	OP11-OP13
OP15	Fix Intake workflow and maintainability findings	Coordinate with client	XL	OP01, OP04, OP06

ID	Outcome	Timing	Effort	Needs
OP16	Fix Production reliability and data findings	Coordinate with client	XXL	OP01, OP04, OP06, OP07
OP17	Fix Scheduling workflow, sync, and SMS findings	Immediate / Near term	XXL	OP01, OP04, OP06

Package cards

OP01 - Immediate containment and credential control

Outcome: Known urgent paths fail closed. Privileged settings and credentials have clear owners. Immediate workflow risks are bounded before more release automation is added.

Includes: Fix Intake file path and access failures. Patch vulnerable Production and Scheduling runtimes. Add direct checks to protected handlers. Replace the Production password cookie with a short lived opaque session until OP12. Guard Production history. Enforce Scheduling assignment state. Stop unsafe logs, closed job reopen behavior, wrong date defaults, and broad bootstrap policies. Remove committed database, log, and CLI artifacts. Scan relevant history. Rotate any confirmed exposed secret. Validate credential bearing destinations. Separate Preview and Production settings before Preview use. Add focused tests for each change.

Needs: Owner review of the cited findings, approved change windows, and access to the affected company services.

Effort: XL

Accept when: Unauthenticated and wrong role checks fail closed. Patched runtimes clear the cited flaws. Approved destinations are enforced. Preview and Production no longer share privileged records. Changed credentials have an owner and a passing service check.

OP02 - Company ownership and GitHub governance

Outcome: The company repository is the release source. Company access can be recovered without one person. Changes to `main` have a protected record.

Includes: Confirm owners, MFA, recovery, members, collaborators, installed apps, and repository need. Classify `internal-apps`. Protect `main`; block force push and delete; require pull requests and checks; limit bypass. Add ownership and pull request templates. Record emergency changes. Limit app access to needed repositories. Define access change, annual review, and release record retention. Retire personal source dependencies after the company path is proven.

Needs: Tom's approval, current GitHub access, and any plan decision needed to enforce the controls.

Effort: S

Accept when: Company recovery works. Access has a stated need. A sample protected change reaches `main` only after required checks. The personal to company source path has a clear retirement point.

OP03 - Coding agent and pull request boundary

Outcome: An AI coding agent can prepare useful changes without power to read Production secrets, merge code, or release to Production.

Includes: Use a company identity with the least useful access. Keep Production secrets out of prompts, logs, tools, and agent environments. Require a branch and pull request for every agent change. Require issue or task scope, tests, diff review, and human merge. Limit commands and external tools. Record prompt and tool handling rules. Add stop rules for data changes, auth, secrets, billing, destructive steps, and unclear scope. Pilot one low risk change.

Needs: OP02 access rules and OP04 checks.

Effort: S

Accept when: The pilot passes checks and human review. The agent cannot merge, deploy, or read Production values. Its actions can be traced to the pull request.

OP04 - Tests, CI, and security checks

Outcome: Unsafe or broken changes cannot pass the normal source gate.

Includes: Add install, lint, type or syntax, test, build, dependency, migration, and secret checks for each app. Start with access, permission, database, sync, SMS, reporting, and voice risks. Use safe test data and provider mocks. Pin approved third party actions to fixed commits where practical. Check artifacts and environment files. Make required checks block merge. Keep the suite small enough to run on each pull request.

Needs: OP01 test cases, OP02 branch rules, and approved app commands.

Effort: M

Accept when: A known bad change fails the right check. A clean change passes. Checks are required on `main`. No test needs a Production secret or live customer data.

OP05 - Vercel Preview to Production release path

Outcome: Stephanie releases the exact revision that passed checks in an isolated company Preview. Every release has a rollback point and short proof.

Includes: Create separate Preview settings, data, and provider paths. Use company owned services. Protect Preview as needed. Link pull requests to Preview. Run smoke checks for access denial, health, integration reach, and headers without changing business data. Promote only an approved revision. Record source SHA, operator, reason, checks, smoke result, observation, and rollback point. Define emergency release and redeploy rules. Classify URLs and domains. Move approved live URLs under `momentum-electric.com` when planned.

Needs: OP02, OP04, Vercel dashboard review, and approved Preview data rules.

Effort: M

Accept when: One release follows the full path. Preview cannot use Production privileged values. The released SHA matches the tested SHA. A rollback exercise works and records database limits.

OP06 - Supabase migrations, RLS, and privileged access

Outcome: Each database can be rebuilt from source. Access is narrow, tested, and clear about when broad service access is allowed.

Includes: Reconcile current schemas. Create a safe baseline and forward migration record. Test clean build and upgrade. Add drift checks. Review grants, RLS, policies, Storage, direct Postgres access, SSL, network limits, keys, and service identities. Remove unused broad Data API grants. Classify each operation as user context, service task, webhook, or admin action. Keep service role use only for named exceptions with direct app checks and audit records.

Needs: Data owner decisions, full schema evidence, app access tests, and a recovery point from OP07 before risky changes.

Effort: L

Accept when: Clean build and upgrade tests pass. Runtime allow and deny tests pass. Direct database access uses approved network and SSL rules. Each privileged path has an owner and reason.

OP07 - Backup, restore, and recovery

Outcome: Momentum knows what is backed up, how much data can be lost, how long recovery may take, and who can restore each service.

Includes: Confirm the one hour Intake and one day Production and Scheduling data loss targets or replace them. Set service recovery targets. Record database and Storage backup scope, retention, encryption, region, access, and cost. Add a Storage copy plan where needed. Run an isolated restore. Check data, files, permissions, migration state, and app startup. Write short recovery and emergency access steps. Set a repeat test schedule.

Needs: Tom, Stephanie, and the data owner to approve targets and retention. Provider access and an isolated restore place.

Effort: S

Accept when: A restore meets the approved target or records the exact gap and owner. The test covers both database and required Storage objects. A second named operator can follow the steps.

OP08 - Monitoring, incident response, and operations

Outcome: Important failures reach a named owner soon enough to act, without placing sensitive data in logs.

Includes: Define health checks and critical user paths. Add safe release IDs, error IDs, and provider request IDs. Alert on failed release, access faults, stale sync, SMS failure, voice processing failure, backup failure, and material cost change. Measure the Scheduling region issue before moving services. Give each alert a threshold, owner, response time, and short action guide. Test one alert and one incident exercise. Review noisy alerts until useful.

Needs: Approved support contacts and response times, OP05 release IDs, and OP07 backup signals.

Effort: M

Accept when: A test outage and backup failure reach the owner. A release can be traced from source to app error. Logs omit secrets, full payloads, and unneeded personal data. The action guide states who decides, what to contain, and how to verify recovery.

OP09 - Identity provider and cross project proof

Outcome: Momentum approves an identity provider only after it proves the key security and operating needs in E10.

Includes: Confirm users, roles, job access, MFA, sessions, recovery, domains, email, contractors, and audit needs. Test the preferred dedicated Supabase identity project and a fallback only if needed. Prove issuer, audience, signature, key rotation, callback, refresh, revocation, and environment separation. Prove that both business databases receive the same stable user ID through `auth.uid()`. Run allow and deny RLS tests for wrong app, environment, user, role, and record.

Needs: E10 decisions, test domains and recovery identities, and isolated Supabase access.

Effort: S

Accept when: Both business projects accept the right user and reject invalid or cross environment tokens. MFA, recovery, revocation, and key rotation meet the approved rules. Momentum signs the provider decision.

OP10 - Identity foundation, Backoffice, and app adapters

Outcome: One company control plane manages users and app access. Each application still checks its own permissions and records.

Includes: Configure identity, domains, email, keys, callbacks, MFA, sessions, rate limits, audit, and recovery. Build Backoffice sign-in, app launcher, users, invites, roles, app assignments, security settings, recovery, and admin audit. Add deny by default access records. Build tested Next.js and Python adapters. Use host only app sessions. Publish direct permission and privileged exception patterns. Add common tests for invalid, expired, revoked, wrong environment, and under privileged sessions.

Needs: OP09 approval, OP04-OP08 base controls, and final domain, email, session, MFA, and audit rules.

Effort: XL

Accept when: Admins can invite, change, suspend, recover, and remove a user with audit events. Users see only assigned apps. Both adapters pass the same deny tests. A future app has a short setup and test guide.

OP11 - Scheduling individual user integration

Outcome: Scheduling uses individual users, direct route checks, record rules, and named actors for schedule and SMS work.

Includes: Add the Next.js identity adapter. Define view, edit, employee, message, SMS, report, and admin permissions. Check each handler and object. Move normal reads and writes to user context with RLS. Keep service access only for reviewed Twilio, webhook, sync, and admin work. Record the user or service behind changes and messages. Retire Basic Authentication after cutover.

Needs: OP06, OP10, and approved Scheduling roles and record rules.

Effort: M

Accept when: Shared Basic Authentication is off. Each protected route checks its permission. Cross role and cross record tests fail. Material changes and SMS sends name the user or service.

OP12 - Production individual user integration

Outcome: Field, PM, and office users have individual access limited by app, job, record, and voice permissions.

Includes: Replace shared Field and PM sessions. Define approved job and crew access. Add user identity to needed records and RLS. Derive `entered_by` from the session. Check voice lookup, transcript access, retry, signed URL, and AI cost actions. Keep service access only for named background and admin work. Test routes, Server Actions, guessed IDs, wrong roles, and wrong records.

Needs: OP06, OP10, and decisions for field devices, jobs, sessions, voice retention, and PM review.

Effort: L

Accept when: Shared Field and PM access is off. Field users see only approved jobs and voice records. Sensitive voice actions need direct permission. New material records name a user or service.

OP13 - Intake individual user integration

Outcome: Intake users have individual app and operation permissions for office, sales, financial, settings, document, and sync work.

Includes: Add the Python identity adapter and fail closed when identity settings are missing. Define office admin, estimator and PM, sales, financial view, settings, sync, and read only permissions. Check permissions inside API operations. Add identity and assignment data where needed. Move normal access toward user context RLS. Record actors for sensitive changes. Keep only named privileged exceptions. Retire Basic Authentication after cutover.

Needs: OP06, OP10, OP01 access fixes, and approved Intake role and financial view rules.

Effort: L

Accept when: Shared Basic Authentication is off. Sensitive operations check permission directly. Financial, settings, sync, and destructive actions follow the approved matrix. Wrong role, record, API, and session tests fail.

OP14 - User cutover, credential retirement, and handoff

Outcome: Every active user uses the new path. Old shared access is removed. Operators can release, recover, restore, and remove access without the implementer.

Includes: Approve the final user matrix. Invite users and test recovery. Run app and field device checks. Announce support and fallback rules. Disable old office, Field, PM, and Basic Authentication paths. Rotate obsolete keys. Check alternate URLs for bypass. Watch sign-in, deny, provider, and app signals. Remove temporary fallbacks. Practice release rollback, identity recovery, restore, and emergency admin. Hand over short guides.

Needs: Accepted OP11-OP13 work, named owners, and an approved rollback window.

Effort: S

Accept when: All active users have approved individual access. Shared human credentials are off. Every temporary fallback has expired or been removed. Remaining risks have a named owner and written disposition.

OP15 - Intake workflow integrity and maintainability

Outcome: Intake writes are atomic, reports follow approved rules, reads do not change data, and high change areas become safer to edit.

Includes: Add transactions to compound writes. Allocate human readable IDs atomically. Limit logs and client errors. Add CSP and private cache rules. Remove writes from GET handlers. Fix duplicate null handling. Repair Sales history and approved metrics. Use shared field and date validation. Make CSRF and origin checks fail closed. Add pagination and database reports. Pin the runtime and add health checks. Split `app.py` by business area in small tested steps. Keep client feature work out of this package.

Needs: OP01, OP04, OP06, approved report and validation rules, and safe sample data.

Effort: XL

Accept when: Failure tests show all or nothing writes. Concurrent IDs do not collide. Approved Sales cases pass. GET does not write. CSRF, headers, logs, pagination, and report tests pass.

OP16 - Production workflow reliability and data integrity

Outcome: Production history, progress, totals, voice work, AI output, and retries behave in a complete and repeatable way.

Includes: Block cross job history changes and unsafe deletes. Make progress writes correctable, attributable, and safe to retry. Use complete database totals and one approved

productivity measure. Use scoped direct audio upload and durable background AI work. Set retention, deletion, source, and PM approval for recordings and output. Resume partial daily logs from saved state. Reject unsafe IDs. Add safe response headers. Split large modules behind tests. Do not redesign Daily Log or the PM view.

Needs: OP01, OP04, OP06, OP07, approved metrics and correction rules, retention and AI decisions, and isolated Storage and AI tests.

Effort: XXL; approve in finding based groups

Accept when: Retries create one accepted result. Cross job moves and unsafe deletes fail. Reports use complete data and the approved measure. Voice work can retry without duplicate output or cost. Interrupted logs resume. Retention and PM review are testable.

OP17 - Scheduling workflow, sync, and messaging

Outcome: Assignment state, dates, Intake sync, and SMS stay consistent under edits, retries, stale input, and provider failure.

Includes: Enforce assignment life cycle and job closure rules. Keep assignment edits from changing global job dates. Make saves, reorders, and deletes transactional. Check schedule rules and payload limits on the server. Add SMS request keys, provider IDs, delivery callbacks, retries, and rate controls. Make inbound webhooks safe to retry. Match one canonical URL and unique phone record. Make Intake sync unique, version aware, safe to replay, and transactional. Add pagination, retention, headers, and approved time zone handling. Split large modules in small tested steps.

Needs: OP01, OP04, OP06, approved schedule, closure, conflict, time zone, and retention rules, plus a Twilio test path or simulator.

Effort: XXL; approve in finding based groups

Accept when: Boards, capacity, reports, and messages agree on assignment state. Assignment edits keep job dates. Stale or replayed sync cannot reopen closed work. SMS is safe to retry and linked to provider state. Transaction, date, page, retention, and header tests pass.

Delivery sequence

Wave	Packages	Exit proof
0. Decide and contain	OP01 plus owner decisions in OP02, OP06, OP07	Urgent fixes pass; owners are named; credential list exists; recovery and retention targets are approved
1. Control change	OP02-OP05	Human and agent changes use protected pull requests; checks block failure; Preview is separate; one release and rollback pass
2. Recover and observe	OP06-OP08	Schemas rebuild; access is reviewed; restore works in isolation; critical alerts reach an owner
2B. Repair selected app risks	Approved groups from OP15-OP17	Each selected source ID has passing tests and a closeout decision; client and engagement edits do not conflict
3. Prove and build identity	OP09-OP10	Provider proof passes; Backoffice and adapters pass common tests; full user life cycle works
4. Integrate and cut over	OP11-OP14	Individual and record access tests pass; shared access is off; operators practice recovery

Scheduling is the recommended technical identity pilot because it has a smaller user and data surface. That does not move it ahead of the client's Intake first feature roadmap. Agree the common identity contract in OP09-OP10, then schedule app work around active client changes.

Client decisions before final price

Decision	Proposed owner	Needed before
Approve package order and first scope	Tom	OP01
Confirm service and data recovery targets	Tom and Stephanie	OP07
Set record, voice, message, and log retention	Tom, Stephanie, data owner	OP07-OP08, OP12, OP17
Approve app roles, field job access, MFA, sessions, and recovery	Tom and Stephanie	OP09
Approve Preview data and provider rules	Stephanie	OP05
Approve Sales, productivity, schedule, closure, and correction rules as needed	Business owner	OP15-OP17
Approve any paid plan or add-on	Tom	Before purchase or enablement

Confirmed people, URLs, current access, and roadmap details live in [Appendix G](#). Open decisions live in [CLIENT_QUESTIONS.md](#).

Cost choices

Choice	Cost effect	Default approach
GitHub checks	May use included minutes or create added use	Start with small required checks. Review usage before adding larger scans.
GitHub security features	Some private repository features may require a paid plan	Use built in features where available. Use a controlled local history scan if needed.
Vercel Preview and protection	Depends on current plan, users, and usage	Confirm plan limits before design. Keep Preview short lived and scoped.
Supabase Preview data	A separate project or branch can add compute, Storage, and transfer cost	Use the smallest isolated company setup that proves safe release.
Supabase backup and point in time recovery	May require a paid plan or add-on	Buy only after the recovery target and restore method are approved.
Backup copy storage	Adds Storage, request, and transfer use	Keep only required copies. Encrypt and test restore.
Twilio testing	A real test number and messages add number, carrier, and message fees	Use mocks in CI. Use a separate number only for end to end tests.
OpenAI testing	Separate keys still use paid API calls	Use mocks in CI and set project budgets and limits for approved tests.
Restore drills	Uses temporary service capacity and labor	Use an isolated short lived target, retain proof, then remove it.

Check the current vendor price and contract before approval: [Supabase pricing](#), [Vercel pricing](#), [GitHub Actions billing](#), [Twilio US SMS pricing](#), and [Cloudflare R2 pricing](#).

No package may enable a paid item until it records the owner, scope, reason, current price, expected use, budget alert, approval, and removal path.

Scope boundaries

The client owns its planned Intake and Production features. These include pricing, estimating, Intake redesign, the Project Manager module, Microsoft Graph work, more

Settings options, dashboards, contract changes, Daily Log redesign, and PM view cleanup. OP15-OP17 fix audit findings only. They do not add those features.

This proposal also excludes wholesale app rewrites, historical data correction unless separately approved, a full penetration test, round the clock support, vendor fees, legal or compliance advice, destructive Production work without a reviewed plan and recovery point, and automatic merge or Production release by an AI agent.

Finding to package crosswalk

E02 - Intake

Source	Package
E02-F01-F02	OP01 containment; OP04 tests
E02-F03	OP13 individual access; OP14 retirement
E02-F04	OP06 migrations
E02-F05	OP15 transactions
E02-F06	OP01 artifact response; OP02 and OP04 prevention
E02-F07	OP04 tests and CI
E02-F08-F12	OP15 app repair; OP08 health and signals
E02-F13-F14	OP01 containment; OP05 separation
E02-F15-F20	OP15 workflow repair; OP04 tests

E03 - Production

Source	Package
E03-F01-F02	OP01 containment; OP12 and OP14 identity
E03-F03	OP12 job and voice access
E03-F04	OP01 review; OP06 migrations
E03-F05	OP01 guard; OP16 full repair
E03-F06-F09	OP16 workflow and retention; OP07 recovery
E03-F10-F11	OP04 tests; OP16 headers
E03-F12-F13	OP16 app boundaries; OP08 signals
E03-F14-F16	OP01 containment; OP05 separation
E03-F17-F20	OP16 workflow repair; OP04 tests

E04 - Scheduling

Source	Package
E04-F01	OP01 patch; OP04 dependency check
E04-F02-F03	OP01 direct checks; OP11 and OP14 identity
E04-F04	OP01 urgent state fix; OP17 full repair
E04-F05-F07	OP17 SMS and sync; OP04 tests; OP08 signals
E04-F08	OP01 log fix; OP08 log rules
E04-F09-F12	OP17 data and workflow repair
E04-F13-F14	OP01 guard; OP06 migrations and RLS
E04-F15	OP01 containment; OP05 separation
E04-F16-F19	OP17 retention and app repair; OP04 tests; OP08 signals
E04-F20-F22	OP01 urgent fixes; OP17 lasting tests
E04-F23-F24	OP17 headers; OP01 cleanup; OP04 prevention

E05 - GitHub and release process

Source	Package
E05-F01	OP02 ownership, MFA, and recovery
E05-F02-F04	OP02 source rules; OP04 checks; OP05 release path
E05-F05	OP01 secret response; OP04 scanning
E05-F06-F07	OP02 app scope and access review
E05-F08	OP05 release proof; OP08 incident link
E05-F09	OP02 later source proof decision
E05-F10	OP04 approved and fixed Action references

E06 - Vercel

Source	Package
E06-F01	OP02 company ownership and recovery
E06-F02-F04	OP01 containment; OP05 isolated release path
E06-F05-F07	OP05 checks, rollback, and URLs; OP08 monitoring
E06-F08	OP08 measure before region decision
E06-F09	OP05 dashboard review; OP08 cost and incident controls

E07 - Supabase

Source	Package
E07-F01	OP06 migrations and drift checks
E07-F02	OP07 backup and restore
E07-F03	OP06 network and SSL controls
E07-F04	OP01 credential list; OP06 key migration
E07-F05	OP06 grant cleanup and deny tests
E07-F06	OP01 unused Auth guard; OP09-OP10 approved identity settings
E07-F07	OP01 MFA action; OP02 access review

Program completion

The full program is done when accepted packages pass their checks and:

- routine changes use protected and attributable pull requests;
- required build, test, security, migration, and release checks block failure;
- Preview is separate from Production;
- each release has source, smoke, observation, and recovery proof;
- both databases have repeatable migrations, tested access, and a shown restore;
- alerts and incident steps have tested owners;
- each active person has approved individual app and record access;
- shared human access and obsolete privileged paths are off;
- named operators can release, remove access, respond, roll back, restore, and recover; and
- every E02-E07 finding is resolved, accepted, transferred, or owned in a later approved package.

Appendix E - Shared Identity and Permission Design

Source: E10 - Shared Identity and Authorization Architecture **Status:** Client review draft; provider choice waits on OP09 proof **Last updated:** 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This appendix defines the target for individual sign-in, app access, permissions, recovery, and action history. Tom and Stephanie should use it to approve the business rules. Identity and application owners should use it to build OP09-OP14.

The source audits remain the authority for current access flaws. This design does not approve a Production change.

Decision summary

Momentum should create one company owned Backoffice portal. It will handle sign-in, users, invitations, app access, broad roles, recovery, MFA, and admin history. After sign-in, each user will see only the apps they may use.

Intake, Production, and Scheduling should stay separate. Backoffice is not the only security boundary. Each app must check the user, permission, and record again inside every sensitive server route, Server Action, data loader, or business operation.

Each app should keep its own secure session. It should use standard OAuth 2.1 and OpenID Connect. The apps should use normal user context and Supabase RLS for ordinary data access where the provider proof supports it. Broad service role access should remain only for named machine or admin tasks after the app has checked permission.

A dedicated Supabase identity project is the first provider to test because Momentum already uses Supabase. OP09 must confirm its current OAuth and OIDC maturity, support, cost, client controls, revocation, and cross project trust. If it does not meet the full contract, use another managed OIDC provider. Do not build a custom sign-in protocol.

Goals

- One identity for each person.
- One Backoffice entry point and app list.
- Direct app and record permissions.

- Removal of one person without changing access for everyone.
- Managed invitations, MFA, recovery, sessions, and offboarding.
- Action records tied to a person or service.
- Separate Development, Preview, and Production clients and settings.
- A small, repeatable setup for future apps.
- Company ownership and tested emergency recovery.

The first release will not merge repositories or frameworks, move business data into the identity project, put every record rule in Backoffice, expose service secrets to a browser, or add public customer accounts.

Current access and trust

Application	Current users	Current access	Main risk
Intake	Tom, Stephanie, Enrique	One shared Basic Authentication password	All signed in users receive broad access; missing settings can fail open
Production	All seven current users	Shared Field and PM passwords in long lived cookies	No individual, job, or voice record boundary
Scheduling	Tom, Stephanie, Enrique	One shared Basic Authentication password	All signed in users receive broad access; handlers trust middleware

All apps use privileged server access to Supabase. Scheduling can also send SMS. Intake calls Scheduling with a credential. Production uses Storage and OpenAI. A missed route check can therefore become broad database or provider access.

The future design must close E02-F02-F03, E03-F02-F03, E03-F14, E04-F02-F03, and the related E08 root causes. OP01 contains urgent risk. OP09-OP14 deliver the lasting design.

Design rules

1. Backoffice proves who the user is and which apps or broad actions they have. Each app owns current business and record rules.
2. Deny by default. A valid sign-in grants nothing by itself.
3. Check permission at each sensitive server operation. Menus, browser checks, middleware, and proxies do not replace that check.

4. Use user context and RLS for normal data access when the proof supports it. Treat service role use as a narrow exception.
5. Use managed standards: OAuth 2.1, OIDC, short lived signed tokens, secure sessions, MFA, and recovery.
6. Keep tokens small. Put user ID, issuer, audience, client, session, and broad app access in the token. Keep job and record assignments on the server.
7. Give each environment its own client, callback, secret, cookie, data, and test users.
8. Keep company control. Stephanie may be the main operator, but Tom needs a tested company recovery path.

Target design

Backoffice owns

- sign-in, logout, MFA, recovery, and security settings;
- invitations, activation, suspension, and offboarding;
- the app list shown to each user;
- broad roles and app assignments;
- identity session views where supported;
- sign-in and admin history;
- safe return URLs from a fixed app registry; and
- company branding and help.

Backoffice does not send database secrets to users, decide every job or document rule, trust roles from browser input, accept arbitrary return URLs, or replace app route checks.

Each application owns

- a registered client for each environment;
- the authorization code flow with PKCE;
- a secure host only session;
- checks for token signature, issuer, audience, client, expiry, session, and required MFA level;
- checks that the user is active and assigned to the app;
- one server helper used by every protected operation;
- local job, estimate, schedule, message, document, and record rules;
- user context Supabase access and RLS for normal work where proven;
- isolated privileged clients for named exceptions;
- actor IDs and request IDs for material actions; and
- safe errors and logs that omit tokens, cookies, and secrets.

Domains and sessions

The first release should use separate app hosts:

```
portal.<company-domain>
intake.<company-domain>
production.<company-domain>
scheduling.<company-domain>
```

Backoffice launches a fixed registered destination. If the app has no session, it redirects to the provider. The app exchanges a one time code and creates its own host only cookie.

Do not share one parent domain cookie with every app. Do not make iframe or path routing part of the first identity release. A later `apps.<company-domain>/...` view may be useful, but it adds path, asset, cookie, cache, monitoring, and failure ties across Python and Next.js.

Provider proof and decision

OP09 is a blocking proof. It must:

1. Confirm the provider's current support, maturity, price, client management, audit, revocation, key rotation, and environment model.
2. Test one Next.js client and one Python verification path.

3. Prove issuer, audience, signature, keys, callbacks, refresh, logout, suspension, and urgent removal.
4. Prove that both business Supabase projects receive the same stable user ID through `auth.uid()`.
5. Prove that an assigned record is allowed and an unassigned record is denied in both projects.
6. Prove that wrong app, environment, issuer, audience, role, record, and revoked tokens fail.
7. Record a go or no go decision and any change to OP10-OP14 effort.

The provider contract must remain standard OIDC. If the preferred provider fails, use a managed provider supported by both apps and Supabase. Do not replace the failed proof with a private Backoffice token format.

App conformance contract

Each app registration must state:

- app ID and owner;
- Production and Preview URLs;
- exact callbacks and logout URLs;
- approved audiences and token source;
- session and MFA rules;
- permission list;
- sensitive data and provider use;
- normal and privileged database paths;
- audit events, alerts, and recovery owner; and
- test and rollback contacts.

Each Python and Next.js adapter must provide the same behavior for:

```
getCurrentUser()  
requireUser()  
requireAppAccess(app)  
requirePermission(permission)  
requireRecordAccess(resource, id, action)  
getUserDataClient()  
getPrivilegedClient(operation)  
logout()
```

Adapters must fail closed. They must not silently create a privileged database client when user context fails. Each new protected route must be denied until it declares a permission and passes direct route tests.

Permission model

Use four layers:

1. **App access:** May the user open Intake, Production, or Scheduling?
2. **Operation permission:** May the user view, edit, approve, send, export, configure, or administer?
3. **Record rule:** May the user act on this job, estimate, voice log, employee, message, or document?
4. **Data rule:** Does RLS allow the same row under the verified user ID?

Store users, apps, roles, permissions, role grants, user app roles, and admin audit in the identity control plane. Keep business assignments in the app that owns them.

Use names such as:

```
intake.view
intake.sales.edit
intake.financial.view
production.progress.write
production.voice.process
scheduling.assignment.edit
scheduling.sms.send
backoffice.users.manage
```

Initial broad role bundles may include company admin, office operator, project manager, and field technician. These are starting bundles, not proof of record access. Tom and Stephanie must approve exceptions and Field job or crew rules.

Data access pattern

For normal user work:

1. Verify the app session.
2. Check app and operation permission.
3. Build a user context Supabase client with the verified user token.
4. Let RLS enforce the row rule.
5. Record the actor for material changes.

Use service role only for a named task that user context cannot perform, such as a verified provider webhook, queue worker, cross project sync, or a narrow admin action. Before using it, check the named permission, object scope, input, rate, and audit need. Record the reason and owner. Review whether the path can later move to user context.

Sessions and browser safety

- Use secure, HTTP only, same site cookies with narrow path and host scope.
- Rotate session IDs after sign-in and privilege changes.
- Set idle and maximum life. Define behavior for provider outage and clock error.
- Use exact callback and return URL lists.
- Use CSRF protection for cookie based writes.
- Set CSP, frame, content type, referrer, and private no store controls.
- Do not put tokens in browser storage, URLs, logs, or error reports.
- Recheck high risk permissions when a role change must apply faster than token life.

Field device rules must be strict enough to protect data and simple enough to avoid account sharing. Approve the final MFA, idle time, maximum time, and recovery rules after device testing.

User and access flows

Event	Required flow
Invite	Admin selects email, apps, and role. User accepts through a short lived link and completes required MFA.
Sign-in	User signs in centrally, sees allowed apps, launches one, and receives a local app session.
Role change	Admin changes access. Sensitive permissions update at once or within the approved short cache time. Existing sessions are revoked when needed.
Recovery	Managed provider recovery uses a verified company channel. High risk recovery creates an alert and audit event.
Offboarding	Suspend user, revoke sessions, remove app grants, transfer owned work if needed, and confirm denial.
Emergency access	Use a named company recovery path. Record use and review it after the event.

Audit records

Keep three related records:

- **Sign-in record:** success, failure, MFA, recovery, session, and provider event.
- **Admin record:** invite, role, app access, suspension, recovery, and settings changes.
- **Business action record:** actor, app, permission, action, target ID, result, time, and request ID.

Do not copy secret values or full sensitive records into audit logs. A sign-in record does not prove who changed an estimate or sent a message. Each app must record those actions.

Environments and service identities

Development, Preview, and Production need separate company controlled clients, exact callbacks, secrets, cookies, data, users where practical, and independent revocation.

Preview must not use Production service role keys or general customer data.

The personal test path must be retired after the company path has equal source, checks, Preview, release, and recovery proof.

Keep machines separate from people:

Integration	Identity and check
Intake to Scheduling	Named service identity with one audience and scope
Twilio inbound webhook	Provider signature plus time and replay checks
Scheduling outbound SMS	User permission plus server held Twilio key
OpenAI work	User and record permission or named queue identity plus server held key
Scheduled work	Narrow service identity with a clear job name and action record

Migration

0. Contain

Close fail open access, stop password cookies, add direct handler checks, patch runtimes, separate Preview, and prove unauthenticated denial through OP01.

1. Decide and prove

Approve users, app access, Field record rules, MFA, sessions, recovery, domains, and provider choice. Complete OP09 in both business databases.

2. Build the base

Create the company identity environment and Backoffice. Configure email, keys, callbacks, limits, audit, and recovery. Build both adapters and common deny tests. Create the operation and privileged path list.

3. Pilot Scheduling

Replace Basic Authentication. Add direct route checks, user context RLS, named service exceptions, actor records, and view, edit, employee, message, SMS, report, and admin permissions.

4. Integrate Production

Replace Field and PM passwords. Add job and crew rules, user IDs, RLS, direct voice checks, actor records, and tests for every route, method, role, and cross record ID.

5. Integrate Intake

Replace Basic Authentication. Fail closed on missing settings. Add office, PM, sales, finance, settings, sync, and read only permissions inside API operations. Move normal data use toward user context RLS.

6. Cut over

Invite users, test devices and recovery, disable shared paths, rotate old keys, check alternate URLs, monitor failures, remove temporary fallbacks, and complete a final access review.

Scheduling is the recommended adapter pilot because its first access model is smaller. This does not change the client's Intake first feature priority.

Rollback

Retain the last known good app release for a short cutover window. Keep schema changes compatible where practical. Name the rollback owner and end time. Test how new actor IDs behave in the old app. Any temporary shared fallback must have an owner, expiry, and removal proof. Rollback must not make shared access the permanent design again.

Validation

Test at least:

- invite, duplicate and expired invite, sign-in, wrong password, MFA setup and recovery;
- refresh, idle and maximum time, logout, global removal, and suspended user;
- key rotation, cached keys, exact callbacks, and cross environment denial;
- no session, bad session, wrong issuer, audience, client, role, MFA level, app, and record;
- direct route and Server Action calls without visiting the screen;
- permission removal during a session and new route deny by default;
- cross job, voice, message, and guessed record IDs;
- positive and negative `auth.uid()` and RLS results in both databases;
- proof that normal code cannot fall back to a privileged client;
- provider webhook and service identities denied from human actions;
- provider or Backoffice outage, alert delivery, audit links, and non Production cutover and rollback.

Key risks

Risk	Response
Provider feature maturity or change	Complete OP09 and keep a provider neutral OIDC contract
Backoffice outage	Define existing session behavior, monitoring, recovery, and emergency admin
Roles become too detailed	Keep broad app permission central and record rules local
Apps trust middleware	Require direct helper use and direct route tests
Cross project trust fails	Stop OP10 and choose a provider supported by both projects
Service role stays broad	List every path, require direct checks, and reduce paths over time
Shared parent cookie exposes several apps	Use host only app sessions
Removed access lasts too long	Use short token life, live checks for sensitive work, and session removal
Field users share accounts	Test real devices and choose usable session rules
Audit logs collect private content	Store IDs and small change facts; set access and retention

Acceptance

The design is ready for implementation when:

- Momentum approves the provider after OP09;
- both databases trust the same stable user ID and pass allow and deny RLS tests;
- Tom and Stephanie approve users, app roles, Field record rules, MFA, sessions, and recovery;
- Backoffice and app responsibilities are clear;
- both adapters share the same fail closed contract and tests;
- every normal and privileged data path is classified;
- Production and Preview clients, settings, data, and callbacks are separate;
- migration, rollback, shared access retirement, audit, and outage behavior are approved; and
- E11 package scope and effort still match the chosen provider.

References

- [E01 - Discovery](#)
- [E08 - Consolidated Finding Register](#)
- [E11 - Implementation Roadmap](#)
- [Supabase Auth](#)
- [Supabase OAuth server](#)
- [Supabase signing keys](#)
- [Supabase MFA](#)
- [Supabase Row Level Security](#)
- [Next.js authentication guidance](#)

Appendix F - Operating Playbook

Source: E09 - Practical Engineering, Security, and Operations Practices **Status:** Client review draft **Last updated:** 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This is the short routine for changing and operating Momentum's applications. Stephanie, future developers, release operators, and support owners should use it after E11 puts the required controls in place.

The [code organization appendix](#) gives app specific examples. E11 states which work package creates each control.

For every change

- Start with one small task and clear acceptance checks.
- Name the app, business rule, data, integration, and risk that may change.
- Use a branch and pull request. Link the task or finding ID.
- Do not place Production secrets, customer data, voice, messages, or database copies in prompts, tests, logs, or tickets.
- Add or update tests for the changed behavior. Test both allowed and denied access when permissions matter.
- Require install, lint, type or syntax, test, build, dependency, migration, and secret checks that apply to the app.
- Review the diff, generated files, lockfile, migrations, settings, logs, and failure paths.
- Test the exact revision in isolated company Preview with safe data and separate credentials.
- Record any migration, provider change, cost change, rollback limit, or operator step.
- Let Stephanie promote the tested revision after required checks pass.
- Run short smoke checks. Watch errors and key services. Keep the rollback point.
- Update the closest useful code or operating guide when behavior or ownership changed.

Stop and ask for approval

Stop before any change that would:

- alter Production data or run a destructive migration;
- rotate a secret, remove access, or disable a working path;

- change sign-in, permissions, RLS, Storage access, or recovery;
- enable a paid plan, service, number, domain, or add-on;
- send real SMS, email, or AI requests outside an approved test; or
- conflict with active client work in the same route, module, schema, or service.

Tom decides material business risk, downtime, destructive data work, and paid changes. Stephanie owns the routine release and coordinates overlapping work.

Release checklist

Check	Required proof
Source	Pull request, approved scope, source SHA, and required checks
Preview	Separate settings and safe data; exact SHA; acceptance result
Database	Reviewed migration; recovery point; clean build or upgrade check
Security	Access allow and deny tests; secret and dependency checks
Release	Operator, reason, time, target, and rollback point
Smoke	Access denial, health, key read path, and safe integration reach
Observe	Error and provider signals watched for the approved period
Close	Result, follow up, and docs recorded

An emergency release still needs an owner, reason, source record, focused checks, rollback point, and later review. Emergency does not mean unrecorded.

Access and secrets

- Give each person an individual account. Grant only the apps, operations, and records they need.
- Use MFA for platform and admin access where supported.
- Keep Tom's company recovery method tested. Do not depend on one personal account.
- Keep human accounts, app secrets, service identities, webhooks, release credentials, and emergency access separate.
- Store secret values only in approved secret stores. Record owner, purpose, scope, environment, rotation trigger, and last check without recording the value.
- Remove access when need changes. Run one short annual review of users, admins, services, apps, recovery contacts, and billing owners.
- Use broad service access only for a named machine or admin task. Add a direct app permission check and audit record.

- Never treat a hidden URL, shared password, middleware check, or client side check as the only permission boundary.

Data, database, and recovery

- Put every database change in the approved migration history.
- Test a clean database build and an upgrade from the last live state.
- Make compound writes atomic. Make retries safe when a request can repeat.
- Enforce rules on the server and in RLS where practical.
- Keep direct database access narrow and encrypted.
- Set record specific retention and deletion rules. A request for indefinite retention is not enough.
- Back up databases and required Storage objects. Limit backup access and keep retention clear.
- Run an isolated restore on the approved schedule. Check data, files, permissions, migration state, and app startup.
- Keep a short recovery guide that a second named operator can follow.

Logs, alerts, and incidents

- Log stable error, release, request, and provider IDs. Do not log secrets, full payloads, raw voice, or unneeded personal data.
- Give each alert a clear condition, owner, response time, and action.
- Watch failed releases, sign-in faults, denied access spikes, stale sync, SMS failure, voice processing failure, backup failure, and large cost changes.
- When an incident occurs: name the lead, contain the issue, protect evidence, tell the right people, recover, verify, and record follow up.
- Test one alert route and one incident exercise. Fix noisy or useless alerts.

Weekly routine

- Review failed checks, releases, alerts, backup status, provider errors, and open urgent findings.
- Review work planned for the next week and mark any conflict between client and engagement changes.
- Check unusual spend or use in Vercel, Supabase, Twilio, OpenAI, and other paid services.
- Confirm that temporary access, test services, and fallbacks still have an owner and end date.

Annual and event based routine

Run a review at least once a year and whenever a person, vendor, app, domain, billing owner, or recovery contact changes:

- confirm company ownership and recovery;
- confirm users, admins, service identities, installed apps, keys, and need;
- remove old access and rotate affected secrets;
- test one restore, one release rollback, one identity recovery, and one alert path;
- review retention, backup scope, domains, vendor plans, and cost alerts; and
- update the owner and date in the short operating guides.

Audit evidence routine

- Use one current platform inventory as the source for each related report.
- Record the source time and sample size in the report.
- Compare report counts with the source before review.
- Map every source finding or record why it is left out.
- Stop when the source finding count and the main register do not match.
- Keep files named latest current. Give older evidence a date.
- Keep evidence files private to the approved owner and location.
- Test report coverage, counts, source times, and file permissions.

AI coding rules

- Give the agent only the repository and tools needed for the task.
- Keep Production credentials and customer data out of its reach.
- Require a branch, pull request, tests, and a human diff review.
- Do not let the agent merge or release to Production.
- Stop on unclear scope, destructive work, data correction, access changes, secrets, billing, or a conflict with active client work.
- Treat generated code like any other code. Its source does not lower the test or review standard.

Definition of done

A routine change is done when the acceptance checks pass, required source and Preview checks pass, release and rollback proof exists, logs and alerts remain safe, the operator accepts the result, and the closest useful guide matches the new behavior.

Appendix F1- Code Organization Guide

Source: E09 Practical Engineering, Security, and Operations Practices **Status:** Client review draft **Last updated:** 2026-07-31 **Entry document:** [Operating Playbook](#)

Purpose

This guide shows how to make Intake, Production, and Scheduling easier to change. It does not call for a rewrite or a new framework. Use small tested steps as part of OP15-OP17.

Before moving code, add tests that capture the approved current behavior. An urgent security fix does not need to wait for broad test coverage. Add the smallest useful test with that fix, then fill wider gaps through OP04.

Review basis

Application	Reviewed revision	Main stack	Current checks
Intake	180bce61e85c	Python standard library, browser JavaScript, Supabase REST	Python and browser syntax pass; no app test suite
Production	8377f25f57c0	Next.js, React, TypeScript, Supabase	ESLint and strict TypeScript pass; no app test suite
Scheduling	31b11c8b0c22	Next.js, React, TypeScript, Supabase, FullCalendar, Twilio	ESLint and strict TypeScript pass; no app test suite

All three apps have clear business ideas and useful parts. Their main code risk is concentration. A few large files mix screen state, validation, business rules, database work, providers, and error handling. Small changes then need broad context and can affect other workflows.

The right response is gradual separation backed by tests.

Intake

`app.py` has about 5,116 lines and 262 functions. It handles routing, access, HTTP, Supabase, validation, Intake, Sales, estimates, contracts, reports, Scheduling sync, and static files. Browser scripts also repeat request, message, format, and HTML safety helpers.

Keep the current Python and browser approach. Move one stable area at a time:

```
intake_app/  
  app.py                # Small WSGI setup and router  
  intake/  
    config.py  
    web/                # Requests, responses, security, static files  
    infrastructure/     # Supabase and Scheduling clients  
    domains/            # Intake, sales, estimates, contracts, reports  
  static/  
    shared/             # Requests, safe DOM work, formatting  
    intake/  
    sales/  
    estimating/  
    reporting/  
  tests/  
    unit/  
    integration/
```

Start with HTTP and access behavior, provider clients, pure money and date rules, then the business area under active work. Do not move every function at once.

Production

Production already has strict TypeScript, ESLint, App Router boundaries, response helpers, types, and parsers. The main pressure is in `ProjectManagerSetup.tsx` at about 1,143 lines, `FieldProgressApp.tsx` at about 786 lines, and `globals.css` at about 1,428 lines.

The large client files mix loading, form state, calculations, recording, writes, errors, and rendering. Some pages and routes also query Supabase directly.

Use this direction:

```
production_app/  
  app/                  # Route entries and page composition  
  features/  
    field-progress/  
    pm-setup/  
    daily-logs/  
    reports/  
  server/  
    auth/  
    repositories/
```

```
services/  
integrations/  
shared/  
validation/  
errors/  
types/  
ui/  
styles/  
tests/
```

First separate field progress from voice and daily log state. Split PM area edits, work item edits, ordering, and deletes. Move productivity math out of React. Put database queries behind focused server functions. Move styles only when a screen is already changing.

Scheduling

Scheduling has the best early separation. Browser pages use an API client. `lib/server` holds server database work. Shared types, validation, date, report, and SMS modules already exist.

The Scheduling Board is about 2,290 lines. Reports is about 850 lines. The external job route is about 553 lines. The schedule SMS route is about 466 lines. These files still mix many business and provider concerns.

Extend the current pattern:

```
scheduling_app/  
  app/                                # Route entries and page composition  
  features/  
    schedule-board/  
    assignments/  
    absences/  
    jobs/  
    employees/  
    reports/  
    messaging/  
  server/  
    auth/  
    repositories/  
    services/  
    integrations/  
      intake/  
      twilio/
```

```
shared/  
  validation/  
  errors/  
  types/  
  ui/  
  tests/
```

Start with pure schedule, status, hours, date, conflict, and message rules. Then move board loading and writes into hooks, split focused panels, make the server own report and message rules, and place Intake and Twilio behind small adapters.

Shared code rules

Organize around business work

Keep code together when it changes for the same business reason. Prefer names such as `sales`, `daily-logs`, `schedule-board`, and `reports`. Avoid dumping unrelated work into `helpers` or `utils`.

Move code to `shared` only after more than one feature uses the same stable rule.

Keep one direction

Page or route

```
-> feature controller or component  
  -> service  
    -> repository or provider adapter  
      -> Supabase, OpenAI, Twilio, or another service
```

Business rules should not depend on React, HTTP request objects, Supabase clients, or provider response shapes. Browser code must not import server modules. Routes should check access, parse input, call a service, and format the result.

Intake can use plain Python modules and functions. It does not need a class or dependency injection framework.

Keep screens focused

Split a component when a section has its own state, writes, validation, life cycle, or business name. Put pure calculations outside render code. Put remote loading and writes in a focused hook or controller.

Do not enforce a fixed line limit. Ask whether the file has more than one reason to change.

Check every boundary

Treat these values as unknown until checked:

- path, query, JSON, and form input;
- environment settings;
- database rows that may drift;
- Intake to Scheduling data;
- Twilio and OpenAI replies; and
- imported files.

Use one contract per workflow. Browser checks help the user, but the server owns the rule. Return stable error codes and safe messages. Keep private detail in redacted server logs.

Hide database and provider details

Screens should not know table query details. Routes should not repeat provider access, URL, timeout, retry, or response rules. Use small named modules such as `ScheduleRepository`, `DailyLogRepository`, `TwilioClient`, or `OpenAITranscriber`.

Avoid a generic repository framework. Write only the interface the business work needs.

Give each rule one owner

Use one approved implementation for estimates, margins, productivity, assignment state, remaining hours, conflicts, report totals, SMS content, and sync version rules. Prefer server owned results for sensitive or often changed math.

Small risk based test suite

Start with behavior that can expose data, alter history, send messages, charge providers, or report wrong totals. Do not chase a coverage percentage.

Level	Purpose
Unit	Math, parsing, validation, dates, state, formatting, and redaction
Service and route	Access, workflow rules, database and provider faults, retries, and response mapping
Smoke	One key workflow per app in isolated company Preview

First tests:

App	Unit and service focus	Smoke path
report-map		

App	Unit and service focus	Smoke path
Production	Route and Server Action access, record limits, totals, retries, history, daily logs, voice faults, IDs, migrations, and RLS	Field entry through PM review
Scheduling	Route access, assignment state, dates, reports, sync replay, SMS retry, webhooks, time zone, migrations, and RLS	Edit a schedule and review the employee message

Suggested tools are `pytest` and `Ruff` for Intake, and `Vitest` plus a small number of Playwright smoke tests for the Next.js apps. Tool choice may change. Each repository should still expose simple `lint`, `typecheck` where useful, `test`, and `check` commands.

Required pull request checks

App	Minimum checks
Intake	Reproducible setup, Python and browser lint and syntax, app tests, migration checks, artifact and secret scan, runtime and dependency review
Production	Clean lockfile install, ESLint, TypeScript, tests, Production build, migration checks, artifact and secret scan, dependency review
Scheduling	Clean lockfile install, ESLint, TypeScript, tests, Production build, migration checks, artifact and secret scan, dependency review

Keep local `check` aligned with CI. Use committed lockfiles. Avoid open ended `latest` dependencies. Let tools handle formatting.

Errors and docs

Use a stable public error code, safe user message, request ID, workflow name, actor when known, and redacted fields. Log unexpected faults once at the boundary that handles them. Tests should prove that logs and replies omit credentials, provider bodies, customer data, voice text, phone numbers, and financial payloads.

Each repository README should state local setup, setting names without values, main code boundaries, migration steps, verification commands, and release entry point. Write a short architecture decision only for a choice that is hard to reverse.

Adoption order

1. Add standard checks, test helpers, and a few high risk behavior tests.
2. Extract pure rules, settings, errors, database access, and provider clients.
3. Split the large active workflows in small pull requests.
4. Add another smoke path only when a serious workflow or past failure justifies it.

Do not rewrite all apps, combine repositories only for symmetry, introduce microservices, demand full coverage, build a generic framework, or move every file in one pull request.

Future **AGENTS.md**

Each repository can later add a short **AGENTS.md** with its setup commands, sensitive areas, app boundaries, and required checks. It should say that an agent may prepare code, tests, docs, and a pull request. It may not receive Production secrets or customer data, merge, change source protection, run a Production migration, or release to Production.

The key source findings are E02-F07-F08, E03-F10 and E03-F12, and E04-F17-F18. OP04 owns the test base. OP15-OP17 own app changes. OP03 owns the coding agent boundary.

Appendix G - Scope, Decisions, and Method

Source: E01 - Discovery and System Inventory **Status:** Client review draft **Last updated:** 2026-07-31 **Entry document:** [Final Engagement Report](#)

Purpose and reader

This appendix is the authority for engagement scope, people, systems, ownership, data, client plans, and open decisions. Other reports should link here instead of repeating this context.

Tom, Stephanie, and report reviewers should read it when a finding depends on who owns a service, who uses an app, which URL is live, what data matters, or which work belongs to the client.

Engagement scope

The review covers three internal applications and their company GitHub, Vercel, and Supabase services. It also covers release practice, security, recovery, operations, and a future individual identity design.

The work was mainly read only. It used source review, safe deployed checks, redacted platform inventory, and client answers. It did not change Production, run destructive tests, copy secret values, or collect customer records.

Application inventory

Purpose and runtime

Application	Purpose	Runtime
Intake	Office intake, sales, estimates, projects, contracts, and documents	Python/WSGI on Vercel
Production	Field progress, PM reports, voice logs, transcripts, and AI summaries	Next.js on Vercel
Scheduling	Jobs, assignments, availability, Intake sync, and SMS	Next.js on Vercel

Data and service dependencies

Application	Main data and services
Intake	Shared Intake and Production Supabase project; Scheduling sync; optional Google Maps
Production	Shared Intake and Production Supabase project; Supabase Storage; OpenAI
Scheduling	Separate Supabase project; Twilio

Current URLs

Application	Live URL	Reported test URL
Intake	<code>intake-app-eight.vercel.app</code>	<code>intake-app-silk.vercel.app</code>
Production	<code>production-app-rho.vercel.app</code>	<code>production-app-pink.vercel.app</code>
Scheduling	<code>scheduling-app-eta-two.vercel.app</code>	<code>scheduling-board-three.vercel.app</code>

The reported test URLs and personal test services belong to Stephanie. The collected company evidence does not prove their source, data, settings, checks, or exact promotion path. They are temporary inputs, not the target company Preview design.

People and authority

Person	Company role	Current app need	Engagement role
Tom	Company owner	Intake, Production, Scheduling	Accountable business owner; decides material risk, downtime, destructive data work, and paid changes
Stephanie	Controller and application developer	Intake, Production, Scheduling	Main operator, release operator, and engagement lead; coordinates work with Tom copied
Enrique	Project Manager	Intake, Production, Scheduling	Business user; exact permissions need approval
Donis	Field Technician	Production	Field user; job and record access needs approval
Hector	Field Technician	Production	Field user; job and record access needs approval
Jose	Field Technician	Production	Field user; job and record access needs approval
Oscar	Field Technician	Production	Field user; job and record access needs approval

These seven people are the current roster. Shared app passwords do not provide individual identity or action history. Future roles must be based on app, operation, and record need, not only job title.

Ownership and current release practice

- GitHub, Vercel, and Supabase are company resources administered by Stephanie and Tom.
- Twilio, OpenAI, Google Maps, domains, and related billing are company resources administered by Stephanie.
- Stephanie's test accounts are personal. They must not remain a final source, release, operating, or recovery dependency.
- Stephanie reports that she reviews changes in testing for function and apparent security, then pushes them live.
- Stephanie and Tom discuss material live changes. They are the current outage and data loss contacts.
- Company Vercel Production tracks each company repository's `main` branch. The reviewed current revisions match.

- The review did not find a required company pull request, automated check, or isolated company Preview gate for the reviewed period. E05 and E06 retain the exact evidence and limits.
- Momentum wants the live app URLs moved under `momentum-electric.com` in a later controlled change.

Data and service boundaries

The applications may hold or process:

- customer names, contacts, addresses, and job details;
- contracts, estimates, pricing, sales, financial, and project data;
- employee contacts, availability, assignments, progress, and productivity data;
- voice recordings, transcripts, and AI summaries;
- documents and generated links; and
- incoming and outgoing schedule messages.

Intake and Production share one Supabase project. Scheduling uses a separate Supabase project. Production also uses Supabase Storage and OpenAI. Scheduling uses Twilio. Intake sends data to Scheduling and may use Google Maps.

The current direction is indefinite retention, but that is not a usable record policy. Momentum still needs record specific retention, deletion, access, backup, and legal guidance from the right owners or advisers.

Continuity and recovery

The preliminary data loss targets are:

System	Preliminary tolerated data loss
Intake	One hour
Production	One day
Scheduling	One day

Momentum could work without the apps for a few days only with significant difficulty. Exact service recovery targets are not approved. Backup availability, Storage coverage, restore steps, second operator ability, and emergency vendor recovery still need proof. E07 records the current evidence. OP07 turns the final targets into a restore test.

Client owned roadmap

The client's first priority is Intake. Production is second. No fixed deadline or active service failure was reported.

Client owned Intake work includes:

- time and material pricing and wider estimating;
- Sales Intake and Project Intake redesign;
- a job centered Project Manager area with change orders, RFIs, and submittals;
- later Microsoft Graph email, calendar, and OneDrive work;
- more Settings items and individual dashboards after individual sign-in; and
- contract changes.

Client owned Production work includes a simpler Daily Log and Project Manager view cleanup.

These are not E11 deliverables. Audit repair may run in parallel when it touches different files, routes, schemas, or services. When work overlaps, Stephanie and the implementation owner must set one baseline, order the changes, and freeze only the smallest conflicting area.

Confirmed positive controls

- The company application repositories are private.
- The live projects link to the expected company repositories.
- Current live revisions match the reviewed source revisions.
- Supabase Row Level Security is enabled on the reviewed business tables.
- The deployed Intake schema includes the expected final effects of its local migrations.
- Vercel protects generated deployment URLs and marks environment values sensitive.
- Stephanie performs a manual test before release and discusses material changes with Tom.

Each source audit states the exact limit of these controls. A control seen in one app or one service should not be treated as a control everywhere.

Open decisions

Decision	Proposed owner	Needed before
Final app roles and sensitive operation access	Tom and Stephanie	OP09-OP13
Field job, crew, and voice record visibility	Tom and Stephanie	OP12
Exact service recovery targets	Tom and Stephanie	OP07
Record, voice, message, document, and log retention	Tom, Stephanie, and data owner	OP07-OP08, OP12, OP17
Preview data, providers, email, and SMS rules	Stephanie	OP05
MFA, session, recovery, and device rules	Tom and Stephanie	OP09
Sales, productivity, schedule, closure, and correction rules	Named business owner	OP15-OP17 as needed
Paid plan and add-on limits	Tom	Before enablement

Use [Client Decisions and Verification Register](#) for the short list of unresolved questions and the evidence needed to close them.

Assessment method

The review used:

- source and dependency review at recorded repository revisions;
- safe requests to selected deployed routes;
- read only GitHub, Vercel, and Supabase inventory with values and customer data excluded;
- client answers about people, ownership, release work, recovery, and priorities; and
- cross checks between source, platform records, and live revisions.

Detailed commands, evidence hashes, collection paths, and completion checklists are internal working material. They should not appear as normal client narrative in the final Word document.

Limits

This was a point in time review, not a full penetration test or guarantee that every issue was found. Personal testing services were not inventoried. E05 still needs full history and owner checks. E06 still needs dashboard checks. E07 still needs broader database and restore tests. No destructive Production test or approved restore was run.

Client Decisions and Verification Register

Source: CLIENT-DECISIONS - Client Decisions and Verification Register **Status:** Client review draft **Last updated:** 2026-07-31 **Entry document:** [Final Engagement Report](#)
Canonical context: [Appendix G - Scope, Decisions, and Method](#)

Purpose

This register contains only unresolved client decisions and the evidence needed to close them. Confirmed people, ownership, URLs, app use, data, recovery inputs, and roadmap facts live in Appendix G.

Do not present every row as one large questionnaire. Ask a question when the related package is ready and include a proposed default.

Proposal decisions

First approved package group

- **Owner:** Tom
- **Proposed default:** Approve OP01-OP08 in stages, starting with OP01-OP02.
- **Needed before:** Implementation start.

Service recovery targets

- **Owner:** Tom and Stephanie
- **Proposed default:** Confirm one hour data loss for Intake and one day for Production and Scheduling; set exact service recovery times.
- **Needed before:** OP07.

Record retention

- **Owner:** Tom, Stephanie, and data owner
- **Proposed default:** Define rules by record type instead of one indefinite rule.
- **Needed before:** OP07-OP08, OP12, and OP17.

Paid plan and add-on limit

- **Owner:** Tom

- **Proposed default:** No paid item without a recorded price, expected use, budget alert, and removal path.
- **Needed before:** Any purchase or enablement.

Client and engagement work order

- **Owner:** Stephanie, with Tom copied
- **Proposed default:** Allow separate areas to move in parallel; freeze only the smallest conflict.
- **Needed before:** Each OP15-OP17 group.

Identity decisions

Application roles

- **Owner:** Tom and Stephanie
- **Proposed default:** Company admin, office operator, project manager, and field technician as starting bundles.
- **Needed before:** OP09.

Field visibility

- **Owner:** Tom and Stephanie
- **Proposed default:** Field users see only assigned jobs, records, and voice logs.
- **Needed before:** OP09 and OP12.

MFA

- **Owner:** Tom and Stephanie
- **Proposed default:** Require MFA for admins and platform users; test a usable rule for field devices.
- **Needed before:** OP09.

Session life

- **Owner:** Tom and Stephanie
- **Proposed default:** Use host only app sessions with idle and maximum limits, with faster review for sensitive role changes.
- **Needed before:** OP09.

Invite and recovery channels

- **Owner:** Tom and Stephanie

- **Proposed default:** Use company controlled email and provider recovery; Tom has tested emergency recovery.
- **Needed before:** OP09.

Identity provider

- **Owner:** Tom approves; identity owner recommends.
- **Proposed default:** Use the preferred Supabase option only if OP09 proves the full E10 contract.
- **Needed before:** OP10.

Audit retention and employee notice

- **Owner:** Tom, Stephanie, and data owner
- **Proposed default:** Keep small event records for the approved period; do not copy sensitive content.
- **Needed before:** OP10.

Application rule decisions

Ask these only when the related finding group is approved.

Intake Sales

- **Decision:** Approved cohort, source, outcome history, and metric rules.
- **Owner:** Named Sales owner
- **Package:** OP15

Intake data

- **Decision:** Date order, required fields, document links, and correction rules.
- **Owner:** Intake business owner
- **Package:** OP15

Production

- **Decision:** Productivity unit, correction, history, and complete total rules.
- **Owner:** Production business owner
- **Package:** OP16

Voice and AI

- **Decision:** Recording, transcript, summary, deletion, retry, and PM approval rules.

- **Owner:** Tom and Production owner
- **Packages:** OP12 and OP16

Scheduling

- **Decision:** Assignment state, job closure owner, conflicts, dates, and time zone.
- **Owner:** Scheduling business owner
- **Package:** OP17

SMS

- **Decision:** Send, retry, delivery, retention, and employee contact rules.
- **Owner:** Scheduling owner
- **Package:** OP17

Engagement verification

These are review tasks, not client policy questions.

GitHub governance and release evidence

- **Evidence to close:** MFA, recovery, access need, app scope, full history, and release and redeploy reasons.
- **Source owner:** GitHub owner
- **Updates:** E05, E08, and E11

Vercel operations evidence

- **Evidence to close:** Owners, tokens, firewall, bypass, spend, logs, URLs, release proof, and rollback.
- **Source owner:** Vercel owner
- **Updates:** E06, E08, and E11

Supabase access and recovery evidence

- **Evidence to close:** Full database access, RLS, key custody, backup entitlement, Storage scope, and restore result.
- **Source owner:** Supabase and data owners
- **Updates:** E07, E08, and E11

Personal test and promotion evidence

- **Evidence to close:** Personal test source, data, settings, checks, and exact company promotion path.
- **Source owner:** Stephanie
- **Updates:** E01, E05, E06, and E11

Identity provider evidence

- **Evidence to close:** Provider proof, cross project user ID, and positive and negative RLS tests.
- **Source owner:** Identity owner
- **Updates:** E10 and E11

Closure rule

A row closes when its owner approves a clear answer or evidence proves the result. Update the canonical source document first, then update E08, E10, or E11 only where that answer changes scope, rating, order, or acceptance.